Hill Climb: technical report and practical guide

Contents

Hill Climb: technical report and practical guide **4**
Contents . . . 4
Executive summary . . . 4
How to read the numbers . . . 5
Building the PDF . . . 6

1. Task, rubric v4, scoring and the station **6**
1.1 The event . . . 6
1.2 Rubric v4 (abridged) . . . 6
1.3 Station hardware . . . 7
1.4 Control stack (Dream-1 on the station) . . . 7
1.4.1 Station HTTP API . . . 8
1.4.2 Timing facts that shape the harness . . . 8

2. Repositories, branches and lineage **8**
2.1 The three repositories . . . 9
2.2 hill-climb-harness: every remote branch . . . 9
2.3 Pull requests . . . 12
2.4 Lineage . . . 12
2.4.1 The three families, in words . . . 13
2.4.2 What the final main does and does not contain . . . 14
2.4.3 Branches named in the brief that do not exist on GitHub . . . 14
2.5 task_simulation and follower_code lineage . . . 15

3. Harness architecture, end to end **15**
3.1 Principles . . . 15
3.2 Component map . . . 15
3.3 A match, step by step . . . 16
3.3.1 Preflight and the guarded `/reset` . . . 17
3.3.2 Faults from Dream . . . 17
3.3.3 The stage loop (`_watch`, `runner.py:572-837`) . . . 18
3.3.4 Stage results and retries (`_run_stage`, `runner.py:1305-1411`) . . . 18
3.4 Two-look confirmation and the done logic . . . 18
3.4.1 The rule . . . 18
3.4.2 Lock, then confirm (`stop_on_pass`, `_lock_success`, `_confirm_check`) . . . 18
3.4.3 Timeouts at four levels . . . 19
3.5 Prompts: from monitor-written text to the prompt engine . . . 19
3.5.1 Prompt engine details (`harness/prompt_engine.py`) . . . 19
3.5.2 The byte-for-byte rule and the safety sentence . . . 20
3.5.3 Mid-grasp rule . . . 20
3.6 Skills: grip assist, pull, drop, lock . . . 20
3.7 Keeping the policy fed: pacing, light frames, hedging, pipelining . . . 21
3.8 The 10 Hz live guard . . . 22
3.9 The start-pose gate (side branches only) . . . 22
3.10 Shadow-mode decisions (side branches only) . . . 22

3.11 Observers and integrations . . . 22
3.12 Logs . . . 23

4. The decision model **23**
4.1 Runtime model of a stage . . . 23
4.2 Done evidence as recursive log-odds . . . 23
4.3 The fail hazard . . . 24
4.4 Choosing an action: the Q-value and the value of time left . . . 24
4.4.1 The optimal-timeout condition . . . 24
4.4.2 Fitted curves (simulation only) . . . 25
4.5 The pass-bar constraint: the α^k bound . . . 25
4.6 Tuning objective (planned, not run) . . . 25
4.7 State-space form (`harness/statespace.py`, `STATE_SPACE.md`) . . . 26
4.8 What shadow mode showed . . . 26

5. Every classifier and estimator **26**
5.0 Overview . . . 26
5.1 VLM judges . . . 27
5.1.1 What a look sends and receives . . . 27
5.1.2 Scheduling . . . 27
5.1.3 The judge benchmark (`tools/bench_judges.py`, `hedged-calls` → cloud-sims) . . . 28
5.1.4 Judges in the stress sim . . . 29
5.2 The HSV colour and shape detector (`harness/detect.py`, `plans/table_colors.yaml`) . . . 29
5.2.1 Algorithm . . . 29
5.2.2 Station retune (branch `fix/classifiers-8nov`, PR 7) [D] . . . 29
5.2.3 The white ball on the wrist cameras (`fix/guard-real-frames`, dcd76c5) [D] . . . 30
5.2.4 Our check: main's live guard on the committed real frames [V] . . . 30
5.2.5 Speed (mac-arm64) [D] . . . 30
5.3 Cable-end classifier (`harness/cable_end.py`, `models/cable_end.onnx`) . . . 30
5.4 Gripper-width rules . . . 31
5.5 Joint-fault detectors (cloud-sims family; encoders on `joint-fault-encoder`) . . . 31
5.5.1 Simulated motion with torque (cloud-sims, a73b628) [D] . . . 32
5.5.2 Real Metal rollouts, torque-free (`joint-fault-real-check` → `joint-fault-encoder`) [V] . . . 32
5.6 Start-pose gate . . . 33
5.7 Planned: USB port finder (`docs/USBC_PORT_CLASSIFIER_PLAN.md`, cloud-sims family) . . . 33

6. Simulators **33**
6.1 The virtual-clock stress simulator (`experiments/stress_match.py`) . . . 34
6.2 The MuJoCo station stand-in (`sim_station.py`) . . . 34
6.2.1 Table v1 (7 Nov; main) . . . 35
6.2.2 The rebuilt realistic table (8 Nov; `cloud-sims` only) . . . 35
6.2.3 Which results came from which table . . . 35
6.3 The rubric referee . . . 36
6.4 The home cluster runner (`cluster/`) . . . 36
6.5 The Google Cloud runners (`cloud/`, branch `cloud-sims`) . . . 36

7. Experiments and results **37**
7.1 Latency and the serving loop (real station) [V] . . . 37
7.1.1 Metal driver on a simulated Damiao bus [D] . . . 37
7.2 Robustness in the stress simulator . . . 38
7.2.1 Before and after the overnight fixes (main 3055bf3 → d938644, 4 seeds) [V] . . . 38
7.2.2 The 185-match configuration sweep (37 scenarios × 5 seeds) [V from raw rows] . . . 38
7.2.3 Our own reruns of the committed stress sim (9 Nov) [V] . . . 39
7.3 Judges . . . 39
7.4 Local classifiers on station data . . . 39
7.5 The policy under the harness in MuJoCo (table v1) . . . 39

7.5.1 Cluster matches, 7–8 Nov [V] . . . 39
7.5.2 Arm naming (dual-arm benchmark, codex/dual-arm-experiment) [D] . . . 40
7.5.3 Sim-time and relay-order batches (prompt-engine, table v1) [U] . . . 40
7.6 The prompt study on the station: 101 rollouts (Daniel Carrera) [D] . . . 40
7.7 Cloud round 1 (8 Nov, MuJoCo, real Dream-1) [V] . . . 41
7.8 The fine-tune vs base on the station . . . 41
7.9 What is missing . . . 41

8. Teleoperation and data collection **41**
8.1 Recorders (darshp848/follower_code) . . . 42
8.1.1 Metal leader/follower details (metal/leader_collect.py) . . . 42
8.1.2 Quest teleop and the continuity fixes . . . 42
8.2 Dataset format: LeRobot v2.1 vs v3.0, and add_eef . . . 43
8.3 What was recorded (TELEOP_PLAN.md) . . . 43
8.4 Data still missing at the end . . . 44

9. Fine-tuning: why relay-final aimed high **44**
9.1 The job . . . 44
9.2 Symptom . . . 45
9.3 Cause 1 (strong): gravity sag in the Quest actions . . . 45
9.4 Other causes, ranked . . . 45
9.5 Offline fixes tried on a fitted Metal model (station-tuning) [D] . . . 46
9.5.1 The EEF override that was built (d5e9948, station-tuning) . . . 46
9.6 The recipe for a corrected fine-tune (not run) . . . 47
9.7 Base vs fine-tune from the station logs [D] . . . 47

10. Operations and failure modes **47**
10.1 Dream errors seen, and what the harness does . . . 47
10.1.1 On the station [V] . . . 47
10.1.2 In sim and cloud runs (8 Nov) . . . 48
10.1.3 Other station failure modes . . . 48
10.2 The match laptop: MacBook Air (Apple Silicon, fanless) . . . 49
10.3 Secrets and environment (names only) . . . 49
10.4 Safety posture . . . 50

11. Practical guide **50**
11.1 Install . . . 50
11.2 Run the tests . . . 51
11.3 Run the stress simulator (no keys, no network) . . . 51
11.4 Run a MuJoCo sim match . . . 52
11.5 Cluster and cloud batches . . . 52
11.6 Run the harness live on the station . . . 53
11.7 Collect teleop data (Metal) . . . 54
11.8 Fine-tune . . . 54
11.9 Add a classifier . . . 55
11.10 Add a plan stage or prompt . . . 55

12. Known issues, open TODOs, dead ends, and what we would do next **56**
12.1 Known issues on main (acc1672), most serious first . . . 56
12.2 Open TODOs carried over (from docs/LAST_NIGHT_2026-11-08.md, STATE_MACHINE.md, the paper) . . . 57
12.3 Dead ends and things that did not work . . . 57
12.4 What we would do next, in order . . . 58

13. Verification log **58**
13.1 Test suites . . . 58
13.2 Simulations run . . . 59
13.3 Numbers checked . . . 59

13.4 report-inputs and the local-only material 60
13.5 Extra material from the 7 Nov directions doc 61
13.6 Branches found but not fully explained 61

Hill Climb: technical report and practical guide

Physical AI Hacks (MakerMods × Dream Labs), 7–8 November 2026. Team: Daniel Carrera, David Carrera, Kaushik Kanna, Muhammad Mukthar, Darsh Patel. Written 9 Nov 2026 from the three public repositories (hill-climb-harness, task_simulation, follower_code), the `report-inputs` branch of task_simulation, the paper on task_simulation's `paper` branch (`paper/main.tex`), and a session digest standing in for the local Claude/Codex transcripts.

This is the long form of the paper. **Accuracy over polish**: every number carries a mark — **[V]** recomputed or re-read from raw data or code for this report, **[D]** taken from an in-repo document or commit whose raw data is not in the repos, **[U]** could not be verified — and §13 lists everything unverified. Code references are `path:line` on **`main@acc1672`** unless a branch is named.

Contents

§	Chapter	File
1	Task, rubric v4, scoring and the station (Metal arms, cameras, Dream-1 serving stack, station API)	report/01_task_and_station.md
2	Repositories, every branch, PRs, lineage, what main does and does not contain	report/02_branches_and_lineage.md
3	Harness architecture: state machine, two-look confirmation, live guard, start-pose gate, guarded `/reset`, prompt engine, grip assist, lock, hedging and pacing, observers, DimOS	report/03_harness_architecture.md
4	The decision model: log-odds evidence, fail hazard, Q-value, optimal timeouts, the α^k bound, state-space form, shadow mode	report/04_decision_model.md
5	Every classifier: VLM judges and the 25-config benchmark, HSV detector, live guard, cable-end CNN, gripper rules, joint-fault TCN/encoders, start-pose gate, the planned port finder	report/05_classifiers.md
6	Simulators: virtual-clock stress sim, MuJoCo station stand-in, table v1 vs the rebuilt table, the referee, cluster and GCP runners	report/06_simulators.md
7	Experiments and results: latency, stress, judges, classifiers, MuJoCo, the 101-rollout prompt study, cloud round 1	report/07_experiments.md
8	Teleop and data: Metal leader/follower, Quest and the continuity fixes, LeRobot v2.1 vs v3.0, `add_eef`, the 170-episode dataset	report/08_teleop_and_data.md
9	Fine-tuning: why `relay-final` aimed 5.5 cm high, the replay fixes, the EEF override, the recipe	report/09_finetune.md
10	Operations and failure modes: Dream errors (DA-1000, DA-4000, SVD, buffer-empty, session expiry), the macOS match laptop, secrets, safety	report/10_operations.md
11	**Practical guide**: install, tests, stress sim, MuJoCo match, cluster/cloud, live station run, teleop, fine-tune, add a classifier, add a stage or prompt	report/11_guide.md
12	Known issues, open TODOs, dead ends, what to do next	report/12_known_issues.md
13	Verification log: test runs, number audit, unverified numbers, gaps, `report-inputs`	report/13_verification.md

Executive summary

The problem. A five-minute, no-human-input relay on two brakeless MakerMods Metal arms (7 Damiao motors each, three 640×360 cameras), driven by Dream Labs' hosted Dream-1 VLA behind an HTTP station agent. Stage 1: unplug USB cables (pass bar ≥ 1); stage 2: a free-standing tower > 4 in; stage 3: legal golf balls into the bin, where binning the white ball loses the match. Touching a later stage's object early is a DQ. The policy returns 48-row chunks played at 20 Hz; the Metal bus

runs at a fixed 30 Hz. The only controls are the words sent, when to stop/home/reset, and fine-tune data.

What we built. A supervisory harness that never commands a motor (§3): a YAML stage state machine with two-look confirmation, a success lock that stops the policy from undoing a pass, a 10 Hz local illegal-ball guard with its own `/stop`, a guarded `/reset`, a prompt engine that composes every instruction from linted templates, a grip-assist skill that regrips and pulls the plug with bounded `/moves`, station-side hedged Dream calls, VLM judges (Gemini Flash-Lite for frequent looks, Sonnet to confirm), and read-only LiveKit and DimOS integrations. Around it: a decision model with log-odds evidence, a hazard and a Q-value (§4, shadow mode only); seven classifiers (§5); two simulators (§6); teleop recorders and a 170-episode fine-tune (§8–9).

What the final code is. `main@acc1672` (00:06 UTC 9 Nov) is Kaushik's bot line (`claude/engine-bot` + `engine-bot-live`, with Darsh's prompt engine and the PR 9 LiveKit viewer) merged onto the 7 Nov team line. The match plan is `plans/bot_v1.yaml` on **base** Dream-1 (`dreamlabs/dream-1-beta`), because the fine-tune was worse on every task. Much of the work described in the paper — the station-retuned detector, the cable-end CNN, joint-fault nets, `decide.py`, the start-pose gate, the realistic sim table, the cloud runners, the mac-arm64 speed-ups — lives on the unmerged `cloud-sims` family, and the last six bot commits (`confirm_every_s`, stacking option B) are on `claude/engine-bot-live`, not main (§2.4.2).

Key results.

Result	Value	Mark
Dream round trip on the station (1,650 chunks, 52 runs)	median 1,116 ms, p90 2,733 ms; 66.7 % of chunks stop-and-wait; sync gap 3.54 s vs async 1.55 s	[V]
Station Dream errors (52 runs)	4 DA-4000 refusals, 4 SVD, 4 empty buffers, 4 session replacements (7 Nov subset: 4/3/2/1)	[V]
Stress sim, 185 matches/config	final hybrid + live guard 69.8 pts, 0 illegal-ball losses; fast judge without the guard lost 28	[V] from rows
Judge benchmark (60 decisions, 25 configs)	GPT-6.1 Terra 98 %/97 % at 2.8 s, $0.047/match; Sonnet 5.5 100/100 at 7.1 s; Haiku 5.5 low 100/93 at 4.5 s	[V]
Detector retune on station frames	held-out ball recall 20/27 → 26/27 at 100 % precision; bin no longer a "white ball" (1,286 → 0 of 2,010 frames)	[D]
Main's live guard on committed held-white station frames	**misses 2/2; the retuned config catches 2/2**	[V]
Cable-end CNN (MobileNetV3-small, ONNX via cv2.dnn)	USB-A recall 0.68 → 0.84 with 478 wrist crops; Blender data did not help	[D]
Joint faults, torque-free, real rollouts	encoder d32×2 blocked-joint timely recall 0.78 vs TCN 0.20 (plain recall ≈ 1.0 for both)	[V]
MuJoCo v1 table, base Dream-1, 25 matches	cable out in 14; put-down 0/54; stack 0/25; 3 DQs, all hub-left	[V] / [D]
Prompt study, 101 station rollouts / 171 steps	name the goal, not the motion; 8 of 12 harness pulls kept the plug; 9 of 15 mid-grasp rewrites sent while holding	[D] (171 [V])
Cloud round 1	10 of 29 matches valid, 0–42 pts; killed by DA-1000 (out of credit) at ~28 sessions	[V]
Fine-tune `relay-final`	Quest demos' actions 5.5 cm above the tool (no gravity feed-forward); live tool stays 8–13 cm up vs 4–6 cm grasp height	[D]
Tests on main (9 Nov, this report)	334 passed, 1 failed (`test_bot_v1_plan`), 10 skipped, 1 collection error without Dream's SDK	[V]

What is still unknown. There is no per-stage success rate from full harness matches on the station; the real judge was never run against the MuJoCo referee; the decision model never made a decision; the joint-fault nets have never seen a real fault; and the most important open safety item is that main's illegal-ball colour config predates the station retune (§12.1).

How to read the numbers

- Simulation (stress sim, MuJoCo) and station numbers are kept apart in every table. The stress sim's unplug probability (0.8 per try) is far above base Dream-1's real ~1 in 6; its scores test

harness logic only.
- The MuJoCo arm is an i2rt YAM, not a Metal arm; the first table (v1) used white balls as legal.
- Small counts throughout: 12 held-out detector frames, 137 cable crops, 5 test rollouts for joint faults, 30 guard questions per judge config, 3–26 steps per prompt family.

Building the PDF

`docs/TECHNICAL_REPORT.pdf` is built from this file and `docs/report/*.md` with pandoc and XeLaTeX:

```
cd docs && pandoc TECHNICAL_REPORT.md report/0*.md report/1*.md -o TECHNICAL_REPORT.pdf
 ↪  --pdf-engine=xelatex --toc --lua-filter=report/pandoc_math.lua -H
 ↪  report/pdf_header.tex -V geometry:margin=0.75in -V mainfont="DejaVu Serif" -V
 ↪  monofont="DejaVu Sans Mono" -V fontsize=10pt
```

`report/pandoc_math.lua` turns the ```` ```math ```` blocks into equations and sizes table columns by content; `report/pdf_header.tex` sets tables and code in a smaller font. Mermaid diagrams appear as code blocks in the PDF; view them rendered on GitHub.

1. Task, rubric v4, scoring and the station

Back to the index

1.1 The event

Physical AI Hacks (MakerMods × Dream Labs), 7–8 Nov 2026. Our team (Daniel Carrera, David Carrera, Kaushik Kanna, Muhammad Mukthar, Darsh Patel) entered the **Hill Climb** track: a timed, five-minute, 1v1 relay on a two-arm MakerMods Metal station driven by Dream Labs' hosted **Dream-1** vision-language-action (VLA) policy. No human input is allowed after "go". The only controls a team has are the words sent to the policy, when to stop, home or reset the arms, and (offline) the data used for a fine-tune.

Sources: rubric table in the paper (`task_simulation@paper:paper/main.tex:125-142`), the plans' `task:` texts (`plans/bot_v1.yaml:30-35`), `docs/STATE_MACHINE.md`, and `docs/DECISIONS.md` on `final-harness`.

1.2 Rubric v4 (abridged)

Stage	Pass bar	Points
1. Unplug USB cables	≥ 1 cable out of the strip	+10 first, +5 each further; +20 per loose cable plugged in (max 4)
2. Stack	tower > 4 in (10.2 cm), free-standing 3 s after release	+20; +3 per full inch; +10 for the taller tower (head-to-head)
3. Golf balls	none	+5 per legal ball in the bin; **illegal ball in the bin = match lost**
Integrations	shown to the referee before the match	+10 LiveKit, +5 Dimensional (DimOS)

Disqualification: touching a later stage's object before the current pass bar is met; any human input after "go".

Two properties of this rubric shaped the whole design:

1. **Gates are irreversible.** A false "stage done" on a pass bar makes the harness move on, and touching a block before a cable is out is a DQ. A supervisor false positive is therefore worse than a supervisor false negative.
2. **One illegal ball loses everything.** On our table the **white** golf ball is illegal and the green, yellow and orange balls are legal (Darsh, 8 Nov; `plans/table_colors.yaml` header). The rubric also mentions black 8-balls; the harness guards against both. In the MuJoCo sim the colours

were at first reversed (white legal, black 8-ball illegal), which matters when reading sim results (§6).

Target tower (Drive photo IMG_2697, encoded as `tower_target` in `plans/hill_climb_v4.yaml`): bottom to top, blue cube, pink cube, blue cube with its hole sideways, yellow square stick upright, pink round stick upright. On 8 Nov Darsh declared that **colour order does not matter** (commit 9db93f6; `runner.tower_matches`, `harness/runner.py:113-118`, compares shapes only). The final bot plan stacks four cubes and ignores the sticks (`plans/bot_v1.yaml:136-183`).

Plug-in stages were dropped on 7 Nov (base Dream-1 never seated a plug; `TELEOP_PLAN.md:9-10`). They come back only in the relay state machine draft (`docs/STATE_MACHINE.md`, `plans/relay_sm_sim.yaml` on `prompt-engine`).

1.3 Station hardware

Item	Value	Source
Arms	Two **MakerMods Metal** arms, 6 joints + gripper each, **7 Damiao motors per arm**, **no brakes**. Not i2rt YAM.	task_simulation `README.md` ("7 Damiao motors, brakeless, not YAM"); paper §3
Bus	Damiao motors over CAN at 1 Mbit/s from an Intel NUC	`docs/STATION_TUNING.md` (station-tuning)
Cameras	Overhead `scene` on a centre pole, `wrist_l`, `wrist_r`; streamed at **640×360** (from 1280×720)	FT diagnosis "Ruled out" table; paper §3
Table objects	Black 12-outlet power strip (ALESTOR, 25.7×9.7×3.0 cm) with a USB bank at the robot end; orange USB-A↔USB-C cables; three 6 cm foam cubes with a 2.8 cm through hole (two blue, one pink, later a green one appears in the bot plan); yellow 2×2×10 cm square stick; pink 2.5×11 cm round stick; green/yellow/orange (legal) and white (illegal) golf balls; white slotted basket	commit 9c89ae3 message; `plans/bot_v1.yaml`
Station SDK	`dreamlabs` **1.0.37** on the NUC (inspected read-only)	`docs/STATION_TUNING.md`

1.4 Control stack (Dream-1 on the station)

```
flowchart LR
    subgraph Cloud[Dream Labs cloud]
      D1[Dream-1 model<br/>~293 ms serve]
      GW[Gateway<br/>~650-990 ms overhead]
    end
    subgraph NUC[Station NUC]
      AG[Station agent HTTP :8091<br/>/status /start /stop /home /reset /prompt]
      ADD[Our add-ons via sitecustomize<br/>/frame /arms /move /pose<br/>+ opt-in PAH_*
↪   hooks]
      PACE[Pacer: async if median RTT < 900 ms<br/>else sync stop-and-wait]
      CMD[Command thread 250 Hz<br/>interpolates 20 Hz rows]
      BUS[Metal bus thread 30 Hz<br/>reads state, sends MIT frames]
    end
    H[Harness on match laptop] -- HTTP --> AG
    H -- HTTP --> ADD
    AG --> PACE --> GW --> D1
    D1 -- 48-row chunk --> PACE --> CMD --> BUS --> M[(7 Damiao motors x 2)]
```

Layer	Rate	Notes
Policy chunk	48 rows played at **20 Hz** = 2.4 s per chunk	`control_hz` 20; the fine-tune was trained at 30 fps ("20 Hz plays every plan at two thirds speed")
Command thread	250 Hz (240 measured)	interpolates rows

Layer	Rate	Notes
Metal bus thread	**30 Hz, hard-coded** (`robots/metal.py ARM.tick_hz`)	so 7 of every 8 interpolated commands never reach a motor
Velocity feed-forward	low-pass `FF_ALPHA = 0.04` per tick ≈ **0.8 s** time constant	lags any motion shorter than ~1 s
Pacer	**async** (arm keeps moving during inference) only while the median round trip **< 900 ms**; otherwise **sync** (wait for each chunk)	station.log `dream.pacing: pacing sync ... over the 900 ms threshold`

All from `docs/STATION_TUNING.md` (branch `station-tuning`) and `docs/DECISION_MODEL.md` F1–F3 (branch `final-harness`).

1.4.1 Station HTTP API

The **station agent** (Dream's) serves, on `127.0.0.1:8091`: `/status`, `/start`, `/stop`, `/home`, `/reset`, `/prompt`. One prompt drives **both** arms; there is no per-arm command and no way to run two sessions on one station (`docs/DECISION_MODEL.md` §2).

Our **station add-on** (`station_addons/sitecustomize.py`, loaded with `PYTHONPATH=.../station_addons dreamcli robot up`) adds:

Route	What	Used by
`GET /frame/<cam>.jpg[?w=640]`	newest controller frame, ~35 KB (vs ~2 MB debug-view trips)	light frames, live guard (`harness/live_guard.py:35-60`)
`GET /arms`	per gripper: measured opening, commanded opening, effort; joints	gripper-width rules (`harness/runner.py:506-530`)
`POST /move`	bounded relative Cartesian move of one arm, ≤ 12 cm per call (`MAX_MOVE_M`, `harness/plan.py:33`)	grip assist (regrip, pull, drop)
`GET /pose`	tool xyz per arm	grip assist `min_z`, pull Δz
`/objects`	**sim only**: ground-truth object poses	oracle verifier, referee

`/move` and `/pose` came from Kaushik's `move_addon.py` (`extras/kaushik/station_addons/` on `kaushik/day1-extras`, NUC copy under `nuc/pah_judge/` on `station-tuning`). Every add-on is off unless enabled, and the tuning hooks refuse to load on any SDK other than 1.0.37 (`docs/STATION_TUNING.md`). The harness client is `harness/station_client.py` (routes at lines 80-146; `/move` refuses > 12 cm at line 139).

1.4.2 Timing facts that shape the harness

- A camera frame from the debug view arrives every **3.4 s** median (`last-run.csv`), which set the original stage check windows (`harness/live_guard.py:3-4`).
- Dream round trip on the station: median **1,116 ms** over 1,650 chunks (verified, §7.1). Because that sits above the 900 ms pacer threshold, **66.7 %** of chunks (1,101/1,650) ran stop-and-wait.
- A fresh `/start` costs **2.9 s** (median) before motion, sometimes 7–12 s of discarded first answers; `/prompt` on a running session costs nothing extra (`DECISION_MODEL.md` F3; commit 86d039d).
- A fine-tune "cold start" took 3+ min, up to 15 min ("the policy is still starting…"; prompt-comparison replay).

2. Repositories, branches and lineage

Back to the index

All times are **UTC**. Commit authors' local zones were −0600 (Darsh, Muhammad), −0800 (Kaushik, Daniel, David) and +0000 (some Claude-authored commits), so `git log` dates mix zones; the tables below normalise them. Data from `git for-each-ref`, `git log --graph --all`, `git merge-base` and the GitHub PR list, taken 9 Nov 2026 ~07:00 UTC.

2.1 The three repositories

Repo	Branches	What it holds
`darshp848/hill-climb-harness`	40 remote branches, 253 commits reachable	The harness that supervises Dream-1 on the station; station add-ons; stress sim; MuJoCo station stand-in; plans; cloud/cluster runners (on `cloud-sims`); classifiers (on the `cloud-sims` family); docs
`darshp848/task_simulation`	`main` (60d6846), `paper` (3ed22bf), `report-inputs` (6cc3224, pushed 07:09 UTC 9 Nov)	A synced copy of the harness at `main@f2f400e`, the 7–8 Nov **result files** (stress, judge bench, cloud round 1, cluster MuJoCo, station logs), plots, and the 16-page paper on `paper`
`darshp848/follower_code`	`main` (e87a17a), `quest-continuity` (8aa415d)	Teleop: i2rt YAM leader/follower (refuses unless `body: yam`), `metal/leader_collect.py` for Metal, `quest/quest_collect.py`; `quest-continuity` adds `WristFlipGuard`, `TrackingJumpGuard` and the replay bench

`report-inputs` was **not** there when this work started and **appeared during it** (re-checked with `git ls-remote` at the end, as the brief asked). It holds the local-only design docs, the Codex reports and three Codex commits as patches; all of it is folded into this report, and §13.4 lists what each file added and what is still missing.

2.2 hill-climb-harness: every remote branch

Status: **in main** = an ancestor of `main@acc1672`; **open PR** = has an unmerged PR; **ahead/behind** = commits relative to `main`.

Branch	Head	Last commit (UTC)	Author	Ahead / behind main	Status	What it is
`main`	acc1672	9 Nov 00:06	darshp848	0 / 0	final	Merge of `claude/engine-bot` (PR 9 LiveKit viewer) and `claude/engine-bot-live`; plan `bot_v1.yaml` on base Dream-1
`claude/focused-lovelace-2z709x`	acc1672	9 Nov 00:06	—	0 / 0	= main	Session branch of this report
`claude/engine-bot`	3e8897b	8 Nov 23:34	kaushik kanna	0 / 12	in main	Bot harness + prompt engine plans (`plans/engine_bot.yaml`); merge of PR 9
`claude/livekit-viewer`	3805cca	8 Nov 23:27	kaupau	0 / 13	in main (PR 9)	View-only LiveKit camera card in the panel + standalone on :8098 (`harness/livekit_view.py`)
`claude/engine-bot-live`	d7aac1e	9 Nov 00:18	kaupau	13 / 11	**6 non-merge commits not in main**	Last bot fixes: orange-cable prompt, stacking option B, `confirm_every_s` (Sonnet re-checks the tower every 10 s)
`claude/engine-bot-local`	d7aac1e	9 Nov 00:18	kaupau	= engine-bot-live	duplicate ref	Same commit as `engine-bot-live`
`claude/harness-stitch`	28336b0	9 Nov 00:18	kaupau	6 / 51	open PR 5 (into `claude/bot-prompt-writer`)	“Bot day” harness: grip assist, `bot_v1.yaml`, hedged calls, live guard, `hands_off`, panel, success lock

Branch	Head	Last commit (UTC)	Author	Ahead / behind main	Status	What it is
claude/bot-prompt-writer	21bad3a	9 Nov 03:16	kaupau	1 / 75	abandoned tip	tools/full_run.sh (all three tasks with the 8 Nov hand-tested prompts) on top of 7397826
relay-on-stitch	f8771cd	8 Nov 22:48	darshp848	5 / 65	open PR 8 (into claude/harness-stitch)	Prompt engine + relay state machine merged into the bot harness (plans/bot_relay.yaml); LiveKit observer token minting
prompt-engine	3e7866c	8 Nov 21:43	darshp848	2 / 73	merged into engine-bot via c46b479, tip not	Template engine (harness/prompt_engine.py), relay state machine (plans/relay_sm_sim.yaml), sim-time clock (--clock sim)
hybrid-harness	b5883e4	9 Nov 03:16	kaupau	1 / 88	in main except tip	Kaushik's hybrid harness (monitor-written prompts, colour/position prompts, "no power" prefix, light frames, label_rollout.py); tip adds one operator lesson
kaushik/cloud-sims-bot-plan	faba96d	9 Nov 03:16	kaupau	48 / 85	abandoned	plans/hill_climb_v4_bot.yaml on top of cloud-sims
claude/sim-hillclimb	400cbde	9 Nov 03:17	kaupau	48 / 77	abandoned	MuJoCo hill-climb fixes (no false cable passes, no early-touch DQs) + merge of cloud-sims
kaushik/day1-extras	6365876	9 Nov 03:18	kaupau	1 / 0	on top of main	extras/kaushik/: first harness, conductor, /move add-on, teleop dashboard patches, v3_to_v21.py, trim_episodes.py
assets	54c8108	8 Nov 23:10	kaupau	1 / 123	orphan-ish	PR 9 screenshots (not used here: may show people)
final-harness	777e389	8 Nov 10:53	darshp848	18 / 85	superseded by cloud-sims	main(7 Nov) + start-pose-live-guard + hybrid-harness; guarded /reset, --match, stitched/parallel judge, FINAL_HARNESS.md, docs/decisions.pdf + DECISIONS.md
start-pose-live-guard	c2098dc	8 Nov 06:52	darshp848	10 / 100	merged into final-harness	10 Hz live guard, start-pose gate, cable-end classifier, harness/decide.py (shadow mode)
hedged-calls	bf33014	8 Nov 11:38	darshp848	22 / 85	merged into cloud-sims	harness/hedge.py, fast loop + state-space decisions (statespace.py), promptgen.py levels 0–5, OpenRouter judges, tools/bench_judges.py
mac-arm64	ad2a55f	8 Nov 11:45	darshp848	26 / 85	abandoned (not merged)	3.6× faster exact guard detector, tools/mac_bench.py, macOS CI, requirements-match.txt, docs/LOCAL_DECISIONS_PERF.md

Branch	Head	Last commit (UTC)	Author	Ahead / behind main	Status	What it is
`cloud-sims`	d2203e0	8 Nov 18:12	darshp848	51 / 85	not merged (“pick the match branch” was never resolved in its favour)	Everything from final-harness + hedged-calls, operator notes as plan data, async 2 Hz pacing, GCP Batch + home-cluster runners, **rebuilt realistic sim table (9c89ae3)**, joint telemetry + learned fault nets, `GOAL.md`, `HILL_CLIMB_LOG.md`
`fix/guard-real-frames`	86121b9	8 Nov 20:03	David Carrera	53 / 85	base of PR 7	White/green retune on station cameras, tighter grip zone; `tools/guard_watch.py`, `guard_replay.py`
`fix/classifiers-8nov`	3202983	8 Nov 21:44	darshp848	54 / 85	open PR 7	Detector + cable-end model retuned on 8 Nov station frames (`docs/CLASSIFIERS_8NOV.md`)
`joint-fault-real-check`	16e4bc3	8 Nov 20:13	David Carrera	53 / 85	superseded	Torque-free joint-fault training on real rollouts, encoder size sweep
`joint-fault-encoder`	6d8ea7e	8 Nov 20:29	darshp848	54 / 85	abandoned	Exported encoder/TCN weights (`harness/models/encoder/`), `torque_free` runtime flag, `docs/JOINT_FAULT_REAL.md`
`station-tuning`	8a96aac	8 Nov 19:18	darshp848	5 / 91	not merged	`station_addons/pah_tuning.py` (opt-in `PAH_*` knobs incl. **EEF override**), bus/replay benches, NUC snapshot under `nuc/`
`ee-pull-force`	d7b4e59	8 Nov 19:24	darshp848	7 / 91	ran on the NUC 8 Nov afternoon	station-tuning + `PAH_PULL_FORCE_N` / `PAH_PULL_DIR` upward pull while the gripper holds
`ft-diagnosis`	11e87ba	8 Nov 18:40	darshp848	1 / 88	open PR 3	`docs/ft_diagnosis/FT_DIAGNOSIS_2026-11-08.md`
`docs/codex-audit-2026-11-08`	f60f0e1	8 Nov 18:48	Claude	1 / 88	open PR 4	Codex (gpt-6.1-sol) audit: 26 findings + Claude’s verification
`docs/how-it-works`	7f0daad	8 Nov 18:06	Claude	2 / 92	open PR 2	`docs/HOW_THE_HARNESS_WORKS.md` + explainer PDF
`docs/prompt-comparison-2026-11-08`	985b659	8 Nov 22:35	Daniel Carrera	2 / 87	open PR 6	101-rollout prompt study + 13:47–14:28 replay review
`tools/episode-review-page`	e5bf84f	8 Nov 05:31	David Carrera	7 / 101	abandoned	`tools/review_page.py` (episode review web page), split specs
`stress-test-fixes`	d938644	8 Nov 02:48	darshp848	0 / 103	in main	Offline stress sim fixes + local ball/block detector
`light-frames-traces`	f4bb933	8 Nov 04:43	kaupau	0 / 102	in main	Light frames from the add-on, per-run trace pages
`combined`	e6fb5a0	8 Nov 04:42	kaupau	6 / 115	absorbed (via hybrid)	`prompt_prefix` for monitor-written prompts; auto-reset on Dream’s IK error
`kaushik-fixes`	67c8c3e	7 Nov 22:49	Kaushik	1 / 115	PR 1 merged an earlier tip	Port-based unplug judge + hub zoom, watcher stage
`sim-and-move-fix`	ab4e46a	8 Nov 04:50	kaupau	7 / 115	abandoned	Kaushik’s own MuJoCo station sim (`sim/`), rubric eval, `move_addon` fix

Branch	Head	Last commit (UTC)	Author	Ahead / behind main	Status	What it is
`codex/watcher-dimos`	89c9621	7 Nov 21:36	darshp848	1 / 118	never merged	Watcher camera evidence + DimOS exposure
`codex/dual-arm-experiment`	4219317	7 Nov 23:15	darshp848	5 / 118	never merged	Sim-only `/objects`; sequential vs combined two-arm prompt benchmark; LiveKit observer (569607e)
`codex/hill-climb-review-fixes`	283b379	8 Nov 00:37	darshp848	9 / 118	never merged	Five-minute match supervisor review fixes (`MATCH_DESIGN.md`)
`codex/hill-climb-sim-table`	b372601	8 Nov 00:37	darshp848	15 / 118	never merged	First simulated relay table + rubric referee + cluster image (the same work later re-landed on main as 3055bf3)

2.3 Pull requests

PR	Head → base	State	Title
darshp848/hill-climb-harness#1	kaupau:kaushik-fixes → main	merged	Station-tested fixes: judge truncation, sticky done, mid-pull extension, gripper feel, cable plans
darshp848/hill-climb-harness#2	docs/how-it-works → main	open	docs: how the harness works end to end
darshp848/hill-climb-harness#3	ft-diagnosis → main	open	Fine-tune diagnosis
darshp848/hill-climb-harness#4	docs/codex-audit-2026-11-08 → main	open	Codex harness audit (26 findings) + Claude's verification
darshp848/hill-climb-harness#5	claude/harness-stitch → claude/bot-prompt-writer	open	Bot day in the hybrid harness
darshp848/hill-climb-harness#6	docs/prompt-comparison-2026-11-08 → main	open	Prompt vs execution comparison
darshp848/hill-climb-harness#7	fix/classifiers-8nov → fix/guard-real-frames	open	Classifiers on the 8 Nov station frames
darshp848/hill-climb-harness#8	relay-on-stitch → claude/harness-stitch	open	Relay state machine on the bot harness
darshp848/hill-climb-harness#9	claude/livekit-viewer → claude/engine-bot	merged	View-only LiveKit camera viewer

2.4 Lineage

The repository has a squashed root (2018fc3, 7 Nov 20:23, "Hill Climb harness for Dream-1: staged supervisor with monitor, recovery and match plans"). Three families grew from it.

```
gitGraph
  commit id: "2018fc3 root (7 Nov 20:23)"
  commit id: "bf585ed force/can_torque"
  branch codex-line
  commit id: "89c9621 watcher-dimos"
  commit id: "569607e LiveKit observer"
  commit id: "4219317 dual-arm"
  commit id: "b372601 sim-table (never merged)"
  checkout main
  branch kaushik
  commit id: "45c122c station fixes (PR1)"
  checkout main
  merge kaushik id: "0a1607a PR1"
  commit id: "3055bf3 sim relay table + referee"
```

```
  commit id: "08d6a86 v4 plan + guard (Muhammad)"
  commit id: "d938644 stress-test-fixes"
  commit id: "0830701 tower target"
  branch spg
  commit id: "26e1fc6 live guard + start pose"
  commit id: "ae30720 cable-end CNN"
  commit id: "c2098dc decide.py shadow"
  checkout main
  branch hybrid
  commit id: "0973c5a hybrid harness"
  commit id: "b78b44a stitched judge"
  commit id: "3ff5c06 hybrid_v4_long"
  checkout main
  commit id: "bb8fb3f relay-final FT ready"
  commit id: "b1f6e4b default to FT"
  commit id: "72bf0b3 audit A01/A08"
  checkout spg
  merge hybrid id: "bb9d0b9 final-harness"
  commit id: "89eaff7 FINAL_HARNESS.md"
  branch cloudsims
  commit id: "b1276d7 hedged-calls"
  commit id: "090df7c operator notes, 2 Hz"
  commit id: "9c89ae3 realistic table"
  commit id: "a73b628 joint fault TCN"
  commit id: "d2203e0 cloud-sims tip"
  branch classifiers
  commit id: "dcd76c5 guard-real-frames"
  commit id: "3202983 classifiers-8nov"
  checkout hybrid
  commit id: "fce05fa bot day (stitch)"
  commit id: "e4b4b19 unplug loop"
  commit id: "c46b479 + prompt-engine"
  commit id: "d7aac1e engine-bot-live"
  checkout main
  merge hybrid id: "acc1672 final main"
```

(The diagram is a simplification: Mermaid's gitGraph cannot draw octopus merges or the many intermediate `Merge branch 'claude/harness-stitch' into claude/engine-bot` commits. `spg` = `start-pose-live-guard`.)

2.4.1 The three families, in words

1. **The 7 Nov team line → main (morning of 8 Nov).** Darsh's root harness, Kaushik's station fixes (PR 1), the simulated relay table and referee (3055bf3), Muhammad's rubric-v4 plan and teleop recording plan (08d6a86), dataset tools, the stress-test fixes and the local detector (d938644), the tower target (0830701). On 8 Nov Muhammad committed the fine-tune `relay-final` (bb8fb3f), Claude switched the match plans' default to it (b1f6e4b), then fixed audit findings A01 and A08 (72bf0b3). Prompts in `plans/hill_climb_v4.yaml` are the TELEOP_PLAN texts byte for byte, because a fine-tune only follows its recorded texts (a fine-tune requirement, not a rubric rule).
2. **The "final harness" / cloud-sims family (Darsh, 8 Nov 06:00–18:00).** `start-pose-live-guard + hybrid-harness` merged into `final-harness` (bb9d0b9), then `hedged-calls`, operator decisions, async pacing, GCP Batch, realistic sim table, joint telemetry and fault nets on `cloud-sims`, and leaf branches `fix/guard-real-frames → fix/classifiers-8nov`, `joint-fault-real-check → joint-fault-encoder`, `mac-arm64`. **None of this family was merged into main.** The team never resolved the "pick the match branch: main or cloud-sims" item (`docs/LAST_NIGHT_2026-11-08.md`, to-do #1); in the end it picked neither and went with family

3.

3. **The bot line (Kaushik, 8 Nov 21:00 → 9 Nov 00:18).** `hybrid-harness` → `claude/harness-stitch` ("bot day": grip assist, `bot_v1.yaml`, hedged calls, live guard, hands-off, panel, success lock, ported from hand tests in `tools/try_prompt.py`), with Darsh's `prompt-engine` merged in (c46b479) to make `claude/engine-bot`, the LiveKit viewer (PR 9) and `engine-bot-live`. Main at **acc1672** is this line merged onto the team line.

2.4.2 What the final main does and does not contain

Feature	On main (acc1672)?	Where it lives
Stage runner, `confirm: 2`, monitor verifier, guard with `max_blind_s`	yes	`harness/runner.py`, `harness/verifiers.py`
10 Hz live guard (`--live-guard`), required by `--live` when the plan has a guard	yes	`harness/live_guard.py`, `harness/cli.py:127-136`
Grip assist (regrip, pull, drop), `stop_on_pass` lock, `hands_off`, `reset_first/last`	yes	`harness/runner.py:840-1059`
Prompt engine (templates, slots, lint, variants, prefetch)	yes	`harness/prompt_engine.py`, `plans/engine_bot.yaml`
Hedged monitor calls (`claude.hedge`), station-side hedged Dream calls	yes	`harness/hedge.py`, `station_addons/pah_hedge.py`
LiveKit observer (read-only) and viewer	yes	`harness/livekit_observer.py`, `harness/livekit_view.py`
DimOS skills adapter	yes	`integrations/dimos_skills.py`, `integrations/station_tools.py`
Panel (`python -m harness.panel`)	yes	`harness/panel.py`
`confirm_every_s` (Sonnet tower re-check every 10 s), stacking option B, orange-cable prompt	**no**	`claude/engine-bot-live` (6 commits after main's last merge)
Station-retuned detector (bin ≠ ball, yellow-green ball) and white range	**no** (main still has the phone-photo white range)	`fix/classifiers-8nov`, `fix/guard-real-frames`
Cable-end CNN (`models/cable_end.onnx`)	**no**	`start-pose-live-guard` and the cloud-sims family
`decide.py` shadow mode, `statespace.py`, start-pose gate	**no**	`start-pose-live-guard`, `hedged-calls`, cloud-sims family
Joint telemetry, rule observer, learned fault nets	**no**	cloud-sims family; encoders on `joint-fault-encoder`
`--match` flag, guarded `/reset` test, `FINAL_HARNESS.md`	partly (the guarded `/reset` is on main, `harness/runner.py:420-433`; `--match` is not)	`final-harness`
Realistic sim table (9c89ae3), GCP/cluster cloud runners	**no** (main has the v1 hub table)	`cloud-sims`
Station tuning (`PAH_*`), EEF override, pull force	**no** (station add-on files for frames/arms/hedge only)	`station-tuning`, `ee-pull-force`

So the paper's description of the match-day system mixes components from family 3 (what main runs) with components from family 2 (what was only on side branches). §3 and §5 always say which branch a component is on.

2.4.3 Branches named in the brief that do not exist on GitHub

`port-to-main`, `nuc-snapshot`, `ft-joint-offset`, `codex/hill-climb-five-minute-validation`, `codex/hill-climb-livekit`, `codex/hill-climb-match-supervisor`, `port-sim`, and three commits of `stress-test-fixes` are **not on the remote**. Equivalents found:

- "nuc-snapshot" = commit 8a96aac on `station-tuning` ("NUC snapshot: pah_judge routes, live station.yaml …").
- "ft-joint-offset" (EEF override) = commit d5e9948 on `station-tuning` ("EEF override: intercept Dream's tool-pose answer before its IK"); the joint-offset patch the FT diagnosis recommends is "Not built" (§9).
- `codex/hill-climb-livekit` survives as 569607e on `codex/dual-arm-experiment` and was ported to main via 76cc686 / b3c27fb.

- The three "local-only" Codex commits — 9e97ba8 (match supervisor), 569607e (LiveKit observer), 189c39d (validate the decision tree on 300 s runs) — are on GitHub after all, inside `codex/hill-climb-review-fixes` and `codex/dual-arm-experiment`, and `report-inputs` also ships them as `git format-patch` files.

2.5 task_simulation and follower_code lineage

task_simulation `main` (7 commits): 1e85390 (initial: harness copy, two sims, scenarios, station data) → 1cbc6a7 (sync with harness 0830701) → 044fed9 (station photo) → c95b5bb (sync with harness f2f400e) → 2157c66 (**results of the 7–8 Nov night**) → ba76131, 60d6846 (README). **`paper`** adds 6d82f2f, 39c8b32, 3ed22bf (the paper and its figure scripts). The harness copy there is a snapshot and does not track hill-climb-harness.

follower_code `main`: 32dd6ca (YAM leader/follower) → bf074cb (sim test, mirror check) → 047a27e (refuse on non-YAM stations) → 69ebdb2 (Metal leader/follower on the kit's `dhk-collect`) → 631658f → 6bc0576 (better Quest teleop) → fdf4bf8 (fix follower sag and lag) → e87a17a (Quest gravity feedforward on by default, **8 Nov 01:19 UTC**). **`quest-continuity`**: f62fc88 (wrist-flip guard, per-joint wrist cap, glitch guard) → 806093c (bench) → 8b6da36, adad9f3, 0c9a631 (homing: lift a low tool straight up) → e4e5bf1 (review fixes) → 8aa415d (README; NUC dry run pending). Not merged into `main`.

3. Harness architecture, end to end

Back to the index

Unless stated otherwise, line references are to **`main@acc1672`**. Components that exist only on side branches are marked with the branch name.

3.1 Principles

1. **Never command a motor.** The harness has no robot driver. It talks only to Dream's station agent and our add-on over HTTP. Speed caps, clamps, the control loop and homing stay inside Dream's code. The one exception is the add-on's bounded `/move` (≤ 12 cm per call), used by the grip-assist skill while Dream is stopped.
2. **Keep safety off the slow path.** Anything that must stop the arms (the illegal-ball live guard) runs locally at frame rate and sends `/stop` itself.
3. **Confirm from independent evidence.** A false "done" on a pass bar is a DQ, so passes need two agreeing looks, sensors can veto images, and the same frames judged twice count once.
4. **No free text to the robot** (prompt-engine line). Every instruction comes from a template that worked on the station, linted at load.

3.2 Component map

```
flowchart TB
    subgraph Laptop[Match laptop - MacBook Air, Apple Silicon]
      CLI[harness.cli<br/>python -m harness PLAN --live ...]
      PLAN[plan.py<br/>YAML -> Plan/Stage/GripAssist/Guard]
      RUN[runner.py Runner<br/>stage state machine]
      PE[prompt_engine.py<br/>templates + slots + lint]
      VER[verifiers.py<br/>MonitorVerifier: stitched grid,<br/>parallel looks, hedge,
↪  confirm, writer]
      DET[detect.py TableDetector<br/>HSV + shape]
      LG[live_guard.py LiveGuard<br/>10 Hz, 3 frames -> /stop]
      LOG[log.py RunLog<br/>runs/TS/events.jsonl + frames]
      PANEL[panel.py operator panel<br/>practice only]
      LKV[livekit_view.py / livekit_observer.py<br/>subscribe only]
```

```
    end
    subgraph APIs[Remote judges]
      HA[Claude Haiku 5.5 / Sonnet 5.5<br/>Anthropic API]
      OR[OpenRouter: Gemini 3.1 Flash-Lite,<br/>GPT-6.1 Terra ...]
    end
    ST[Station agent + add-on on NUC]
    CLI --> PLAN --> RUN
    RUN --> PE
    RUN <--> VER
    VER --> HA
    VER --> OR
    VER --> DET
    RUN -- fast_guard per poll --> DET
    LG --> DET
    LG -- GET /frame/wrist_*.jpg --> ST
    LG -- POST /stop --> ST
    RUN -- /start /prompt /stop /reset /move --> ST
    RUN -- GET /status /arms /pose --> ST
    VER -- frames --> ST
    RUN --> LOG
```

File (main)	Lines	Role
`harness/cli.py`	260	Flags, wiring, refusals (e.g. `--live` + plan guard requires `--detect … --live-guard`, lines 127-136)
`harness/plan.py`	685	Schema and validation of plans; dataclasses `Recovery` (37), `GripAssist` (50), `Stage` (102), `Guard` (172), `Watch` (189), `Plan` (204)
`harness/runner.py`	1,411	The supervisor state machine
`harness/verifiers.py`	942	`manual`, `time`, `claude`, `monitor` verifiers; `MonitorVerifier` (531) with stitch (58), parallel/hedge (555-562), `confirm` (791), `write_prompt` (828)
`harness/prompt_engine.py`	325	`Library.lint` (79), `parse_library` (105), `DefaultFiller` (182), `ClaudeFiller` (195), `PromptEngine.compose` (271), `prefetch` (289)
`harness/detect.py`	394	Colour + shape detector; `TableDetector.guard` (348)
`harness/live_guard.py`	196	`LiveFrames` (35), `LiveGuard` (86-)
`harness/hedge.py`	80	`first_of`: N copies, first good answer
`harness/station_client.py`	172	HTTP client; `/move` ≤ 12 cm (139)
`harness/frames.py`	96	Debug-view trips and light frames
`harness/panel.py`	397	Operator panel (practice)
`harness/livekit_observer.py`, `livekit_view.py`	323, 192	Read-only LiveKit
`harness/force.py`, `can_torque.py`	258, 160	Torque/force estimates from Dream's trace and a passive candump
`integrations/station_tools.py`, `dimos_skills.py`	—	DimOS / MCP tools
`station_addons/sitecustomize.py`, `pah_hedge.py`, `pah_topdown.py`	—	Station-side add-ons (frames, `/arms`, hedged Dream calls)

3.3 A match, step by step

```
sequenceDiagram
    participant H as Runner
    participant S as Station agent
    participant M as Monitor (fast) / Confirm (Sonnet)
    participant G as LiveGuard (10 Hz)
    H->>S: GET /status (preflight: idle, arms alive, cameras < 2 s old)
    H->>S: POST /reset (reset_first; skipped while the live guard is armed)
    loop each stage
      H->>G: arm(stage) if the plan guard covers it
      H->>H: compose prompt (template+slots) or stage.prompt
```

```
  H->>S: POST /start (first) or /prompt (later)
  loop every check_every_s
    H->>S: GET /status, GET /arms (grip assist polls)
    alt gripper shut on nothing / holding plug (grip_assist)
      H->>S: /stop, /move (regrip / pull up), /start (put-down or re-approach)
    end
    H->>M: look (stitched grid, grip note) - async, parallel
    M-->>H: verdict {done, progress, status, holding, guard}
    alt done twice (confirm 2)
      H->>S: lock: /prompt "open both grippers and move the arms up"
      H->>M: confirm_check (Sonnet, synchronous)
      M-->>H: yes -> stage passes / no -> resume
    end
  end
  G-->>S: POST /stop (3 yes frames in a row on one camera)
end
H->>S: POST /stop; POST /reset (reset_last, not after a guard trip)
```

3.3.1 Preflight and the guarded `/reset`

`Runner._preflight` (`runner.py:293-314`) refuses to start unless the station is `idle` or `error`, every arm is alive and every camera frame is younger than `MAX_CAMERA_AGE_S = 2.0` s (line 22). In `--dry-run` the same problems are logged as warnings.

`_reset_and_wait` (`runner.py:420-433`) sends `/reset` through `_motion` (so an operator stop or a guard trip blocks it) and **skips it while the live guard is armed**, because `/reset` moves both arms and cannot be cancelled once begun. This fix came from the `final-harness` merge ("hybrid's `_reset_and_wait` posted `/reset` directly"); `/home` in `home_and_retry` is refused the same way (`runner.py:1395-1396`).

`_motion` (`runner.py:347-356`) wraps every command that can move an arm: it checks the stop event, the clock and the guard **before** sending, and again **after**, because a guard trip that lands during the command may have been overtaken by it; in that case it sends `/stop` again and ends the match.

`_safe_stop` (`runner.py:358-370`) sends `/stop` **first and logs after**, so a failing log write cannot keep the stop from going out (Codex audit A01; `run()` now also catches any `Exception`, `runner.py:267-273`, commit 72bf0b3).

3.3.2 Faults from Dream

`_fault_kind` (`runner.py:91-98`) classifies `last_error` text:

Kind	Keys	Action
refusal	`DA-4000`, `DreamBadRequest`, `unsafe instruction`	restart the same prompt `REFUSAL_RESTARTS = 1` time, then the attempt fails (`_Refused`) but the match goes on
transient	`DreamAPI`, `DreamServerError`, `DreamProtocolError`, `DreamSessionExpired`, `DA-5000`, `DA-6000`, `DA-0001`, `GATEWAY_FAULT`, `connection_timed_out`, `ControlLoopError`, `action buffer empty`, `LinAlgError`, `SVD did not converge`	restart with the same prompt, at most `MAX_STATION_RESTARTS = 5` per run
other (arm, motor, safety stop)	—	never retried: `_Fail` → `/stop`

For `SVD did not converge` / `LinAlgError` the runner first sends a `/reset` (grippers open, arms to the start pose) and waits for idle, at most `AUTO_RESET_MAX = 2` times per run (`runner.py:37-40, 435-441`), because a plain `/start` after the IK error often failed again on the station. Before any restart it waits `ARCHIVE_WAIT_S = 4` s so the station's archiver (3 s poll) copies Dream's run record (run 2 on 8 Nov lost 9 min of record without it, `runner.py:41-43`).

3.3.3 The stage loop (`_watch`, `runner.py:572-837`)

Per poll (every `check_every_s`, 1.0 s in `bot_v1.yaml`):

1. **Grip assist** (if the stage has one) replaces the nap and may drive the arm (§3.6). Time the harness spends moving the arm is **not** charged to the stage clock (`t0 += spent`, line 599).
2. **Health** (`_poll_health`): station `error` → fault handling and restart; dead arm → `_Fail`.
3. **Operator** actions (practice only): notes to the monitor, skips, typed prompts.
4. **Local guard** on every poll (`self.fast_guard`, the detector on the latest frames, lines 648-660): `guard.confirm` yes answers in a row → `_Guard` → `/stop`.
5. After the gate (`max(min_run_s, settle_s)`), **ask the verifier** (async: the call may still be in flight; the verdict that comes back is the newest finished one). Log it with the frames it saw.
6. **Guard from the monitor**: a `guard` yes counts; `max_blind_s` without any guard answer → `_Guard` ("guard blind").
7. **Rewrites / refills** (only without `hands_off`, §3.5).
8. **Progress bookkeeping**: regression (`p ≤ max_p − 0.3`) and stall (`p` not up by 0.05 for `stall_s`) mark a failing look, again only without `hands_off`.
9. **Done logic** (two-look confirmation, §3.4) → lock → `confirm_check` → pass.
10. **Timeouts** with two one-time extensions (§3.4.3).

3.3.4 Stage results and retries (`_run_stage`, `runner.py:1305-1411`)

`on_fail ∈ retry_in_place` (default), `home_and_retry`, `skip`, `abort`, `retry_then_skip` (`plan.py:16`). Retries use `/prompt` on the running session, not `/stop` + `/start`, because a fresh `/start` costs 2.9 s median. Pass-bar stages use `max_attempts: 99` ("keep trying until the clock runs out"), because moving on from an unmet pass bar means touching the next stage's objects (a DQ). The relay state machine adds `retry_then_skip` with Darsh's rule (2 tries, 3 at ≥ 20 points; pass bars exempt) and `not_after_s` match-clock cutoffs (`runner.py:1242-1243, 1385-1386`; `docs/STATE_MACHINE.md`).

Stage-level skips before a stage starts: `done_before` (`--skip`), `requires_stage` (e.g. no cable came out → nothing to put down), operator skip, `not_after_s` cutoff, `skip_unless` precondition (one synchronous look), `skip_if_done` entry check (one synchronous look at confidence ≥ 0.8; audit A02 flagged this for `tower_pass`).

3.4 Two-look confirmation and the done logic

3.4.1 The rule

```
done verdict on new frames      -> consec_done += 1
  pass if  verdict.immediate
       or  consec_done >= confirm                    (confirm = 2 in bot_v1 and v4)
       or  a previous done within DONE_WINDOW_S = 15 s ("sticky done")
same frames as the previous done -> not counted ("same_frames" event)
done but the stage needs a grip and no gripper held anything -> ignored
 ↪  ("done_without_grip")
failing verdict within 15 s of a done -> ignored (one flicker does not undo a done)
```

Code: `runner.py:766-801`. The frame signature (`frames_sig`) stops a frozen camera from confirming its own verdict twice. `needs_grip` uses the gripper-width rules (§5.4).

3.4.2 Lock, then confirm (`stop_on_pass`, `_lock_success`, `_confirm_check`)

When a look would pass, and `plan.stop_on_pass` is set, `_lock_success` (`runner.py:1047-1059`) switches Dream to `lock_prompt` ("open both grippers and move the arms up") **before** the confirming look, or `/stop`s if no lock prompt is set. Reason (station, 8 Nov): "stack … as high as possible" built a blue-on-blue stack and then took it apart; in the 13:47–14:28 replay both the 2-high and the 3-high stacks were destroyed by the next prompt. Using a prompt instead of `/stop` keeps the session warm (a fresh `/start` costs 7–12 s of first answers, commit 86d039d). Stacking

stages set `lock_prompt` to the **next** cube's prompt so the tower "flows" without an arms-up pause (`plans/bot_v1.yaml:140,156`).

`_confirm_check` (`runner.py:1061-1076`) then asks the confirm model (`claude.confirm_model`, Sonnet 5.5) the stage's `confirm_check` question synchronously. A "no" resets the done count, appends the reason to the fast monitor's operator notes, and (with `resume_on_no`) sends Dream back to the prompt it was running. A confirm that cannot answer passes ("None to pass").

Why a second model: on 8 Nov Gemini 3.1 Flash-Lite (~2.6 s) missed a real unplug and rewrote prompts 11 times in one run, so every unplug pass is confirmed by Sonnet (`plans/bot_v1.yaml:20-23`). On `claude/engine-bot-live` (not main) Sonnet also re-checks the tower every 10 s on its own (`confirm_every_s`, commit 28336b0) because the fast monitor missed a four-cube tower.

3.4.3 Timeouts at four levels

Level	Value	Code
Match budget	`time_budget_s: 295` (bot_v1); checked in `_check_global`	`runner.py:316-323`
Per-stage cutoff on the match clock	`not_after_s` (relay plans: no new plug-in try after 150 s, tower extras by 230 s)	`runner.py:1242`; `docs/STATE_MACHINE.md`
Retry budget	`max_attempts` (99 for pass bars, 1 + `on_fail: skip` for bot_v1 stacking)	`runner.py:1365-1369`
Per-try timeout	`timeout_s` + one `CONFIRM_GRACE_S = 10` s when a done awaits its confirming look + one `EXTEND_S = 20` s when the monitor sees the gripper on the target with progress ≥ 0.5	`runner.py:826-837, 1078-1084`

In the final `bot_v1.yaml` every stage has `timeout_s: 295` ("no per-stage limit (Kaushik, 8 Nov): only the 5 min match cap").

3.5 Prompts: from monitor-written text to the prompt engine

Four generations, all still in the code:

Generation	Branch	Mechanism	Status on main
Recorded texts, byte for byte	main 7 Nov (`hill_climb_v4.yaml`)	Stage prompts equal the TELEOP_PLAN task texts (T1 `unplug the USB cable at the end of the hub nearest the right arm`, T2 `put the held cable down on the table away from the hub`, …); `python -m harness.tasks PLAN DATASETS` checks every prompt has demos	kept for a fine-tune (`mode: verbatim`)
Monitor-written prompts	`combined` / `hybrid-harness` (Kaushik)	The monitor returns `next_prompt`; the runner sends it when the robot is stalled/failing, with a cooldown (15 s) and a cap (3 per attempt), prefixing `prompt_prefix`	kept, but `bot_v1` sets `rewrite_max: 1`, `rewrite_cooldown_s: 30` and `hands_off: true`, which disables mid-attempt rewrites
Stage-prompt writer	`claude/bot-prompt-writer` → stitch	`write_prompts: true`: Sonnet writes one instruction per stage from the frames; `writer_notes` constrain it; `SKIP` skips the stage	on for balls only in `bot_v1`
Prompt engine	`prompt-engine` → engine-bot	`prompt = template[state, step, variant](slots)`; the monitor's free text is never sent; a stall re-fills slots with the monitor's reason as a hint	on in `plans/engine_bot.yaml`

3.5.1 Prompt engine details (`harness/prompt_engine.py`)

- **L0 library** (plan `prompt_engine.templates` + `vocab`). Each template has `text` with `<slot>` holes, optional `prefix: true` (put the "no power" sentence in front), `arm: none|left|right|any`, and

slots: {hole: vocab_list}.

- **Lint at load** (Library.lint, lines 79-94): word cap (max_words, 30 by default, 60 in engine_bot.yaml); no negation (regex not|no|never|don't|do not|without|except|avoid|instead of|other than: "not the white ball" put the white ball in play in the 8 Nov hand tests); no arm named in an arm: none template (stacking: Dream picks the near arm); no forbid phrase (white ball, 8-ball, ...); no unfilled slot. Every vocabulary option is linted, so whatever a filler picks is already clean.
- **L1 filler**: default (stage defaults, no model), claude (one cheap call returning option numbers as <slots>{"piece": 1}</slots>), or the sim's OracleFiller (ground truth). A bad reply or an over-long combination falls back to the defaults rather than ending the match.
- **L2 policy** (runner): compose at stage start and on each retry; **variants** — try n uses ([template] + variants)[(n−1) mod k] (runner._variant, lines 1245-1251), so a failed try is worded differently and the slots are re-filled from the scene as it is now; **refill** on a stalled/failing verdict, re-sent only if the text changed (each change restarts Dream's motion); **prefetch** of the next stage's slots on the first done verdict (_prefetch_next, lines 1262-1269), used at the boundary if younger than prefetch_ttl_s (20 s).
- **Verbatim mode** for a fine-tune: stage prompts byte for byte; the loader refuses rewrite_prompts, write_prompts, tower_prompts or templates with it (docs/PROMPT_ENGINE.md).

Second-try variants name **the goal, not the motion** (commit 3e7866c), from the 101-rollout study (§7.6).

3.5.2 The byte-for-byte rule and the safety sentence

A fine-tuned Dream-1 only follows its recorded task texts, so for a fine-tune every prompt must equal a recorded text byte for byte. That is a property of the fine-tune, **not a rubric rule**. The audit (A08) found main and cloud-sims had switched the default to the fine-tune while still sending untrained texts (tower_prompts's "stand a stick upright on top of the stack"); 72bf0b3 made hill_climb_v4.yaml send only recorded texts on the fine-tune.

Base Dream-1 refuses cable work without a true safety statement (4 × DA-4000 on 7 Nov). The safe variable (plans/bot_v1.yaml:43): "The power strip is unplugged from the wall and has no power; the orange cords are insulated low-voltage USB cables, not bare wires." FINAL_HARNESS.md insists: keep it only while it is literally true. We did not try to trick the filter.

3.5.3 Mid-grasp rule

Nothing is re-prompted while a gripper is closed on an object: on the station 9 of 15 judge-written micro-rewrites were sent while the arm held the plug, and 5 of those dropped it (prompt study U4). hands_off (commit 2487a17) leaves Dream alone between pass, guard and timeout; typed operator prompts turn the grip assist off for that attempt.

3.6 Skills: grip assist, pull, drop, lock

GripAssist defaults (plan.py:50-98) and the values bot_v1.yaml uses (lines 105-116):

Key	Default	bot_v1	Meaning
arm	right	right	arm the skill drives
regrip	2	2	re-grips allowed when Dream shut on air
regrip_dz	−0.015	−0.025	lower this far before closing (0 of 6 re-grips held at −1.5 cm live)
pull_up_m	[0.06, 0.04]	[0.10]	lift steps (each /move costs 3–6 s of arm re-sync on the station)
pull_s	3.0	4	seconds for the lift
seat_dz	0.0	0.0	optional "open a little, lower" before the squeeze
repull	2	0	re-pulls after a slip (a re-pull stopped Dream ~19 s live)
hold	(0.045, 0.20)	(0.062, 0.20)	gripper reading band that means "holding a plug" (empty reads 0.03–0.06 on this station)

Key	Default	bot_v1	Meaning
`open_above`	0.5	—	the gripper must have been open on the way in
`empty_below`	0.04	—	after the squeeze/lift, below this = lost
`min_z`	None	0.035	never pull from below this tool height (a cable lying on the table reads ~0.023 m)
`pass_on_pull`	true	true	a held pull + the confirm count = pass
`pull_count_dz`	0.08	—	lifted ≥ 8 cm with something still in the jaws → the plug is out, no confirm needed
`put_down_prompt`, `put_down_s`	—	the cable prompt, 15 s	hand back to Dream to place the cable
`drop_dxyz` / `drop_at`	None	—	carry and open at a relative / absolute point (basket)
`loop_until_s`	None	—	keep unplugging the next plug until this match time

Trigger logic (`_grip_assist_wait`, `runner.py:850-882`): poll `/arms` every `poll_s`; require that the gripper was open (> `open_above`) on the way in; **regrip** if the last two readings are ≤ `hold[0]` (shut on nothing); **pull** if the last two are inside the hold band, nothing since the last open was ≤ `hold[0]`, and Dream is commanding the jaws shut (`cmd ≤ 0.2`) — the last condition came from 8 Nov live data where a half-open approach (0.07–0.17) fired 9 pulls that squeezed air.

Action (`_grip_assist`, lines 891-938; `_pull`, 940-980): `/stop`, wait idle; regrip = open 0.6 → lower `regrip_dz` → close → re-read; check `min_z` from `/pose`; squeeze in place and re-read (Dream's stop can loosen the jaws); lift by `pull_up_m`; if it slipped, lower back to where Dream left the hand (a restart from 10+ cm up caused Dream's SVD error and a full `/reset`); then put-down / drop / re-approach. Every `/move` goes through `_motion`.

The harness pull was the only thing that unplugged reliably on the bot: "2/2 plugs held through the pull on 8 Nov" (`plans/bot_v1.yaml:4-6`), and in Daniel's study 8 of 12 harness pulls still held the plug after a 10 cm lift (§7.6). An opt-in station hook (`ee-pull-force`, `PAH_PULL_FORCE_N=r:20`) adds a 20 N upward force while the right gripper holds; in the 13:47–14:28 replay it **fired 70 times, including on empty closes at home**, and 65 of 70 ended "moved: out, or nothing held it", which cannot tell a pulled plug from an empty gripper. Verdict there: gate it to unplug steps or turn it off.

3.7 Keeping the policy fed: pacing, light frames, hedging, pipelining

Change	Where	Effect (as measured)
Light frames: `/frame/<cam>.jpg?w=640` (~35 KB) instead of ~2 MB debug-view trips	hybrid → main	Dream's round trip went from 2–6 s to ~0.8 s once the harness stopped pulling trips (`FINAL_HARNESS.md`)
Dream async pacing at 2 Hz in the sim (`execution_horizon` 10 at 20 Hz), physics never holds for inference	cloud-sims (090df7c)	A laptop match logged **5.96 requests/s at 0.625 s p50 RTT** in a live snapshot (`results/cloud_round1_20261108/laptop/.../hybrid_v4-6/live/status.json`); the final `match.json` of that match says 5.52 Hz
`PipelinedPolicy`: two sessions in turn so a request leaves before the previous answer returns	cloud-sims	doubles Dream usage
Hedged Dream calls on the station: `HedgedPolicy`, N sessions fed the same observation, first reply drives the arms, losers' sockets cut	`station_addons/pah_hedge.py` (main)	env `PAH_HEDGE_N=2` `PAH_HEDGE_CUTOFF_S=1.5` `PAH_HEDGE_MAX_CALLS=3`; a cut call may still be billed; tested offline only (`tests/test_pah_hedge.py`)
Hedged monitor calls: `claude.hedge: 2` Parallel staggered looks: `claude.parallel`	`harness/hedge.py` `verifiers.py:555-557`	first parseable answer wins; cost × N on the station, verdict interval ~9.7 s → ~4.9 s with 2 in flight (`FINAL_HARNESS.md`)

Pacing facts behind this (`docs/DECISION_MODEL.md` F1–F2, verified in §7.1): Dream's client goes **sync** whenever the median round trip is over **900 ms**, waiting for each 2.4 s chunk; sync chunks arrived every **3.54 s** (median) against **1.55 s** in async mode, so the arms stood still about 30 % of the time. The round trip is ~950 ms even with a fast uplink because roughly **650 ms is gateway overhead** we

cannot change: on chunks with ≥ 50 Mbit/s uplink the median RTT was 900 ms with a 293 ms model time (our recomputation: ~607 ms overhead; over all chunks RTT − serve − network has a median of 740 ms). Each request carries ~1.6 MB of images, so uplink matters: chunks at ≤ 9 Mbit/s had a median RTT of 2.2 s.

The Dream Labs engineer's advice behind hedging (8 Nov): "fire a new call every 1–2 s and use the first good answer" (`station_addons/pah_hedge.py:4-7`).

3.8 The 10 Hz live guard

`harness/live_guard.py`. A thread reads both wrist frames (`/frame/wrist_l.jpg`, `/frame/wrist_r.jpg`) in parallel at `rate_hz = 10`, each fetch with a 0.25 s timeout so a dead camera never holds up the live one. The detector's guard (`TableDetector.guard`) judges each **new** frame (SHA-1 of the bytes): an unchanged frame neither adds a yes nor refreshes `last_answer_t`, so a frozen camera shows up as **blindness** (`max_blind_s`, 20 s in `bot_v1`) rather than "all clear". Yes answers are counted **per camera**; `confirm_frames = 3` in a row on one camera send `/stop` immediately (0.3–0.5 s after the ball enters view), without waiting for the runner. Arming has a generation number so a judgement started before a disarm is dropped. The runner re-checks `tripped` around every motion command (`_motion`) and inside every wait (`_wait`, 1 s slices). The live guard only reads `/frame` and only sends `/stop`.

`--live` refuses to start a plan that has a `guard:` without `--detect plans/table_colors.yaml --live-guard` (`cli.py:130-136`; override `--allow-no-live-guard` for practice).

Known limitations (audit A06/A07 and this report's own test, §5.2.4): the guard returns "none held" if any readable wrist view is clear, so a missing right-wrist frame is masked by the left; and the white range on main is the phone-photo range.

3.9 The start-pose gate (side branches only)

`harness/start_pose.py` (on `start-pose-live-guard` and the cloud-sims family, **not on main**). Reference: the first frame of every task segment in the teleop episodes, keyed by the exact task text (`plans/start_poses.json`, built by `tools/start_poses.py`). Joint state comes from `GET /arms positions_rad` or the newest trip's observation. Before each stage the harness logs the distance to the recorded starts for that prompt; on stages marked `home_before` it homes once if any joint is more than **0.35 rad** from every recorded start (paper §4.7). Motivation: LIBERO-Plus reports the initial state as the largest single robustness loss for a VLA, and the FT diagnosis found only 9 of 69 unplug demos started within 0.3 rad of the home pose every live run starts from. The audit (A25) notes the gate is advisory on several failure paths.

3.10 Shadow-mode decisions (side branches only)

`harness/decide.py` (cloud-sims family) runs the decision model of §4 **next to** the runner and logs what it would have done ("Decide shadow mode", c2098dc). With default weights it reproduces the runner's pass decision exactly on 5,000 random look sequences (`tests/test_decide.py:48`). `harness/statespace.py` (hedged-calls → cloud-sims) is the same logic as a vectorised state-space system for hill-climbing thousands of weight sets over logged runs (§4.5). Neither ever made a match decision.

3.11 Observers and integrations

- **LiveKit observer** (`harness/livekit_observer.py`, ported from `codex/hill-climb-livekit` 569607e via 76cc686 and b3c27fb): subscribes to the room's camera tracks and can run the monitor on a stage; logs advisory verdicts; never part of the run. `relay-on-stitch` (f8771cd) mints a **subscribe-only** token from `LIVEKIT_API_KEY` / `LIVEKIT_API_SECRET` + `--room` when `LIVEKIT_TOKEN` is unset.

- **LiveKit viewer** (PR 9, `harness/livekit_view.py`): view-only camera card in the panel and standalone on port 8098; minted tokens have publish and publish-data off; `livekit` is imported on the viewer's thread only. Purpose: the rubric's +10 for showing LiveKit to the referee.
- **DimOS** (`integrations/dimos_skills.py`, `integrations/station_tools.py`): the harness's primitives as DimOS skills / MCP tools (`list_stages`, `start_stage`, `check_stage`, `recover`, `stop_robot`, `status`, `run_plan`, `plan_status`). All go through Dream's station API; there is deliberately no tool that moves a joint, homes or resets; `send_prompt` (free text) is off unless `allow_free_prompts=True`. Purpose: the rubric's +5 for Dimensional.
- **Watcher arm** (`codex/watcher-dimos` → main 6264ca9): `stage.watch` puts one arm's wrist camera on the work and re-sends a park prompt if the monitor says the watcher drifted (`watch_lost`, `runner.py:753-765`). Base Dream-1 does not reliably obey arm naming (§7.5), so this was never used in a match plan.
- **Panel** (`python -m harness.panel`, kaupau): starts a run (base or fine-tune, which stages), shows cameras, every prompt sent and why, the harness's own actions and monitor looks; sends `!prompt` / `!skip` overrides. Practice only.

3.12 Logs

`runs/<UTC timestamp>/events.jsonl` (one JSON event per decision: `stage_start`, `prompt_change`, `verdict`, `grip_assist`, `grip_pull`, `stage_locked`, `confirm_check`, `guard_tripped`, `station_restart`, …), the frames each verdict looked at, `segments.jsonl`, `manifest.json`, `summary.json`. `tools/label_rollout.py` joins these to Dream's own run record (`inputs.jsonl`) to label rollouts with stage, attempt, outcome and a clean task text — the bridge from harness runs to fine-tune data. `tools/bot_report.py` and `tools/trace_report.py` render run pages.

4. The decision model

Back to the index

The runner's rules are hand-set constants. To make them tunable, Darsh (with Claude) wrote them as a model in which today's rules are a special case: design in `docs/DECISION_MODEL.md` (branch `final-harness` and the cloud-sims family; it is the committed form of the local `HARNESS_DECISION_MODEL_2026-11-07.md`), code in `harness/decide.py` (337 lines) and `harness/statespace.py`, state-space write-up in `STATE_SPACE.md` (cloud-sims). **None of it made a match decision**: it ran in shadow mode only.

4.1 Runtime model of a stage

$$T_{\text{stage}} = T_{\text{start}} + \frac{T_{\text{robot}}}{u} + T_{\text{detect}} + T_{\text{decide}}$$

with u the share of time the arms actually move. On the station $u \approx 0.70$ because 2/3 of chunks were stop-and-wait (F1). The levers ranked by expected gain (`DECISION_MODEL.md` §1): L1 get Dream into async (network, light frames), L2 event-driven stage-done from local sensors (4–7 s per boundary), L3 never `/start` mid-match (2.9 s each), L4 skip VLM checks the gripper sensors can answer (~4 s per skippable stage), L5 faster judge.

4.2 Done evidence as recursive log-odds

State x_t: stage s, elapsed τ, time left $L = T - t$, evidence features $\phi(t)$, attempt count n.

$$\ell_t = \gamma\, \ell_{t-1} + \sum_i w_i\, \phi_i(t), \qquad \text{done} \iff \ell_t \geq \theta_{\text{done}}(s)$$

ϕ_i	Source	Weight today
monitor done (+1) / not, × confidence	VLM	$w_{\text{monitor}} = 1$; a no resets to 0
progress $p_t \in [0, 1]$	VLM	stall and regression only
gripper closed-on-object → open near the target	`/arms`	0 (`w_sensor_release`)
object at goal (detector box in the bin / on the tower)	detector	0 (`w_goal_box`)
plug count fell (cable-end boxes)	cable-end CNN	0 (`w_plug_count`)
same frames as last time	SHA-1	$\phi = 0$

Today's rule is the special case $\gamma = 1$ on agreement and 0 on disagreement, unit weight on the judge, $\theta_{\text{done}} = \texttt{confirm} = 2$, plus the 15 s "sticky done" window. Code: `DoneWeights`, `done_step`, `run_looks` (`harness/decide.py:38-102` on cloud-sims).

4.3 The fail hazard

$$h_t = \sigma\Big(b + w_f\, n_{\text{fail}} + w_r \max(0,\, p_{\max} - p_t - \delta) + w_s \frac{\tau - \tau_{\text{rise}}}{\text{stall}_s} + w_T \frac{\tau}{\text{timeout}_s} - w_{\text{on}}\, \mathbb{1}[\text{on target}]\Big)$$

The attempt ends when $h_t > \theta_{\text{fail}}$. In code (`FailWeights`, `fail_terms`, `fail_score`), each term g_k is a ratio that reaches 1 exactly when today's hard rule fires (consecutive failing looks / confirm, consecutive stalled looks / confirm, time since progress rose / `stall_s`, elapsed / `timeout_s`) and they are combined by `max` (today's tree: any rule fires) or `sum` (weak signals add up, for tuning).

4.4 Choosing an action: the Q-value and the value of time left

At a decision point, with $a \in$ {continue, re-prompt/rewrite, reset prompt, home + retry, skip, abort}:

$$Q(a \mid x) = P_a(\text{succ within } c_a)\, V_s \;+\; W(L - c_a) \;-\; \lambda_{\text{risk}}\, R_a$$

- V_s: the stage's points (first cable +10, extra +5, tower pass +20, +3/inch, ball +5) plus what it unlocks.
- c_a: time cost (re-prompt ~1 s; reset 5–20 s; home ~10 s + 2.9 s `/start`).
- R_a: risk (DQ from a false done or out-of-order touch, knocked tower, illegal ball). Default $\lambda = 50$.
- $W(L)$: expected points of the remaining stages with L seconds left, by backward dynamic programming on a 1 s grid over each stage's success-vs-time curve F_s:

$$W_i(L) = \max_{T \le L} \Big[F_i(T)\big(V_i + W_{i+1}(L - \bar{t}_i(T))\big) + (1 - F_i(T)) \cdot \begin{cases} 0 & \text{gate} \\ W_{i+1}(L - T) & \text{otherwise} \end{cases} \Big]$$

with $\bar{t}_i(T)$ the mean success time given success by T (`StageModel.mean_success_time`), and the argmax T being that stage's timeout for L seconds left (`plan_timeouts`, `decide.py:213-232`). The implemented action value is `action_value` (`decide.py:244-253`): $Q(a) = p(V_i + W_{i+1}(L-c)) + (1-p)W_i(L-c) - \lambda R_a$.

Stage success curve (`StageModel.F`, `decide.py:165-181`):

$$F_s(\tau) = p_{\max}\left(1 - e^{-\max(0,\, \tau - t_{\min})/\tau_c}\right)$$

4.4.1 The optimal-timeout condition

Keep going on stage s while its hazard of success, times its value, beats the marginal value of time elsewhere:

$$\frac{f_s(\tau)}{1 - F_s(\tau)}\, V_s \;\ge\; \frac{dW}{dL}$$

So each stage gets a timeout tied to its own success curve and to how much match is left: optional stages get short timeouts late in a match and longer ones early. This replaces a fixed `timeout_s`.

4.4.2 Fitted curves (simulation only)

`tools/fit_stage_curves.py` fits F_s by censored maximum likelihood over attempts; `plans/stage_curves_sim.js` (cloud-sims family) holds the fit from **Dream's YAM MuJoCo sim only**:

Stage	$p_{\max}$	$t_{\min}$ (s)	τ_c (s)	Episodes	Successes
unplug_cable	0.64	22.84	105.6	35	19
put_cable_down, release_before_stack, stack_two, plug_loose_cable	0.02 (floor)	0	292.84	19 / 9 / 8 / 5	0
watcher_into_place	1.0	5.71	2.81	—	—

The commit message (c2098dc) and the paper say "unplug p_max 0.64, none before ~23 s, **median 56 s**". **Not reproducible from the JSON:** with these parameters the time to half the ceiling is $t_{\min} + \tau_c \ln 2 \approx 96$ s (88 s if conditioned on success within 292 s). The 56 s may be the empirical median of the 19 successful attempts; the attempt-level data is not in the repo. Flagged in §13.

4.5 The pass-bar constraint: the α^k bound

A false done on a pass bar is a DQ. If each look has false-positive rate α and k independent looks are required:

$$P(\text{false done}) \approx \alpha^k$$

Choose $\theta_{\text{done}}(s)$ per pass bar so that $P(\text{false done}) \leq \varepsilon$ (e.g. 1 %), then minimise the expected detection delay. Example: $\alpha = 0.10$, $k = 2$ gives 1 %; $k = 3$ gives 0.1 %.

The independence assumption is the weak point. The stress sim found two DQs (`false_done_unplug` and `false_done_all`, same seed) where two false "cable out" verdicts from **two calls in flight** landed seconds apart on near-identical frames and passed `confirm: 2` (`FINAL_HARNESS.md`). Correlated looks make the effective k close to 1. The fixes are a minimum gap between confirming verdicts (`confirm_gap_s`, 3 s default in `statespace.py`, 0 on cloud-sims' runner per audit A03) and a confirming look from a different model (Sonnet `confirm_check` on the bot line). Audit A03 also notes that the 15 s sticky-done window lets a tower that falls and is re-placed pass.

4.6 Tuning objective (planned, not run)

$$J(\theta) = \mathbb{E}[\text{score}] - \mu\, P(\text{DQ}) - \nu\, P(\text{illegal ball})$$

over $\theta = \{w_i, \gamma, \theta_{\text{done}}(s), w_f, w_r, w_s, w_T, w_{\text{on}}, \theta_{\text{fail}}, \lambda, \theta_{\text{rw}}, \dots\}$, estimated in the stress sim with common random numbers (~200 sim matches per candidate), searched by CMA-ES or coordinate hill-climbing, then confirmed on Metal (best 2 vs today, 5 supervised matches each). Step 1 — calibrate the sim's evidence model from real logs — was never done, so this was never run.

The rewrite gate in the same formalism (`DECISION_MODEL.md` §4):

$$g_{\text{rw}} = w_{\text{wrong}}\mathbb{1}[\text{wrong object}] + w_{\text{stall}}\frac{t - t_{\text{rise}}}{\text{stall}_s} + w_{\text{risk}}P(\text{knock}) - w_{\text{churn}}\, n_{\text{rewrites}} > \theta_{\text{rw}}, \quad t - t_{\text{last prompt}} \geq \tau_{\text{cool}}$$

Arm choice from layout (`DECISION_MODEL.md` §3), never built:

$$c_a(o) = w_{\text{reach}}\frac{\|p_o - b_a\|}{R_a} + w_{\text{cross}}\mathbb{1}[\text{past midline}] + w_{\text{clear}}\sum_k e^{-d_k^2/\sigma^2} + w_{\text{busy}}\mathbb{1}[\text{busy}] + w_{\text{skill}}(1 - s_a)$$

4.7 State-space form (`harness/statespace.py`, `STATE_SPACE.md`)

State $x \in \mathbb{R}^{K\times 9}$ (`done, fail, stalled, guard_m, guard_l, pmax, since_rise, t, since_done`), input $u \in \mathbb{R}^{1\times 12}$ (`dt, v_new, v_done, v_same, v_failing, v_stalled, v_rise, v_guard_yes, v_guard_no, l_guard_yes, l_guard_no, obs_release`), K weight sets evaluated at once:

$$X_{k+1} = (\mathbf{1}_K \otimes (1 - \text{clip}(u_k R, 0, 1))) \odot (X_k A^\top) + \mathbf{1}_K (u_k B)$$

with $A = I_9$, B integrating `dt` into the clocks and counting guard yeses, R resetting on `v_rise` and guard noes, plus a switched jump map on verdict ticks (done/failing/blank) and a fixed-gain observer on the task estimate $\hat{z} \in [0,1]^5$ (progress, holding, at_goal, illegal_grip, cable_out):

$$\hat{z} \leftarrow \tfrac{1}{2}\mathbf{1} + \rho(\hat{z} - \tfrac{1}{2}\mathbf{1}), \quad \hat{z} \leftarrow \hat{z} + L_f H_f (y_f - \hat{z}), \quad \hat{z} \leftarrow \hat{z} + L_s H_s (y_s - \hat{z})$$

$\rho = 0.98$, $L_f = 0.25$ (local detector), $L_s = 0.8$ (VLM). With defaults it makes the same pass decision as `decide.py`'s reference on **3,000** random verdict sequences (`tests/test_statespace.py`); one tick over 1,000 weight sets takes < 5 ms (46 µs after the mac-arm64 optimisation for K = 1000).

4.8 What shadow mode showed

`Shadow` (`decide.py:287-337`) is fed every runner look and logs `shadow_*` events. The tests prove agreement with the runner on 5,000 random look sequences (`tests/test_decide.py:48`) and on a real `Runner` with a scripted monitor. No station run used shadow mode (the bot line on main does not contain `decide.py`), so there is **no field evidence** for or against it.

5. Every classifier and estimator

Back to the index

Verification marks used in this chapter: **[V]** recomputed or re-read from raw data/code in this report; **[D]** taken from an in-repo document or commit message whose raw data is not in the repos; **[U]** could not be verified (see §13).

5.0 Overview

Component	Decides	Model	Latency	Evidence	On main?
VLM judge ("monitor")	stage done, progress, status, holding, guard, slot hints	Haiku 5.5, Gemini 3.1 Flash-Lite, GPT-6.1 Terra (OpenRouter); Sonnet 5.5 confirms	2–9 s per call	60-decision benchmark, 25 configurations [V]	yes
Colour + shape detector	balls, cubes, sticks, hole direction	HSV thresholds + blob shape tests, no learned weights	guard on both wrist views 76 ms (236 ms before the first speed-up) on 7 Nov [D: report-inputs `HARNESS_SPEED_STATE_SPACE`]; 5.3 ms per wrist frame after the mac-arm64 rewrite (19.2 ms before) on x86 [D]	26 hand-labelled station frames [D]	old version only
Live illegal-ball guard	illegal ball between the fingers	detector on both wrist frames, 3 new frames in a row	10 Hz, stop in 0.3–0.5 s [D]	0 trips on 2,145 station wrist frames [D]; **main's config misses both held-white test frames [V]**	yes

Component	Decides	Model	Latency	Evidence	On main?
Cable-end classifier	USB-A end / USB-C end / neither	orange-blob proposals + MobileNetV3-small, 96 px crops, ONNX via `cv2.dnn`	12 ms per 640-px frame [D]	137 held-out wrist crops from 25 runs [D]	**no**
Gripper-width rules	holding / closed on nothing / regrip / pull	thresholds on `/arms` readings	< 1 ms	20 Hz joint logs of 101 rollouts [D]	yes
Joint fault nets	joint in fault now / within 0.25 s	dilated causal TCN or transformer encoder, 16-step windows	0.83 ms for 12 joints (numpy) [D]	injected faults; 31 real rollouts [V]	**no**
Rule joint observer	limit approach, breach, overspeed, torque saturation, divergence, oscillation, overheat	hand rules	—	90.7 alarms/joint-hour on untouched real runs [V]	**no**
Start-pose gate	arm far from every recorded start	L∞ joint distance to teleop starts per prompt (0.35 rad)	< 1 ms	recorded starts	**no**
USB port finder	free port, seated plug	strip template fit + patch CNN	—	**planned, not built**	no

The design rule: slow, general judgements gate progress; fast, narrow ones guard safety and veto the slow ones.

5.1 VLM judges

5.1.1 What a look sends and receives

`MonitorVerifier._build` (`harness/verifiers.py:664-`) composes one request per check:

- **Images.** With `claude.stitch: true`, one labelled grid of the stage-start frame, up to `history_frames` (4 in `bot_v1`) earlier overhead frames and the newest frames of every camera (`stitch`, `verifiers.py:58`), with the local detector's boxes drawn on (`TableDetector.annotate_frames`). Frames are light 640-px JPEGs.
- **Text.** The plan's `system_prompt` (station rules and lessons: "the gripper sensors are measured and beat the images"; "every cable's far end lies loose on the table from the start"; …), the stage `success` question, `expect`, `camera_notes`, the done stages, operator notes (including a failed `confirm_check`'s reason), and a **grip note** from `/arms` (`runner._grip_note`, `runner.py:506-530`).
- **Reply** (JSON): `done`, `confidence`, `progress` (0–1), `status` (`progressing|stalled|failing`), `holding` (arm → object), `guard` (illegal ball held?), `tower` (bottom → top), `scene_ok`, `recovery` (menu index), `next_prompt`, and in composed mode a slot hint. A stage counts as done only for `done: true` with confidence ≥ 0.7 (README).

5.1.2 Scheduling

- `claude.parallel: N` keeps N calls in flight, launched at least latency/N apart. On the station this halved the time between verdicts from ~9.7 s to ~4.9 s with N = 2 [D: `FINAL_HARNESS.md`]. `bot_v1` uses `parallel: 3`.
- `claude.hedge: N` sends N identical requests per look and keeps the first that parses (`harness/hedge.py`); `bot_v1` uses 2.
- Backends: Anthropic API (`ANTHROPIC_API_KEY`), Claude subscription through `claude -p` (no tools, no MCP, API key stripped from its environment), or OpenRouter (`--monitor-model google/gemini-3.1-flash-lite`). In `bot_v1` the frequent looks go to Flash-Lite on OpenRouter and `confirm_model` / `writer_model` (Sonnet 5.5) stay on the Anthropic API. On 8 Nov evening the OpenRouter key hit its limit ("every look was HTTP 403"), so the panel was switched to Haiku on the Anthropic API (commits 8ec93b1, a516488).
- Alternating/race judges (`race`, `alternate`) were added for the cloud runs (65ce121).

5.1.3 The judge benchmark (`tools/bench_judges.py`, `hedged-calls` → cloud-sims)

Same frames for every configuration: the real `MonitorVerifier` request built from station photos and two MuJoCo scenes with known answers, translated to OpenRouter's Chat Completions format (`docs/OPENROUTER_RESEARCH.md`). 60 decisions per configuration, 30 of them the guard question ("is a gripper holding the illegal ball?"). Results in `task_simulation/results/judge_bench_20261108/bench_fair/*/s` [V], sorted by accuracy then latency:

Config (backend:model, reasoning)	Done acc. %	Guard acc. %	Parse fail %	Transport fail %	p50 s	p90 s	$ per 60 decisions
openai_gpt-6.1-terra_rlow	100	100	0	0	3.72	4.85	$0.066
openai_gpt-6.1-sol_rlow	100	100	0	0	3.74	5.97	$0.106
subscription_claude-sonnet-5-5	100	100	0	0	7.14	9.34	subscription
google_gemma-4-26b-a4b-it_rmedium	100	100	0	0	11.51	24.36	$0.024
google_gemma-4-26b-a4b-it_rlow	100	100	0	0	11.95	30.78	$0.027
openai_gpt-6.1-terra_rnone	98	97	0	2	2.78	3.14	$0.047
anthropic_claude-haiku-5.5_rlow	100	93	0	0	4.50	5.38	$0.032
anthropic_claude-haiku-5.5_rmedium	100	90	0	0	5.03	6.05	$0.035
subscription_claude-haiku-5-5	100	90	0	0	8.86	11.01	subscription
google_gemma-4-26b-a4b-it_rnone	97	90	0	0	2.36	3.93	$0.007
openai_gpt-6-luna_rlow	95	83	0	0	6.53	8.52	$0.015
google_gemini-3.1-flash-lite_rlow	80	97	0	0	2.39	3.05	$0.058
anthropic_claude-haiku-5.5_rnone	100	77	0	0	2.71	3.02	$0.021
google_gemini-3.1-flash-lite_rnone	90	83	0	0	1.81	2.27	$0.041
google_gemini-3.1-flash-lite_rmedium	80	87	0	0	3.53	4.96	$0.092
openai_gpt-5.6-luna_rlow	98	67	0	0	5.79	10.33	$0.031
mistralai_mistral-small-2603_rnone	88	63	0	0	2.44	3.04	$0.017
meta-llama_llama-4-scout_rnone	87	63	10	0	4.19	8.08	$0.023
openai_gpt-5-nano_rlow	77	67	0	0	8.06	10.80	$0.020
openai_gpt-5-nano_rmedium	78	63	0	2	21.37	28.49	$0.068
openai_gpt-5.6-luna_rnone	73	67	0	2	2.90	3.52	$0.015
meta-llama_llama-4-scout_rlow	77	53	15	0	4.14	7.54	$0.022
mistralai_mistral-small-2603_rlow	58	63	3	8	7.47	22.00	$0.074
openai_gpt-6-luna_rnone	62	7	0	37	2.07	2.85	$0.004
openai_gpt-6.1-sol_rnone	0	0	0	100	0.57	0.83	$0.000 (every call failed)

Readings:

- Done questions are easy for strong models; **the guard question separates them**. GPT-6.1 Terra with no reasoning: 98 % done, 97 % guard at a 2.78 s median, **$0.047 per 60-decision match** [V]; with low reasoning 100 %/100 % at 3.72 s, $0.066 (the paper's "about $0.07"). Haiku

5.5 low 100/93 at 4.5 s; Sonnet 5.5 (subscription) 100/100 at 7.1 s. Gemini Flash-Lite was the fastest usable judge but missed stage completions (80–90 % done).

- With 30 guard questions a perfect score has a 95 % lower bound near 88 % (rule of three: $1 - 3/30 = 0.90$; Wilson ≈ 0.886), so the ranking is coarse.
- A separate 120-decision run (`bench_fair/TABLE.md`, hedge 1) caught a held 8-ball with Haiku 5.5 only **55 %** of the time, Gemma 4 85 %, Flash-Lite 70 % [V]. **No judge stands alone on safety**, which is why the guard decision is on the local detector.
- The paper says "30 model and reasoning-level configurations"; the committed `bench_fair` holds **25** (24 usable) [V]. The other five are not in the repos [U].

5.1.4 Judges in the stress sim

The stress sim's scripted judge answers from the table state at the newest frame's time after a virtual latency. Judge **speed** is the main lever on points because a slow judge rarely reaches the balls: at Haiku's measured 6.5–13 s, configurations binned 0–0.09 balls per match; at 4–6 s ("fast"), 2.4–3.4 [V, §7.2]. A fast judge without the live guard lost **28 of 185** matches to the illegal ball [V]. **The real judge was never run against the MuJoCo referee**, so false and late done rates on simulated frames are unmeasured.

5.2 The HSV colour and shape detector (`harness/detect.py`, `plans/table_colors.yaml`)

5.2.1 Algorithm

1. Shrink to `WORK_W = 640` px; HSV on Pillow's 0–255 scale (hue 0–255, not 0–360).
2. Overhead only: ignore blobs outside `table_roi` (`[0.0, 0.32, 1.0, 1.0]`: people and laptops behind the table).
3. Per colour class (`classes:` in the YAML), threshold `[lo, hi]`, morphological close and hole fill, label connected blobs, keep blobs between `min_frac` and `max_frac` of the image.
4. Classify each blob by **shape**, not colour (`_kind`): compact round → ball; compact square → cube (an upright stick from above looks the same); long and thin → stick lying down. `kinds:` maps shape → role (`avoid` / `target` / `info`).
5. Overhead hole axis (`_hole_axis`, `HOLE_SURE = 0.6`): which way a cube's hole runs; an upright stick standing in a cube's hole (`_mark_upright`).
6. **Guard** (`TableDetector.guard`): on wrist views (`guard.wrist_roles`), any `avoid` ball whose centre lies in `grip_roi` and that fills at least `grip_min_frac` of the view → "held".

5.2.2 Station retune (branch `fix/classifiers-8nov`, PR 7) [D]

Checked against hand labels on Dream's own 8 Nov frames (640×360). An object counts as found when a box of the right class covers its labelled centre.

	Tuning (14 frames) before	after	Held-out (12 frames) before	after
Balls found	24/46	42/46	**20/27**	**26/27**
Ball boxes that are balls	77 %	100 %	83 %	**100 %**
Blocks found	32/40	39/40	16/23	**23/23**
Block boxes that are blocks	86 %	95 %	80 %	**92 %**

All 2,010 overhead frames of 8 Nov: **the white bin was boxed as a white ball in 1,286 frames before, 0 after**; merged "blue stick" boxes 0.36/frame → 0.11. The live guard tripped on none of 2,145 wrist frames before or after (none shows a held white ball, so this measures false alarms only).

What changed: yellow now spans H 30–70 from S 50 (the pale yellow-green ball sat at H 56–65, between the old yellow 30–55 and green 72–125); yellow has no cubes on this table so a compact yellow blob is a ball (one centred on a cube is an upright stick); a ball-coloured blob ≥ 50 px with elongation ≤ 2.1 and roundness ≥ 0.28 is a ball (half-lit balls read as crescents); overhead, a ball-colour blob over 0.6 % of the image (the bin) is never a ball (`OVERHEAD_BALL_MAX_FRAC`); a cube-colour blob ≥ 14 px wide is split at a dip in its distance ridge (two touching cubes). Still missed: balls

inside the bin (1 of 4 labelled), a white ball pressed against the bin. The labels and frames stay outside the repos because the frames show people. Regression frames: `tests/data/table_real/`, `tests/test_detect_real_table.py`.

5.2.3 The white ball on the wrist cameras (`fix/guard-real-frames`, dcd76c5) [D]

On the right wrist camera the white ball reads H 147, S 47, V 169–227 (median 193); the phone-photo range required **V ≥ 225**, which matched 0.1 % of it. The retune moved white to `[[130, 22, 165], [170, 75, 255]]` with `not_inside: [blue]` (a blue cube's pale rim also falls in that range), and tightened the grip zone to `grip_roi: [0.33, 0.6, 0.67, 1.0]`, `grip_min_frac: 0.015` (the old box let balls lying past the fingertips through: 3 false trips in one take).

5.2.4 Our check: main's live guard on the committed real frames [V]

`fix/classifiers-8nov` commits six station test frames (`tests/data/guard_real/`): two **composite** frames with a white ball pasted between the right fingers, three real frames with a white ball visible but not held, one overhead. Running `TableDetector.guard` with each branch's own `plans/table_colors.yaml`:

Frame	main @ acc1672	fix/classifiers-8nov @ 3202983
held_white_composite_rw_ep39_2730	**False** ("none held")	True (2.2 % of the view)
held_white_composite_rw_ep39_3720	**False**	True
not_held_white_rw_ep40_1743 / 3366 / 3465	False ×3	False ×3
overhead_green_ep39_0	None (no wrist view)	None

Main's guard configuration does not see a held white ball on these station frames; the retuned configuration does. Caveats: the held frames are composites, and we do not know which `table_colors.yaml` the station laptop actually loaded in the final runs. This corroborates audit finding A07 and is the most important open safety item (§12).

5.2.5 Speed (mac-arm64) [D]

Exact-equivalence rewrite (`tests/test_guard_equivalence.py` compares verdicts, boxes and annotated JPEGs with the reference detector): morphology without scipy's iterated loops, crop before labelling, per-blob reads inside `find_objects` boxes, a guard that only runs colours that can trip it, uint8 HSV planes, a stitch tile cache.

Component	before p50 / p95 ms	after p50 / p95 ms	Speed-up
Guard, one wrist frame (x86 laptop)	19.2 / 21.7	5.3 / 7.4	3.6×
Fast-loop tick, 2 frames	38.3 / 42.5	11.2 / 13.9	3.4×
Monitor annotate, 3 frames	137 / 139	27.0 / 29.2	5.1×
Whole monitor request build	152 / 158	32.9 / 36.1	4.6×
Guard on GitHub `macos-15` M1 VM	34.1	5.7	6.0× [V: `bench_results/mac_bench_gha_macos15.json`]

5 minutes of match load: process CPU 107 % → 26 % of one core; overruns 1–12 → 0. **The real MacBook Air was never benchmarked** (no Mac was available), and the branch was never merged.

5.3 Cable-end classifier (`harness/cable_end.py`, `models/cable_end.onnx`)

Not on main. On `start-pose-live-guard` (ae30720) and the cloud-sims family; retrained on `fix/classifiers-8nov`.

Architecture: (1) proposals = compact orange blobs (the anodised plug shells; the long thin cable and slivers are rejected; skin, faces and the orange golf ball pass and are left to step 2); (2) a 96×96 crop (`CROP_SCALE = 2.6`) classified `usb_c | usb_a | none` by **MobileNetV3-small**, a ~6 MB ONNX

file run by OpenCV `cv2.dnn` with a fixed batch of 8 (OpenCV 5.0 kills the process when a net is reused with a larger batch), so the match laptop needs no deep-learning runtime. Boxes (magenta USB-C, amber USB-A) are drawn on the monitor's frames; it never touches Dream's inputs. 12 ms per 640-px frame [D].

Training (`tools/cable_end/train.py`): ImageNet-normalised crops. 7 Nov model: **891** hand-labelled crops from the teleop videos and table photos, split by source video: 94 % held-out accuracy; at p ≥ 0.7 USB-C P 0.70 / R 0.82, USB-A 0.96 / 0.84 (17 and 51 test crops) [D: commit ae30720]. Blender renders (`tools/cable_end/render_cable.py`: random pose, lighting, lens, backgrounds from station frames) **did not help**: synthetic + real 92 % vs real-only 94 %, and worse on photos, so the shipped model was real-only. (The module docstring still says "Blender renders plus hand-labelled crops"; the commit message is the authority.)

8 Nov retrain [D: `docs/CLASSIFIERS_8NOV.md`]: 700 wrist crops from 89 runs hand-labelled, **478 usable** (115 USB-A, 82 USB-C, 281 neither; 222 unsure left out). Held out by run (25 runs, 137 crops), p ≥ 0.7:

Training data	USB-C P	USB-C R	USB-A P	USB-A R	Accuracy
7 Nov: 891 video and photo crops	0.83	0.91	1.00	**0.68**	0.91
+ 478 station wrist crops	**0.92**	**1.00**	0.95	**0.84**	**0.96**

The shipped model is retrained on all 1,369 crops (15 epochs, one seed). A hand check of every box on held-out run 103905 found about 62 of 64 right (the old model missed ~13 flat USB-A ends there), with one new false box on the strip's red switch. The 7 Nov model called a USB-A end lying flat "USB-C" at 0.71–0.95. Neither model finds ends on the overhead camera (a shell is a few pixels wide).

5.4 Gripper-width rules

From the station's 20 Hz joint logs (prompt study §7.6) [D]: jaws stop at **4.5–14 mm on a plug body**, ~3 mm on a wire or nothing, ~48 mm on a golf ball, 25–55 mm on a cube. In `/arms` units on this station (0–1 opening):

Reading	Meaning	Where
≤ 0.06	right gripper shut on nothing: "NOT holding anything" (overrides any judge claim)	`runner._grip_note`, `runner.py:520-521`
0.06–0.15	a thin plug or nothing; the wrist image decides	`runner.py:522-524`
commanded closed but stopped > 0.15	holding something	`runner.py:527-529`
commanded > 0.15	opening	`runner.py:518-519`
`hold: [0.062, 0.20]`	grip-assist band for a plug (0.053 was empty live; a pull from 0.27 lost it)	`plans/bot_v1.yaml:112`
two readings ≤ `hold[0]` after being open	regrip	`runner.py:866-867`
`empty_below: 0.04` after squeeze/lift	lost	`plan.py:91`

Three uses: (1) "closed fully on nothing" overrides any judge `holding`; (2) a closure on nothing at the plug triggers the regrip skill; (3) a gripper stopped on an object blocks re-prompting (and `needs_grip` stages ignore a done verdict until something was held). The station grip hold torque was raised from 0.5 → 1.5 → **2.0 N·m** on both arms at 19:14 UTC 8 Nov (`nuc/station.yaml` note on `station-tuning`); grippers latch off at 55 °C.

5.5 Joint-fault detectors (cloud-sims family; encoders on `joint-fault-encoder`)

Not on main and never used live. The arms have no brakes, so a per-joint fault monitor was built as a safety net.

Inputs (`docs/JOINT_FAULT_NET.md`): per joint, the last 16 readings (0.8 s at 20 Hz) of 8 features — position in range (−1..1), margin to the nearer limit / 30°, velocity toward it / 200 °/s, tracking error / 10°, commanded velocity toward the limit / 200 °/s, measured torque / limit, commanded torque / limit, their difference. **Outputs**: two flags, now (in fault) and soon (now or within 0.25 s); an alert needs 2 readings in a row over threshold. Runtime: numpy, 0.83 ms for all 12 joints per reading (~2 % of a core at 20 Hz).

Data. Real policy motion with injected faults: normal, near miss (fast move stopping 1–12° short: no fault), limit run, blocked/crawling joint, oscillation (1.5–6 Hz), overload, overspeed. Labels come from the resulting signals, not from rules.

5.5.1 Simulated motion with torque (cloud-sims, a73b628) [D]

6 sim matches (Dream driving both YAM arms), 72 joint series, 413k samples; split by match file (4 train = 16,000 clips, 1 val, 1 test). 15 architectures, 3,000 steps each, AdamW + one-cycle, BCE with a balanced positive weight; threshold for ≤ 2 false alarms/joint-hour on val; ranked by "timely recall" (alarm ≤ 0.25 s after onset, mean over 5 fault kinds).

Model	Weights	Val timely	AUROC soon
logistic	258	0.00	0.600
MLP 128-64	24,898	0.96	0.979
GRU 32×2	10,434	0.95	0.979
Transformer d32×2	18,018	0.98	0.980
TCN 32 ch × 4 layers (dilations 1-2-4-8, kernel 3, residual, last-step + mean pooling)	**10,530**	**0.98**	**0.980**

Top 3 over 3 seeds × 6,000 steps: TCN 0.987 ± 0.001, transformer d32×2 0.985 ± 0.004, d32×1 0.953 ± 0.013. Test file at ≤ 1 false alarm/joint-hour (soon 0.998, now 0.95):

	FA / joint-h	Timely	Limit run	Blocked	Oscillation	Overload	Overspeed
TCN (soon)	**0.2**	**0.91**	1.00	0.94	0.92	0.99	0.68
Rules	197	0.50	1.00	0.01	0.22	0.34	0.93

The rules' false alarms are almost all `limit_approach`: its constant-acceleration projection fires on every fast move toward a limit. Overspeed bursts last 1–3 readings, so the 2-in-a-row rule misses a third: keep both.

5.5.2 Real Metal rollouts, torque-free (`joint-fault-real-check` → `joint-fault-encoder`) [V]

`tools/joint_fault_real.py` on copies of the NUC's `last-run.csv` files (20 Hz measured and commanded joints, both arms, **no torque**, so the torque features are zeroed and overload is dropped). Split by run: 31 rollouts with enough ticks → **train 21 (5.07 joint-h), val 5 (1.38), test 5 (0.83)**; 3,000 steps, seed 0. Verified against `harness/models/encoder/sweep_real.json` (created 2026-11-08T20:29:19Z, `fa_budget` 2.0):

Model	Weights	Val timely	Blocked (timely)	Overspeed	Val FA / joint-h	Alarms on untouched test runs
TCN 32×4	10,530	0.75	**0.20**	1.00	1.9	0
Encoder d16×1	2,690	0.67	0.42	0.36	1.9	0
Encoder d32×1	9,474	0.86	0.67	0.90	1.7	0
Encoder d32×2	**18,018**	**0.87**	**0.78**	0.84	**1.2**	0
Encoder d64×2	68,802	0.91	0.79	0.91	1.9	0
Encoder d64×4	135,746	0.92	0.85	0.90	1.9	0

Model	Weights	Val timely	Blocked (timely)	Overspeed	Val FA / joint-h	Alarms on untouched test runs
Rule observer	—	—	—	—	—	**90.7** (limit_approach 87.0, tracking_divergence 3.6)

Important nuance: the "blocked-joint recall 0.78 vs 0.20" in the brief is **timely** recall (alarm ≤ 0.25 s after onset). Plain recall on blocked joints is ≥ 0.996 for every model in the JSON; without torque the TCN still detects a blocked joint, just late (median latency 0.35 s vs 0.20 s for d32×2). The afternoon `--quick`/first sweep (different, smaller split) gave d32×2 0.96 val timely, 0.87 blocked — the numbers differ because the split did.

No model alarmed on the 0.83 untouched test joint-hours, so that column cannot rank them. The rule observer fires ~90–100 times per joint-hour on real runs because `shoulder_lift` and `elbow_flex` rest at their 0° soft limit at home and every move back home projects past it. One seed, small val/test sets, all faults injected: **none of these models has seen a real fault.** Files: `harness/models/encoder/joint_fault_<name>.json` (weights, normalisation, threshold, split, metrics), loadable with `FaultModel.load(...)`; the runtime zeroes torque when the file says `torque_free: true`. The default `harness/models/joint_fault_net.json` is still the sim-trained TCN with torque, and `JointTelemetry` loads it only when its file says it beat the rules on held-out traces (`beats_baseline`).

5.6 Start-pose gate

See §3.9. Reference starts per exact task text (`plans/start_poses.json`); 0.35 rad threshold; logs a `start_pose` event before each stage and homes once on `home_before` stages. Evidence: only 9 of 69 unplug demos start within 0.3 rad of home (§9).

5.7 Planned: USB port finder (`docs/USBC_PORT_CLASSIFIER_PLAN.md`, cloud-sims family)

Not built. A template fit to the strip's four rigid ports gives port indices that survive partial occlusion; a small patch classifier labels each port empty / occupied / occluded; "seated" requires the judge plus three consecutive "occupied" frames. Open question recorded in `docs/STATE_MACHINE.md`: the real strip's USB bank may be four USB-A ports (no USB-C), while the rebuilt sim has USB-C + 3 USB-A; confirm with the operator before writing plug-in templates.

6. Simulators

Back to the index

Four simulators exist, plus two offline benches. They test different things and must not be mixed.

Simulator	Where	Policy	Judge	Clock	Tests	Blind spot
Virtual-clock stress sim	`experiments/stress_match.py` (main and every branch)	rate-based table model	scripted from table state at frame time	virtual (5-min match in ~1–5 s)	harness logic under faults	no physics; judge never errs unless told; unplug p = 0.8/try is far above reality
MuJoCo station stand-in, v1 hub table	`sim_station.py` + `sim_scenes/hill_climb_table.xml` (main)	**live Dream-1** API	oracle over `/objects`	real time (or sim time on `prompt-engine`)	policy + harness on a physical table	YAM arm, not Metal; stand-in objects; white balls legal
MuJoCo, rebuilt realistic table	same files on **`cloud-sims`** (9c89ae3, 0f47c0d)	live Dream-1	oracle or API judge	real or sim time	as above, closer to the station	YAM arm; no clean comparison runs (credits ran out)

Simulator	Where	Policy	Judge	Clock	Tests	Blind spot
Kaushik's MuJoCo station sim	`sim/` on `sim-and-move-fix`	live Dream-1	rubric eval	real time	wrist-camera calibration, gains (`sim/calib/`)	abandoned branch
Metal driver bench	`station_addons/benchmetal_tick.py`, `fake_damiao.py` (`station-tuning`)	replayed rows	—	—	Dream's Metal driver on a fake Damiao bus	no physics beyond the bus
Metal replay sim	`station_addons/benchmetal_replay.py`, `metal_plant.py`, `sweep_replay.py` (`station-tuning`)	replayed `commanded_r` rows	—	—	gravity-sag and EEF fixes for the fine-tune (§9)	open loop, no contact

6.1 The virtual-clock stress simulator (`experiments/stress_match.py`)

The real `Runner` and real `MonitorVerifier` run **unchanged**. Around them:

- **`VClock`**: virtual time in 0.25 s steps (`STEP`); every harness sleep advances it, so a judge call returns only after its simulated latency and the races between checks, verdicts and the policy happen as they would live.
- **`Table + Behaviour`**: cables in/out of the hub, what each gripper holds, the tower, the bin. Each task prompt (T1–T6 of TELEOP_PLAN) has a try time and a success chance. Switches: `releases` / `open_frees` (the policy won't let go of the plug), `black_p` (picks an illegal ball), `reach_across_p` (brushes a block during unplug = DQ), `knock_p`, `unplug_never`.
- **`SimStation`**: the station API in-process with `Fault(at, kind)` and `refuse={prompt substring: n}` using the exact error types and messages from `data/station_nuc_20261107/robots-inference.jsonl` (DA-4000, ControlLoopError, LinAlgError, DA-5000, dead arm).
- **`SimFrames`**: frames carry their capture time: freeze, empty, corrupt (non-JPEG) and lag (`frame_lag_s`).
- **`Scripted`**: replaces only the HTTP call to Claude. It finds the stage in the real prompt the harness built and answers from the table **as it was at the newest frame's time**. Options: `false_done`, API `outages`, 400 on a corrupt image, `fast_guard` (stands in for the local detector).
- **`referee()`**: rubric v4 from the table's event log: points, DQ (block touched before a cable is out, ball before the tower passes), illegal ball in the bin = loss.

Scenarios. The committed `SCENARIOS` dict has **24** entries on every branch (`--list`): happy, unplug_never, slow_unplug, no_release, stuck_cable, false_done_unplug, refusal_at_start, refusal_twice, starve_mid, linalg_mid, gateway_x3, refusal_put_down, arm_loss, api_outage_30s, api_outage_balls, black_ball, black_ball_slow_monitor, black_ball_lagged, black_ball_local_guard, black_ball_local_guard_api_down, frozen_cam, empty_frames, corrupt_first_frame, reach_across. The **37-scenario** sweep cited everywhere (`FINAL_HARNESS.md`, the paper) also contains api_flaky, black_heavy_lagged, black_heavy_lagged_local, calibrated_base, calibrated_base_faults, calibrated_ft, false_done_all, frozen_long, knocky_tower, realistic, realistic_faults, refusal_storm and slow_everything; those 13 were defined in a local driver (`results/stress/final_20261108/jobs.txt` runs `--tree <Windows scratch worktree>`), **not in any committed file** (§13). The `calibrated_*` scenarios use base Dream-1's real unplug rate (~1 try in 6) and are the realistic rows.

CLI: `-s/--scenario` (repeatable), `--seeds`, `--seed0`, `-j/--jobs`, `--out FILE.jsonl`, `--list`, `-v`, `--latency S|LO-HI`, `--plan`. `experiments/live_stress.py -s NAME --speed 10` replays one match in the browser (port 8765); `experiments/sweep_variants.py` A/Bs plan variants over every scenario at a given latency.

Results are in §7.2.

6.2 The MuJoCo station stand-in (`sim_station.py`)

A drop-in replacement for Dream's station agent: the same HTTP routes on 8091 and the same debug view on 8093, on **Dream's MuJoCo YAM example** (`dream_examples/yam_mujoco` from `pip install dreamlabs`), using Dream's own control loop (camera windows, `get_action`, `validate_chunk`, 20 Hz stepping) and the **live Dream-1 API**. The harness cannot tell it from the station. Extra routes:

`/objects` (ground truth), `/joints` (cloud-sims), `/live` (three policy cameras, a ground-truth top-down map, the referee, station state and round trips).

Modes: `--policy dream [--checkpoint ...]`, `--policy scripted` (small sinusoidal chunks, no key, no network), `--scene hill_climb`, `--no-realtime`, `--viewer` (MuJoCo's 3D window; `mjpython` on macOS), `--open`. Headless: `MUJOCO_GL=egl` (GPU) or `osmesa` (software).

`experiments/sim_match.py` runs the unchanged harness and a plan against it with an **oracle verifier** (reads `/objects`), reports "not met yet" as can't-tell like the real monitor, and scores with the referee. `--clock sim` (77dce31, on `prompt-engine`, default there) runs the harness on simulation time, because the cluster's Inspiron ran the sim at ~0.28× real time: a 295 s wall-clock match gave the robot 79–87 s of motion, so every timeout was ~3.5× too short. `--filler oracle|default|plan` picks the prompt engine's slot filler.

6.2.1 Table v1 (7 Nov; main)

`sim_scenes/hill_climb_table.py` on main: a stand-in. A silver four-port hub (two plugged, two loose cables), ten plain blocks, six **white golf balls counted as legal** and black 8-balls illegal, a wire bin; the hub placed **in front of the right arm** "as the plan's prompts assume". It got the rules right and the scene wrong. Audit A17: "main's MuJoCo table treats white balls as legal". The cloud runner translated ball colours in the plans in memory for this table (`cloud/run.py`).

6.2.2 The rebuilt realistic table (8 Nov; `cloud-sims` only)

Commits 0f47c0d (11:45 UTC) and **9c89ae3** (12:00 UTC), modelled on station photos:

Object	Model
Power strip	ALESTOR 12-outlet 4-USB surge protector at catalogue size 25.7 × 9.7 × 3.0 cm: black body, two raised 6-outlet panels, red switch, lights, cord; **USB-C + 3 USB-A** slots in a row at the robot end; plugs go in vertically
Cables	USB-A plug, orange shells, black reliefs, a 7-link ball-jointed orange cord, USB-C plug with a tip site; the referee seats USB-A in a free A port or the USB-C tip in the C port
Blocks	6 cm cubes with a 2.8 cm through hole (blue, magenta, blue); yellow 2 × 2 × 10 cm square stick; magenta 2.5 × 11 cm round stick; tower height over every geom of a block
Basket	white, slotted, ~26 × 18 × 9 cm
Balls	green, yellow, orange legal; **white illegal** (`eight_0`), a black 8-ball (`eight_1`) illegal; `SIM_BLACK_8BALLS=N` adds more
Table	light laminate, tuned so the white ball stays detectable; `plans/sim_table_colors.yaml` measured from a segmentation render

Rendering: three 4096-px shadow maps and 4× MSAA cost ~320 ms per three-camera capture under software rendering, so physics ran at 0.55× the match clock; `apply_render_quality` drops shadows and MSAA under software rendering (~50 ms per capture, real time); full quality with a GPU (EGL on cluster nodes). `tools/render_bench.py` measures it. `paper/render_sim_table.py` (task_simulation `paper`) renders this table with no policy and no network.

6.2.3 Which results came from which table

Result	Table	Branch at run time
25 cluster matches, 7–8 Nov (`results/cluster_mujoco/`)	**v1** hub table	main / codex sim-table line
Sim-time template-engine matches (six matches 42, 42, 41, 15, 15, 15) and the relay-order batch (12/12 first unplug)	**v1** (they branched from main)	`prompt-engine`
Stage curves in `plans/stage_curves_sim.json`	v1 (Dream's YAM sim runs before 9c89ae3)	cloud-sims
Cloud round 1 (r1-a/b/c/E)	**both**: it ran while the table was being rebuilt; its saved frames show both versions	cloud-sims images `cloud-v*`
Paper teaser figure (b, c)	rebuilt	`paper/render_sim_table.py`

No clean comparison runs exist on the rebuilt table, because the Dream credits ran out (§10).

6.3 The rubric referee

`sim_scenes/hill_climb_table.py` scores rubric v4 from object poses each tick: `unplugged`, `seated`, `tower_in` (height in inches), `stage1_at/stage2_at` (pass-bar times), `golf_in_bin`, `eight_in_bin`, `first_touch` per object (for DQs: e.g. `{"plugged_1": 51.95, "loose_0": 161.05, "world": 204.6}`), `dq`, `loses`, `points`. Tests: `tests/test_hill_climb_scene.py` (places objects by hand and checks scoring; needs `dream_examples`). Audit A15/A18: the simulated tower gate does not establish a supported, stable tower, and the bin check counts objects below the table and does not require rest/release (sim-only issues, not re-verified).

6.4 The home cluster runner (`cluster/`)

`cluster/Dockerfile` + `cluster/run_matches.sh` (main; `run_stress.sh` in task_simulation). The head node (an Inspiron: 8 threads, Intel iGPU, no CUDA) holds the image and a shared folder. `run_matches.sh W M LABEL` gives each of W workers its own container (own ports), mounts the Dream key read-only from `~/.dreamlabs/credentials`, and staggers start-up by 20 s; about 3 MuJoCo workers fit. `cluster/run_stress.sh N J LABEL [scenarios]` runs the offline sweep over N seeds on J cores; `SEED0=` splits seed ranges across nodes. On cloud-sims, `cloud.cluster` (c2fe683, 77fa186) runs the **same image and entrypoint** as GCP on the home cluster (push, keys, run, status, pull, stop), running containers as the node user with readable sources and keys via gcloud without a shell.

6.5 The Google Cloud runners (`cloud/`, branch `cloud-sims`)

`cloud/README.md`, `COST.md`, `RESEARCH.md`, `docs/GCP_CONSOLE_GUIDE.md`.

- `run.py` wires Runner, station, light frames, detector, live guard, start-pose gate and read-only lessons together without an operator console; records the effective plan, its hash, the original plan hash, versions, skips and the colour adaptation in the manifest.
- `judge.py`: `--judge oracle` (the monitor's two-call async scheduler with referee answers captured at request time and uniform 6.5–13 s delays; unknown questions stay unknown) or `--judge api` (the real Claude monitor on the API, never the subscription CLI) or OpenRouter judges (race/alternate, hedged). Oracle and API differ in perception and rewrites, so their scores are not interchangeable.
- `entrypoint.py`: a fresh process and sim per match, JSON and logs under `/out`, upload, then a completion marker; each Spot attempt has a unique prefix; only complete attempts are collected. One experiment arm per job (`--config`, `--seeds`, `--seed-start`, `--run-args`).
- `setup.sh`, `build.sh`, `submit.sh` → `control.py` (default **dry-run**; `--execute` to change anything); `collect.py` (dedups Spot retries by job/config/seed). Cloud Build runs render/API/detector checks before publishing an image.
- Safety rails added after round 1 started: **never retry a paid match** (c80189c), mark infrastructure-broken matches **invalid** (`invalid_because`), preflight upload before any paid match, upload errors logged, live snapshots (`live/scene.jpg`, `live/status.json`) every 15 s uploaded every 30 s.
- Cost and quota (report-inputs Codex reports): one match per `e2-highcpu-2` Spot VM; a smoke match took 301 s wall, 137 CPU-seconds, 689 MiB peak RAM, and advanced physics 194 s; at an **assumed** $0.030/VM-hour, 1,000 matches ≈ $2.50 of compute, plus Dream usage (~5,000 min for 1,000 matches), API judges, storage and builds. The project budget was $50 with 50/90/100 % alerts. **Both CPU quota increase requests (to 200) were denied**, leaving 32 (all regions) and 24 (us-central1 E2) vCPUs; Spot obtainability for 100 VMs was 0.9 in `us-central1-f`. The `pilot2-*` jobs ended CANCELLED with upload failures and no `match.json`.
- The default matrix was 1,000 matches (500 seeds each for `hill_climb_v4` and `hybrid_v4 --skip cable_extra`); round 1 used 8 seeds per arm.
- Secrets: the Dream key goes from `~/.dreamlabs/credentials` to Secret Manager via gcloud's stdin and is never printed; the Anthropic key is uploaded by the user; worker identities can read

only those two secrets and cannot delete results.

The `GOAL.md` targets (Darsh, 8 Nov) that these runs were meant to hill-climb:

#	Target	Done when
1	Dream calls at 2 Hz, arms moving during inference	median ≥ 2.0 Hz in valid matches, sim/wall ≥ 0.95
2	Every pass-bar stage succeeds	100 % over the latest 100 valid matches
3	Zero losses (DQ, illegal ball)	0 in the latest 100 valid matches
4	Points as high as possible	each accepted change raises the mean with the 95 % bootstrap CI lower bound above the previous mean; ≥ 20 valid paired matches per arm
5	Infrastructure works	100 % valid, 0 paid re-runs

`HILL_CLIMB_LOG.md` round 1 (arms A baseline, B async 2 Hz + notes, C B + real judge, D B at 1 Hz) is **empty**: no arm produced comparable valid matches (§7.7).

7. Experiments and results

Back to the index

Data sources are kept apart: **(a)** the station's inference log (real, outcomes not logged), **(b)** 101 scored station rollouts (real, small counts), **(c)** the stress sim (scripted policy), **(d)** MuJoCo matches (simulated table, real policy), **(e)** offline benches. Marks: **[V]** recomputed here from raw files; **[D]** from an in-repo document whose raw data is not in the repos; **[U]** not verifiable.

7.1 Latency and the serving loop (real station) [V]

Raw file: `task_simulation/data/station_nuc_20261108_late/robots-inference.jsonl` (also on the `paper` branch). 52 `run_start` events, 1,650 `inference` rows, 18:06 UTC 7 Nov → 05:22 UTC 8 Nov.

Quantity	Value
Round trip (rtt_ms): median / p10 / p90 / max	**1,116 / 800 / 2,733 / 8,655 ms**
Model serving (`serve_infer_ms`) median	293 ms
Network (`network_ms`) median	40 ms
Gateway (`gateway_ms`, as logged; includes upload) median	992 ms
RTT − serve − network, median over all chunks	740 ms
Chunks with uplink ≥ 50 Mbit/s: n, RTT median, gateway median	619, 900 ms, 780 ms (→ ~607 ms fixed overhead beyond the 293 ms model)
Chunks with uplink ≤ 9 Mbit/s: n, RTT median	266, 2,228 ms
Request size median	1.61 MB
Sync vs async chunks	**1,101 sync (66.7 %)**, 549 async
Inter-chunk gap per session, median	sync **3.54 s**, async **1.55 s**
`stall` events	100
Errors	16 (table in §10)

The first 19 runs alone (`data/station_nuc_20261107/`, 1,186 chunks to 00:02 UTC 8 Nov) give median 1,062 ms, 767 sync / 419 async. The paper's "at 8.8 Mbit/s up, calls took 4.7 s" is a single example we did not re-derive.

Interpretation: the pacer switches to stop-and-wait above a 0.9 s median, and the median sits right on it, so the arms stood still ~30 % of the time (2.4 s of motion per 3.54 s). The harness adds 5–8 s per stage boundary (a poll, a judge call, a confirming look), 25–40 s of a 300 s match [D: `DECISION_MODEL.md` F4].

7.1.1 Metal driver on a simulated Damiao bus [D]

`station_addons/bench_metal_tick.py` on `station-tuning`: Dream's own Metal driver, 20 Hz rows interpolated at 250 Hz, 0.4 Hz sweep, gripper late half the time. Tracking RMS per joint, degrees:

Setting	Bus tick (Hz)	j0	j1	j2	j3	j4	j5
Stock: 30 Hz, FF τ 0.8 s	30.0	2.24	2.89	3.64	4.46	1.73	1.74
60 Hz requested	47–52	2.16	2.80	3.53	4.44	1.63	1.63
30 Hz, FF τ 0.15 s	29.8	1.37	1.73	2.19	2.31	1.18	1.19
60 Hz, FF τ 0.15 s	50.4	**1.27**	**1.60**	**2.03**	**2.27**	**1.05**	**1.05**

The feed-forward filter, not the bus rate, limits tracking (−40 to −50 % RMS). Measured on the station (run 3, the fine-tune): the arm trails the command by 150–250 ms. These knobs (`PAH_FF_TAU_S`, `PAH_METAL_TICK_HZ`) were never A/B-tested on the arm.

7.2 Robustness in the stress simulator

7.2.1 Before and after the overnight fixes (main 3055bf3 → d938644, 4 seeds) [V]

Files: `task_simulation/results/stress/main_3055bf3_part{1,2}.txt`, `main_d938644.txt`, `illegal_ball_guard_6seeds.txt`.

Scenario	Before (3055bf3)	After (d938644)
Happy path	87, 74, 90, 116	83, 94, 95, 95
Policy refuses on first start	0, 0, 0, 0 (match ended)	83, 94, 95, 95
Empty action buffer mid-run	10, 10, 15, 15 (ended)	79, 94, 95, 100
IK SVD failure	10, 0, 10, 10 (ended)	83, 84, 95, 95
Corrupt first frame	20, 15, 20, 20	83, 94, 95, 95 (the paper prints “83–90”)
Policy keeps the unplugged plug	re-plugged it 4/4 (`replug=4`)	never
Illegal ball, 6 seeds, frames 2 s old	lost 4/6 (`black_ball_lagged`)	lost 2/6 with the local detector guard
Unplug brushes a block (`reach_across`)	DQ 4/4	DQ 4/4

The plug case mattered most: when the policy kept the unplugged plug, the old harness’s next prompt led it to plug the cable back in, which the rubric forbids.

7.2.2 The 185-match configuration sweep (37 scenarios × 5 seeds) [V from raw rows]

Recomputed with `results/stress/final_20261108/analyze.py` over `out/*.jsonl`:

Config	Matches	Mean pts	Lost (DQ / illegal ball)	Unplug passed	Tower ≥ 3 at end	Mean balls	Ended early
main_v4	185	60.5	4 (4 / 0)	179	143	0.00	5
guard_v4 (per-poll guard)	185	56.8	4 (4 / 0)	179	139	0.00	5
guard_v4_live (10 Hz guard)	185	56.8	4 (4 / 0)	179	139	0.00	5
hybrid_v4	185	61.2	5 (4 / 1)	179	149	0.09	5
hybrid_v4_fast (4–6 s judge)	185	70.5	**32 (4 / 28)**	179	140	3.01	31
hybrid_long (fast)	185	63.2	**39 (5 / 34)**	179	156	3.38	134
final_v4_fast_live	185	66.6	6 (6 / 0)	177	136	2.37	35
final_hybrid_fast_live	185	**69.8**	**4 (4 / 0)**	179	139	2.90	35
build_v4 / build_hybrid (8 scenarios, 5 seeds)	40 / 40	50.0 / 52.0	0	35	29	0.00 / 0.03	0

`FINAL_HARNESS.md` and the paper give a 95 % CI of 65.3–74.4 for final_hybrid_fast_live; we did not recompute the bootstrap. Readings: judge speed is the main lever on points (a slow judge rarely reaches the balls); a fast judge **without** the live guard lost 28 (hybrid) and 34 (long plan) matches to the illegal ball; with the guard and sensor overrides none. The 4 shared losses are `reach_across` DQs, which no supervisor can undo. Two extra DQs in final_v4_fast_live came from two correlated false “cable out” verdicts with two calls in flight (§4.5). Removing synchronous entry checks from stages the sensors can answer raised mean points 54.9 → 60.0 with no new losses (24 scenarios × 4 seeds, `--latency 6.5-13`; task_simulation README) [D].

Caveat: the sweep's 13 extra scenarios are not in any committed file and the worktree commits it used are not recorded, so it cannot be re-run exactly (§13).

7.2.3 Our own reruns of the committed stress sim (9 Nov) [V]

`python experiments/stress_match.py --seeds 5 -j 8` on each branch's own code and default per-scenario settings (each scenario's own plan, default latency, no `--live-guard`), 24 scenarios × 5 seeds = 120 matches:

Branch	Mean pts	Losses	DQ	Illegal ball in bin
main @ acc1672	**71.8**	17	4 (`reach_across`)	13 (`black_ball` 4, `black_ball_slow_monitor` 4, `black_ball_lagged` 4, `black_ball_local_guard` 1)
cloud-sims @ d2203e0	**53.5**	7	4 (`reach_across`)	3 (`black_ball` 2, `black_ball_lagged` 1)

These numbers are **not** comparable with the 185-match table (different scenario set, no live guard, default latency). They show the harness code on both lines still runs end to end in the stress sim, and that without the live guard the monitor-only guard still loses matches to the illegal ball on main. cloud-sims scores lower on the happy path (70 vs 96) because its v4 plan carries the operator-notes stages and retry rules.

7.3 Judges

See §5.1.3 (full 25-configuration table, verified).

7.4 Local classifiers on station data

See §5.2 (detector: held-out ball recall 20/27 → 26/27 at 100 % precision; blocks 16/23 → 23/23), §5.3 (cable ends: USB-A recall 0.68 → 0.84, USB-C precision 0.83 → 0.92), §5.5 (joint faults), and our main-vs-retune guard check (§5.2.4).

7.5 The policy under the harness in MuJoCo (table v1)

7.5.1 Cluster matches, 7–8 Nov [V]

`task_simulation/results/cluster_mujoco/` (excluding `dream_20261107_oracle_bug`, a batch with a referee bug):

Batch	Matches	≥ 1 cable out	DQ	Points per match
dream_20261107b	6	4	0	10, 10, 0, 10, 0, 10
dream_20261107c	6	4	0	0, 10, 10, 10, 10, 0
dream_20261107d_dropcable	6	5	0	10, 10, 10, 0, 10, 10
main_v4_20261108 (hub on the **left**)	3	0	**3**	0, 0, 0
main_v4_righthub	3	1	0	0, 0, 10
smoke	1	0	0	0
Total	**25**	**14**	**3**	

Base Dream-1 unplugged a cable in 14 of 25 matches; first unplug median 57 s, range 28–269 s [D: `LAST_NIGHT_2026-11-08.md`; per-match `stage1_at` is in the referee JSON]; then it **kept holding the plug**: put-down 0 of 54 tries, stack 0 of 25, balls never reached [D]. All 3 DQs came with the hub on the left, where the prompts no longer matched the scene.

7.5.2 Arm naming (dual-arm benchmark, `codex/dual-arm-experiment`) [D]

YAM sim, n = 12 (`runs_dual_arm/20261107T230412Z/results.jsonl`, not in the repo): "with the left arm, …" succeeded **0/4** and the right arm still moved for 10–19 s; "left does X while right does Y" 0/4; two goals without naming arms finished one goal in 2/4 trials. Hence: name the object and its position, not the arm (and on the station: "the right arm" sent the far arm for stacking and failed; with no arm named Dream used the near arm and stacked; `plans/bot_v1.yaml:8-9`).

7.5.3 Sim-time and relay-order batches (`prompt-engine`, table v1) [U]

Reported in the paper: six sim-time matches of the template-engine plan scored 42, 42, 41, 15, 15, 15 (mean 28.3), passed the tower bar in 3 of 6 and binned no balls; a relay-order batch got the first unplug in 12 of 12 matches and a second in 8 of 12, seated no plug, then crashed at the first stacking stage on a slot-filler bug (fixed with a fallback and a test). **No logs for these batches are in any of the three repos.**

7.6 The prompt study on the station: 101 rollouts (Daniel Carrera) [D]

`docs/prompt_comparison_2026-11-08/` (branch `docs/prompt-comparison-2026-11-08`, PR 6). Every archived station rollout 09:56–13:29 on 8 Nov (times as written in the study, station-local; the zone is not stated): **101 rollouts, 171 prompt steps**, scored read-only on the station by `prompt_stats_remote.py`, grouped by `prompt_families.py`. The committed `families_20261108.json` has 30 families whose steps sum to **171** [V]; the raw `segments_20261108.jsonl` and the two scripts are referenced but not committed.

Outcomes come from first/last overhead frames (balls, blocks) and earlier video reviews (cables); execution proxies from the 20 Hz joint log: grip width (plug body 4.5–14 mm vs wire/empty ~3 mm; ball ~48 mm; cube 25–55 mm), Dream lift ≥ 5 cm while holding, scripted pull still holding after +10 cm, closures/min, % frozen.

Task	Prompt family (abridged)	Outcome
Unplug	U10 "With the right gripper **half open**, move down onto the plug…, close…" + harness +10 cm `/move`	jaws on the plug body in **15 of 26** steps (58 %); 12 scripted pulls, **8 still held the plug** after the lift
Unplug	U7 "Remove the first cable on the right… then place it on the right side"	one cable by Dream alone, then 0 in 4 repeats
Unplug	U6 "grip the plug body of the rightmost cable… pull it straight up"	1 cable, lifted 23 cm (n = 1)
Unplug	U3 writer default "grip the **hard** plug body…"	4 plug / 7 wire grips (31 % on the plug); 0 confirmed
Unplug	U5 574-character coached prompt	**0 cables in 286 s**
Unplug	U4 writer micro-rewrites ("regrip", "hover", "lower")	0; **9 of 15 sent while holding the plug**, the plug released in 5; 6.7 closures/min
Unplug (FT)	U1 the fine-tune's trained text	**0 new grips in ~10 min** (on base, the same text grasped and lifted 11 cm once)
Put down	"Put the held cable down on the table to the right…"	11 steps started holding; released in 7
Ball	B1 "With the right arm, pick up the golf ball and drop it into the white basket"	**2 of 5** in (both near the right gripper); the misses were mid-table
Ball	B2 left arm / B3 two arms in one prompt	no grip
Stack	S8 "stack the blocks and sticks as high as possible" (the rubric's wording)	stacks in **2 of 3** runs
Stack	S2 "put the pink cube on top of a blue cube"	most decisive grasps (6 of 6); 1 correct stack, 1 wrong cube
Stack	S1 motion script ("lift it high… lower it gently… then open")	0 of 10 steps
Stack	S3 "build a tower on X: put A, then B, then C"	0

Patterns: **name the goal, not the motion** (Dream does the first verb of a chained prompt and dithers on the rest); the cable grasp is the exception where describing the gripper opening helped; Dream almost never does the pull itself (4 Dream-made lifts all day); never re-prompt a held grasp;

one arm, one object, within reach; short steps (15–18 s) spend 31–45 % of the time frozen at step starts vs 6–16 % for 25–90 s harness steps.

Replay review 13:47–14:28 (`replay_1347-1428/README.md`, 23 runs) [D]: 1 cable fully out (with the "…and toss it onto the table to the right" prompt), **4 balls in the basket** (all right arm), a 2-high and a **3-high stack** (blue/pink/blue, the best of the day) — **both stacks destroyed by the next prompt**. Root causes ranked: (1) no stop-on-success; (2) steps cut short and reworded too fast (6 of 10 failed ball steps ran ≤ 17 s; the orange ball got 5 wordings in 3.5 min); (3) resets sent while holding (8 of 17 "open both grippers…" dropped a held object); (4) reach and placement; (5) cable grasp precision (7 of 14 unplug steps ended on wire or nothing); (6) fine-tune cold starts; and the pull-force add-on firing 70 times uselessly. "Was it the prompts? **Mostly not**" — it was when prompts were sent. These findings produced `stop_on_pass`, the flowing `lock_prompts` and `hands_off` on the bot line.

7.7 Cloud round 1 (8 Nov, MuJoCo, real Dream-1) [V]

`task_simulation/results/cloud_round1_20261108/`: 29 `match.json` files.

Arm	Valid	Invalid	Valid points	Dream call Hz (valid)
r1-a-baseline (`--execution-horizon 0`, notes off, oracle judge)	2	6	10, 42	n/a (sim rate 0.64)
r1-b-async2hz (`--execution-horizon 10 --dream-pipeline 2`, notes on)	**0**	7	—	—
r1-c-haiku (B + OpenRouter Haiku 5.5 low)	4	4	0, 0, 10, 15	0.16–1.18
laptop r1-E-gemini	4	2	15, 15, 15, 15	1.62–5.52
Total	**10**	**19**	**0–42**	

Invalid reasons: "harness stopped on an error" + "no judge verdicts" (11), "harness stopped on an error" (8). The run stopped at **12:04:48 UTC** when every Dream call began returning `DA-1000 UNAUTHORIZED (disabled)` — the account had run out of inference credit while account calls (listing models) kept working; ~28 sessions were open. Arm B has no valid match at all, so `HILL_CLIMB_LOG.md` stays empty and **round 1 cannot separate the arms**. A rerun of the relay batch hit the same limit 34–99 s into each match [D]. Six invalid laptop-run errors were Dream refusing the plug-in prompt (DA-4000 "Plugging cords into power strips…"). Lesson: probe Dream with one call before every batch (asked for at 14:57, never built; `LAST_NIGHT` to-do #3).

7.8 The fine-tune vs base on the station

See §9 (FT diagnosis). Summary [D]: 7 fine-tune unplug runs reached a minimum tool height of 8.3–12.8 cm (median 11.7) against 0–6 cm for base the same day; 0 cable grips in ~10 min; the only time the fine-tune closed on anything all day was one block carried for ~110 s at 13:18; cold starts of 3+ min.

7.9 What is missing

There is **no logged per-stage success rate from complete five-minute harness matches on the station**. The inference log records calls and errors, not outcomes; the prompt study scores individual rollouts; match-day results (if any were recorded) are not in the repos. The harness was also never compared against plain Dream-1 on the station with the same prompt.

8. Teleoperation and data collection

Back to the index

8.1 Recorders (`darshp848/follower_code`)

Recorder	Arm	How	Writer	Status
`leader_follower.py`	i2rt **YAM**	one arm in i2rt zero-gravity mode is the leader, the other copies joints (rate-limited 2 rad/s); stops if the follower lags > 0.35 rad for 0.5 s	`runs_teleop/<time>/episode_NNN/` (`data.npz`, `meta.json`, per-camera mp4)	**refuses to run unless `~/.dream-labs/station.yaml` says `body: yam`** (047a27e): the hackathon stations are Metal
`metal/leader_collect.py`	MakerMods **Metal**	one arm runs MakerMods' gravity-compensated `metal_leader` (kp = 0, Pinocchio gravity + friction feed-forward), moved by hand; the other follows	the kit's `dhk-collect` (dashboard, cameras, device checks, station lock, homing, lag check) and its **LeRobot v2.1** writer	used for the 97 leader episodes
`quest/quest_collect.py`	Metal, Meta Quest headset	the kit's `dhk-collect` with four pieces swapped in (below)	same v2.1 writer	written after the Quest data in the fine-tune was recorded
The kit's own Quest teleop (`dhk-collect`)	Metal	Quest controller → IK → the kit's `MetalFollower`	v2.1	used for the **73 Quest episodes**; sends **zero torque**, so the arm sags (§9)

8.1.1 Metal leader/follower details (`metal/leader_collect.py`)

- Keys (type, then Enter): Enter opens the arms, then starts or saves an episode; `d` discards; `h` sends the follower home; Ctrl+C walks both arms back to their start pose and releases them.
- `--leader left` (default). By default the leader's side records its **parked** pose (at deploy time that arm stays parked); `--record-leader` records its real motion. The left wrist camera still sees the leader and the hand, which is why the `-frozen` dataset variants replace that camera with a still (`tools/freeze_camera.py` in the harness).
- Follower: closes the gap at 0.4 rad/s for ≤ 3 s, then follows at `--follow-speed` 4 rad/s (MakerMods' 280°/s cap still applies); re-syncs if > 0.35 rad behind.
- **Gravity feed-forward (`FollowerGravity`, on by default, fdf4bf8):** without it every joint sits under its command by gravity torque / kp. The first real episodes showed shoulder_lift +4 to +6° and elbow_flex +2.5 to +4.3° on average, and the old 1.5 rad/s cap was hit on every joint. With MakerMods' own Metal gravity model in the follower's MIT frames, shoulder-lift tracking went from 4–6° low to **+0.2°**. `--no-gravity-ff` turns it off.
- `metal/tools/tracking_report.py <dataset>` prints per-joint tracking error of any dataset.
- Tests: `cd ~/physical-ai-hacks && .venv/bin/python -m pytest -q ~/follower_code/metal/tests` runs the kit's real Collector and writer over fake Metal devices, then `tools/add_eef.py` (Dream's dataset checks). "All 10 pass on the hackathon NUC" [D]. They need the kit and cannot run outside it.

8.1.2 Quest teleop and the continuity fixes

`quest/quest_collect.py` on `main` (6bc0576, e87a17a):

Problem	Fix
Wrong direction	heading calibration: press **Y**, move the right controller ~20 cm forward, press **Y**; saved to `~/.cache/follower_code/quest_heading.json`
Jitter	a **One Euro filter** per controller pose, restarted at every engage; IK damping 1e-3 → 1e-2
Lag	loop at 60 Hz instead of 30; joint cap 2.5 rad/s instead of 1.5
Arm sits low	gravity feed-forward on by default (Metal), the same MakerMods model (e87a17a, 8 Nov 01:19 UTC)
Wrist goes wild	2° rotation dead band, `--rotation-scale`, **X** locks/unlocks the wrist

Offline on the kit's test arm with 2 mm tracking noise: tool jitter 4.7× lower than the kit's (0.47 vs 2.2 mm) at ~17 ms more lag on a 5 cm move [D]. Tests: 14 pass over the kit's fakes [D].

`quest-continuity` (not merged) adds `quest/continuity.py`:

- **`WristFlipGuard`** (line 26): stops big joint swings near the joint-5 limit. In a replay of 40 fast wrist events, 7 swung a wrist joint > 90° with the kit's IK and 2 with the guard, tool position held within 8 mm [D].
- **`TrackingJumpGuard`** (line 74): a controller jump of 8 cm or 45° in one reading is a glitch; the arm holds until the clutch re-engages.
- **`per_joint_caps`** (line 102): wrist joints 4 rad/s instead of 2.5, plus a faster rotation filter: wrist lag 105 → 27 ms in the offline replay [D].
- Homing: a low tool lifts straight up at 0.1 m/s before the joint-space move home; pressing home twice within 2 s discards the episode and homes.
- `quest/bench/` (`bench_ik.py`, `resp.py`, `final.py`, `ratio.py`, `guard.py`, `metal.urdf`): the replay bench behind these numbers; needs numpy, pandas, pyarrow ≥ 21, mujoco and mink 1.1.0 and the datasets copied off the NUC.
- **Never run on the robot**: the NUC dry run was held at 02:49 UTC 8 Nov while the team was recording; the 5 new tests have not run on the NUC either. An uncommitted NUC edit to `metal/leader_collect.py` on this branch is saved as `nuc/follower_code_quest-continuity_uncommitted.diff` on `station-tuning` (dashboard grip override with ramp, kinesthetic mode, read retry per Kaushik's notes).

8.2 Dataset format: LeRobot v2.1 vs v3.0, and `add_eef`

- Dream's fine-tune takes **LeRobot v2.1** (one parquet and one mp4 per camera **per episode**). The kit's `dhk-collect` writes v2.1, so no conversion was needed.
- MakerMods' own recorder writes **LeRobot v3.0** (many episodes per parquet and one mp4 per camera). Kaushik wrote `tools/v3_to_v21.py` (on `kaushik/day1-extras`) as a backup; on a 20-episode MakerMods dataset (13,411 frames) it passes Dream's validator [D: `extras/kaushik/PREP.md`]. "Format risk resolved 2026-11-07: the kit's `dhk-collect` writes v2.1."
- **`tools/add_eef.py`** (the kit's, run with `~/dream-env/bin/python tools/add_eef.py DATASET --arm metal [--replace]`) adds end-effector columns by FK and runs Dream's dataset checks. Metal fine-tunes require `use_eef=True`. The FT diagnosis checked `add_eef`'s FK against the URDF with a 0.135 m tool: < 0.001 mm.
- Dataset tools in the harness: `tools/split_episodes.py` (cut long takes into task episodes, each with its own text), `tools/auto_split.py` (picks the working arm and labels pick-and-place), `tools/freeze_camera.py` (replace one camera with a still "parked" view), `tools/check_dataset.py` (deep checks beyond Dream's `validate_dataset`: meta vs tables, index and timestamp continuity, NaN, ranges, one working arm with the other parked, frame counts, frozen leader cameras, duplicates, how each task ends), `tools/label_rollout.py` (label Dream's own rollouts with harness outcomes), `tools/review_page.py` (`tools/episode-review-page`, David's keep/drop review page).

8.3 What was recorded (TELEOP_PLAN.md)

Decision at 17:50 on 7 Nov: **one fine-tune on UNPLUG and PUT-DOWN** ("Unplug is everything": Stage 1's pass bar; base Dream-1 unplugs ~1 try in 6). Stacking and balls stay on base skill; the fine-tune mixes in Dream's data (`base_share` 0.5).

Task texts (set in the dashboard before every take): **T1** `unplug the USB cable at the end of the hub nearest the right arm`; **T2** `put the held cable down on the table away from the hub`; plus T3 plug-in, T4 block, T5 ball, T7 cylinder in the hole.

`~/hackathon_data/ft-final`, built 22:30 from 42 takes (13:20–21:07), unplugs re-cut to the top of the pull, David's review applied:

Folder	Source	Episodes
`quest-split-v4-final-eef`	Quest 18:12–18:31, David's 9 drops	65
`unplug-split-v1-final-eef`	Quest	5
`unplug-v1-split-eef`	Quest	3
`leader-split-v4-final-frozen-eef`	leader takes 25–35	28
`leader-split-v6-frozen-eef`	leader takes 39–40	21
`leftarm-split-v5-frozen-eef`	left arm 36–38, 41 (balls, cylinders, blocks)	16
`leader-split-v1-frozen-eef`	leader	14
`plugin-split-v1-final-frozen-eef`	leader	11
`stack-split-v1-frozen-eef`	leader-right	4
`leader-split-v2-frozen-eef`	leader	3
Total	**73 Quest + 97 leader**	**170**

Per task: 69 unplug, 55 put down, 19 plug-in, 12 ball, 10 cylinder in a block's hole, 4 block, 1 cylinder on top; 26.9 min; **48,413 frames** at 30 fps (Dream's check) — the kit's check said 48,428. State 34 wide, action 32. Final checks at 23:00 all passed (`check_dataset.py --videos`: 0 errors, 0 warnings; laptop copies byte-identical).

Human teleop speeds: unplug median 8.5 s (15 demos), put-down 4.8 s (9 demos) [D].

8.4 Data still missing at the end

From `docs/LAST_NIGHT_2026-11-08.md`: more stack demos (any colour order), golf-ball demos (had 1), watcher-arm demos (none), harness runs with per-stage success, real motor torque while pulling (`candump` not on the NUC; `harness/can_live.py` receive-only listener on cloud-sims), grip at 2.0 N·m (later done), station frames for the detector (later done on `fix/classifiers-8nov`), and unplug demos **from the home pose** (§9).

9. Fine-tuning: why `relay-final` aimed high

Back to the index

Primary source: `docs/ft_diagnosis/FT_DIAGNOSIS_2026-11-08.md` on branch `ft-diagnosis` (PR 3). Everything there was read from the NUC over SSH (no robot commands, no acting station calls, no Dream calls). Marks as in §7.

9.1 The job

Item	Value
Model	**`dream-1-ft:bb4b9e5f-relay-final`**, ready 07:02, warm 07:14 (8 Nov)
Job / dataset	`ftj_fdff469db8e2b643` / `ds_5ae1c3b4e8739af1` (`relay-final`), queued 22:54–23:00 7 Nov from tmux `finetune`
Data	170 episodes, 48,413 frames (§8.3)
Hyperparameters	1,000 steps, lr 1e-5, batch 32, `use_eef: true`, constant schedule, `base_share` 0.5; final loss 0.0034
Training time	queued 23:00, training 23:20 → 05:56 (0.4 min/step), saving to 07:01
Cost	~$200 for 1,000 steps (plan estimate); a 500-step minimum retrain ≈ 5 h and ~$100
Tooling	`tools/finetune_watch.py start … --max-requeues 1 --i-accept-charges` (retries, renames, one re-queue, warm-up, `MODEL.txt`); `tools/start_finetune.sh`

Only the fine-tune account's key runs the model; the station's saved key is a different account. Main defaulted the match plans to the fine-tune at b1f6e4b (8 Nov 18:09 UTC) and the final `bot_v1.yaml` went back to **base `dreamlabs/dream-1-beta`** ("the 8 Nov fine-tune was worse on every task", `plans/bot_v1.yaml:36`). `plans/bot_v1_ft.yaml` keeps the bot harness on the fine-tune with trained texts only.

9.2 Symptom

On the station the fine-tune's tool stayed **8–13 cm above the table** while reaching and closed at **11–16 cm**; the demonstrations grasp at **4–6 cm**; base Dream-1 reached 0–6 cm the same day.

Run	Model	Min z (m)	Median z (m)	Right gripper min
095731	base	0.060	0.064	0.03
095854	fine-tune	0.083	0.157	0.03
100247	fine-tune	0.099	0.130	0.15
100709	fine-tune	0.121	0.162	0.33
101053	fine-tune	0.126	0.161	0.37
101617	fine-tune	0.128	0.152	0.95
101716	fine-tune	0.132	0.166	0.41
101840	fine-tune	0.115	0.150	0.22
102040 (long no-power prompt)	fine-tune	0.102	0.137	0.18

Its y was on target (0.13–0.22 m): the approach direction was right and the height wrong. Prompt study: 0 cable grips in ~10 min (§7.6).

9.3 Cause 1 (strong): gravity sag in the Quest actions

Mean action − state, right arm, **slow frames only** (joint speed < 0.15 rad/s, so steady state, not lag):

Folder	Source	Frames	Joint 2 (rad)	Joint 3	Joint 4	Tool offset x / y / z (mm)	Median norm (mm)
quest-split-v4-final-eef	Quest	2,137	**+0.078**	**+0.047**	**+0.043**	+18 / +3 / **+54**	**55.9**
unplug-split-v1-final-eef	Quest	96	+0.093	+0.030	+0.004	+9 / 0 / +37	37.2
unplug-v1-split-eef	Quest	152	+0.086	+0.039	+0.033	+16 / +3 / +48	50.9
leader-split-v1-frozen-eef	Leader	1,121	+0.007	+0.008	+0.002	+3 / +1 / +6	8.0
leader-split-v2-frozen-eef	Leader	467	+0.010	+0.010	+0.013	+4 / 0 / +7	6.1
leader-split-v4-final-frozen-eef	Leader	4,015	−0.002	+0.005	+0.001	+1 / 0 / +1	5.5
leader-split-v6-frozen-eef	Leader	2,310	−0.006	+0.003	+0.012	+1 / +1 / +1	7.8
plugin-split-v1-final-frozen-eef	Leader	2,987	−0.014	+0.003	−0.002	−2 / +3 / −5	7.6

Mechanism: the kit's `MetalFollower` sends zero torque, so each joint sits below its target by gravity over kp; the leader path adds `FollowerGravity`; Dream's station driver also sends gravity torque (live command − measured on slow frames ≤ 0.016 rad). In a Quest episode the model learns that holding the arm at height z takes a command at **z + 5.5 cm**. Live, that command is reached, so the arm rises and is asked to rise again. Leader episodes say the opposite for the same scene and state: **two incompatible state → action maps**. Quest datasets also show 9–10° median rotation error between action and state (leader 2–3°).

Where demos grasp (first frame the right gripper drops below 0.35): leader state z 0.043 / action z 0.049 (n = 28); Quest state z 0.059 / action z **0.110** (n = 32). Mix: unplug 36 Quest / 33 leader; put-down 37 / 18; plug-in, stack, balls, cylinder 0 / 46.

The effect on behaviour is an inference from these numbers; the policy was not run on held-out frames.

9.4 Other causes, ranked

#	Cause	Evidence	Fix without retraining?
2	Almost no demos start from home: **9 of 69 unplugs** start within 0.3 rad of it (median start distance 2.14 rad); put-down 0 of 55; plug-in 0 of 19	solid as a fact, effect unproven	no (the start-pose gate logs it)
3	Stack and balls thin and left-armed (4 block demos, all left arm; 12 balls); "open both grippers and move the arms up" and `tower_prompts` texts are not in the trained set	solid	yes: base Dream-1 for those stages
4	Left arm parked elsewhere in 17 Quest episodes (4.4 / 2.1 / 2.5–2.7 rad from live home)	moderate	no
5	20 Hz playback of a 30 fps-trained model; live command→measured lag 160–200 ms vs 83–117 ms in the recordings (~80 ms extra)	real but ~1 cm at < 0.1 m/s	optional A/B (`control_hz: 30`, `PAH_FF_TAU_S=0.15`) — not together with a data fix
6	One 257 mm action jump (Quest ep 36, frame 245); ~20 Quest episodes with 30–52 mm steps	minor	drop ep 36

Ruled out: camera keys, swaps, rotation, colour, resolution (640×360 from 1280×720), overhead camera moved (< 2 px), schema (one signature across 10 folders), joint order and sign, units (rad; gripper 0–100 in state, 0–1 in `grip`), TCP (FK vs URDF < 0.001 mm), action space. Not fully ruled out: table clutter (headset, jar, cards near the hub).

9.5 Offline fixes tried on a fitted Metal model (`station-tuning`) [D]

`station_addons/bench_replay.py`, `metal_plant.py`, `sweep_replay.py`: a MuJoCo model of one Metal arm (Dream's link table, kp/kd, 30 Hz bus, gravity and velocity feed-forward as in `robots/damiao.py`) replaying the saved `commanded_r` rows of the 7 fine-tune unplug runs. Calibration: effective load 1.18 / 1.30 / 1.48 × URDF on joints 2/3/4; replay of Quest actions without feed-forward reproduces recorded tool height within 10 mm RMS (+1.7 mm mean, 9 episodes); live with feed-forward 5–6 mm RMS (4 runs).

Fix	Needs	Grasp z (cm)	Reach median z	Reach min z, worst run	Passes 6 cm bar	Hits table
Baseline	—	13.1	15.2	8.6	no	no
Gravity × 0.5 / × 0	env var	10.8 / 8.5	13.3 / 11.5	6.4 / 4.3	no	no
Joint offset × 1 (full Quest sag)	code	7.9	10.2	3.9	no	no
Joint offset × 1.5 (live-implied), right arm, faded in by reach	code	**5.5**	7.9	1.8	**yes**	no (1.8 cm margin)
z −5.5 cm	code	7.8	9.3	−3.1	no	yes
z −8 cm	code	5.2	4.8	−7.3	yes	yes
gravity 0 + z −2.4 cm	both	6.4	9.1	2.1	no (close)	no

No env-var-only fix passes. The only fix that passes without going through the table is a right-arm joint offset of ~1.5× the Quest sag, faded in over tool x 0.20–0.25 m: ~20 lines in `station_addons/pah_tuning.py` (a goal offset in `DamiaoChain.send_action`, opt-in by env var). **Not built.** Fidelity limits: open loop, one offset for two kinds of output (leader-style rows would go ~8 cm too low), no contact, and the bar is a height check, not an unplug.

9.5.1 The EEF override that was built (d5e9948, `station-tuning`)

Dream's answer for a Metal fine-tune is tool poses per arm (`action.<side>_eef_xyz`, `_eef_rot6`, `_grip`, 48 rows); the client turns them into joints in `dream.client_ik.ClientIK.solve` on the station. `pah_tuning.py` wraps that call:

Variable	Effect
`PAH_EEF_OFFSET_M`	shift targets per arm before the IK, base frame, metres, capped at 5 cm (e.g. `0,0,-0.02`; `r:0.01,0,-0.03;l:0,0,0` moves only the right arm)
`PAH_EEF_Z_MIN_M`	floor on each arm's tool height
`PAH_EEF_HOOK`	`module:function(side, xyz, rot6, grip, obs) -> (xyz, rot6, grip)`
`PAH_EEF_LOG`	JSONL per chunk: Dream's targets, what went to the IK, solved joints

Checked in sim on run-3 chunks with Dream's real ClientIK (`station_addons/bench_eef_override.py`): no override → solved path within 0.1 cm (right) / 0.3 cm (left); `0,0,-0.02` → right hand −1.97 cm, left −1.57 cm. This is the "ft-joint-offset / EEF override" of the brief. It was never tried on the arm with the fine-tune; the station ran base Dream-1. (The Cartesian z-offset variants hit joint limits on 971–4,138 of 13,595 ticks in the replay, which is why the diagnosis preferred a joint-space offset.)

9.6 The recipe for a corrected fine-tune (not run)

1. For the three Quest folders, **recompute the arm action as the measured joints ~3 frames later** (keep the commanded gripper), then rerun `tools/add_eef.py --replace`.
2. Drop quest-split-v4-final episode 36; consider dropping episodes 4–17 and unplug-v1-split 0–2 (left arm parked elsewhere).
3. Add unplug demos **from the home pose**.
4. Screen every folder for steady-state action − state offsets above ~1 cm before training.
5. Consider `lr_schedule: cosine`.

A 500-step job takes ~5 h from queueing; `resume_from` also has a 500-step minimum and starts from the biased weights. **A fair sim A/B cannot run with our sims**: `sim_station.py` drives Dream's YAM simulator and the fine-tune is a Metal embodiment.

9.7 Base vs fine-tune from the station logs [D]

	FT (trained text)	FT (long text)	Base 8 Nov	Base 7 Nov
Runs	7	1	3	5
Tool min z on cable prompt, median (range)	11.7 cm (8.3–12.8)	10.0	4.0 (0–6.0)	2.1 (0.8–5.6)
First close while forward, z median	15.8 cm (n = 5)	12.8	28.6 (early closes)	7.8
Time to first close, median	41 s	17 s	6 s	13 s
Dream RTT p50 / p90	700 / 857 ms	700 / 917	750 / 1,603	890 / 2,135

10. Operations and failure modes

Back to the index

10.1 Dream errors seen, and what the harness does

10.1.1 On the station [V]

From `robots-inference.jsonl` (`kind: "error"`):

Error	7 Nov log (19 runs, to 00:02 UTC)	Full log (52 runs, to 05:22 UTC 8 Nov)	Text	Harness action (`runner.py:79-98`)
Safety refusal	4	4 (19:55, 20:33, 23:01, 23:30)	`DreamBadRequest DA-4000 BAD_REQUEST (malformed): unsafe instruction, refused: …`	restart once, then fail the attempt (match goes on)
IK failure	3	4 (23:03, 23:34, 23:50, 03:11)	`LinAlgError: SVD did not converge`	`/reset` + restart (≤ 2 auto-resets per run)

Error	7 Nov log (19 runs, to 00:02 UTC)	Full log (52 runs, to 05:22 UTC 8 Nov)	Text	Harness action (`runner.py:79-98`)
Action buffer empty	2	4 (19:57, 23:37, 03:10, 03:12)	`ControlLoopError: action buffer empty for 241 consecutive ticks (12.05 s)`	restart (≤ 5 per run)
Session replaced	1	4 (23:36, 03:11 ×3)	`DreamSessionExpired DA-0001 … was replaced by …`	restart
Total	**10**	**16**		

The digest's "DA-4000 ×4, SVD ×3, buffer-empty ×2, session expired ×1" is the 7 Nov log; the paper's "4 + 4 + 4 + 4" is the full log. Both are right. No HTTP 402 or 429 appears anywhere.

Refusal reasons Dream gave (`LAST_NIGHT_2026-11-08.md`): "inserting metal or objects into a power strip", **"a person's hand is holding the plug"** (a real scene problem), "unplug cables from a live power strip", "unplug live bare wires". Another refusal on 8 Nov 13:54 was triggered by a person's arm over the green ball. Remedies: a **true** safety sentence in every cable prompt (the strip is unplugged; insulated low-voltage USB cords), and hands out of the camera view during runs. Plug-in prompts were still refused in the sim ("Plugging cords into power strips carries electrical/equipment…").

10.1.2 In sim and cloud runs (8 Nov)

Error	When	Meaning
`DA-1000 UNAUTHORIZED (disabled)`	12:04:48 UTC to the end of the runs, 83 log lines	**Out of inference credit.** Reads like a revoked key while account calls (listing models) keep working. ~28 concurrent sessions were open; happened twice on 8 Nov. Topped up by Darsh.
DA-4000 (plug-in prompt)	round 1, 6 lines	safety refusal
`DA-6000 GATEWAY_FAULT (connection_timed_out)`	night of 8 Nov, 2 mentions	transient; restart

Operational rule from this: **probe the policy with one call before every batch and stop on DA-1000** (never built; `LAST_NIGHT` to-do #3), never run ~28 sessions at once without checking credit, and never let Batch retry a paid match (c80189c).

10.1.3 Other station failure modes

Failure	Cause	Mitigation
Arm stands still ~30 % of the time	median RTT above the 900 ms pacer threshold → sync	light frames; wired network; station-side hedging; keep the harness's own traffic small
Calls of 2–6 s	the harness pulled ~2 MB debug-view trips	`/frame/<cam>.jpg?w=640` (~35 KB)
7–12 s discarded first answers after `/start`; fine-tune cold start 3–15 min	session warm-up	never `/start` mid-match; lock with a prompt instead of `/stop`
Lost run record	Dream overwrites `last-run.csv` on every `/start`; the archiver polls every 3 s	`ARCHIVE_WAIT_S = 4` before a restart
Restart from 10+ cm up → SVD error → full `/reset`	IK seeded far from Dream's last state	the pull skill lowers back before handing back to Dream (`runner.py:970-973`, e8e1489)
Dream went for its own black arm cable	"the USB cable" was ambiguous	"the orange USB cable" (59f439a, **not on main**)

Failure	Cause	Mitigation
Dream went for the balls mid-unplug	“toss it”	removed “toss” (bec9975), but the 13:57 replay’s only confirmed unplug used the “toss” wording, and 716c4d0 put “toss … to the far right” back — this is why `test_bot_v1_plan` fails on main
Monitor blind	OpenRouter key over its limit (every look HTTP 403)	switch the panel’s monitor to Haiku on the Anthropic API (8ec93b1, a516488); `max_blind_s` stops the run if the guard gets no answer for 20 s
Gripper too weak	hold torque 0.5 N·m originally	1.5 → 2.0 N·m (Dream’s max) at 19:14 UTC 8 Nov; watch gripper temperature (latches off at 55 °C)

10.2 The match laptop: MacBook Air (Apple Silicon, fanless)

Constraints (`docs/LOCAL_DECISIONS_PERF.md`, `mac-arm64`):

- 4 performance + 4 efficiency cores, **no fan**: fast for 30 s, slower after a few minutes of sustained load. The live guard (10 Hz × 2 wrists), the per-poll guard, the monitor’s request build (annotation + stitch), and HTTP threads share 4 P-cores. Rule: **vectorise and do less; do not add threads** (`harness/perf.py` caps BLAS/OpenMP to one thread).
- **Exact or not at all**: any change to `detect.py` must keep `tests/test_guard_equivalence.py` green (verdicts, boxes and annotated JPEGs identical to `tests/_detect_reference.py`).
- Measured speed-ups are in §5.2.5; on the GitHub `macos-15` M1 VM the sustained bench managed only ~28 guards/s of ~50–60 because the VM stretches short sleeps — check `guards/s` on the real Air.
- `requirements-match.txt` pins the match environment; `tools/mac_bench.py --sustained 300` shows p95 per 30 s window and `pmset -g therm`.
- macOS specifics elsewhere: `sim_station.py --viewer` must be started with `mjpython`; the fake Claude CLI tests had to be made portable (422e3b8 for Windows; two Windows-shebang failures documented in `OPENROUTER_RESEARCH.md`).
- **None of this was measured on the actual Air**, and `mac-arm64` was never merged, so the match laptop ran the slower detector from main.

10.3 Secrets and environment (names only)

Secret / variable	Used by	Notes
`DREAM_API_KEY` / `~/.dreamlabs/credentials`	station agent, `sim_station.py`, cloud	the fine-tune runs only under the fine-tune account’s key; kept in a tmux shell (`read -rs DREAM_API_KEY; export DREAM_API_KEY`)
`ANTHROPIC_API_KEY`	monitor, confirm, writer, filler on the API backend	never logged; stripped from `claude -p`’s environment on the subscription backend
`OPENROUTER_API_KEY` (via `--claude-backend openrouter` / `--monitor-model`)	fast monitor, judge bench	`docs/OPENROUTER_SETUP.md`
`HARNESS_CLAUDE_BACKEND` (api/subscription/openrouter), `HARNESS_ANTHROPIC_MODELS`	backend selection	
`LIVEKIT_URL`, `LIVEKIT_TOKEN` or `LIVEKIT_API_KEY` + `LIVEKIT_API_SECRET` + `LIVEKIT_ROOM`, `LIVEKIT_PUBLISHER`, `LIVEKIT_TRACKS`	LiveKit observer/viewer	minted tokens are subscribe-only
`PAH_HEDGE_N`, `PAH_HEDGE_CUTOFF_S`, `PAH_HEDGE_MAX_CALLS`, `PAH_HEDGE_MAX_IN_FLIGHT`, `PAH_HEDGE_LOG`	station-side hedged Dream calls	off unless `PAH_HEDGE_N ≥ 2`

Secret / variable	Used by	Notes
`PAH_FF_TAU_S`, `PAH_METAL_TICK_HZ`, `PAH_IK_TRAJ`, `PAH_TORQUE_CAP_NM`, `PAH_GRAVITY_SCALE`, `PAH_EEF_*`, `PAH_TICK_LOG`, `PAH_PULL_FORCE_N`, `PAH_PULL_DIR`	station tuning (`station-tuning`, `ee-pull-force`)	opt-in; refuse to load on any `dreamlabs` ≠ 1.0.37
Doppler (`doppler run -p … -c … --`)	how keys were injected for the match command (`plans/bot_v1.yaml:24-26`)	keys never on the command line
GCP Secret Manager	cloud runner	uploaded by the user; workers read only these two
`anthropic-api-key`, the Dream key secret		
`MUJOCO_GL` (`egl` / `osmesa`), `PYOPENGL_PLATFORM`	headless rendering	

10.4 Safety posture

- The physical e-stop is the real safety. Every station guide says to keep a hand on it, people clear, and that only a person runs commands that move the arms.
- `--dry-run` sends no POST; `--live` asks you to type `yes` after the safety lines; `--no-operator` (and `--match` on final-harness) refuses human input after go.
- The harness never homes at the end of a run unless `reset_last` is set, and never after a guard trip (the gripper holds an illegal ball).
- Open safety items: audit A09 (blocking I/O can carry motion past the deadline: station client timeout 70 s, monitor transport 60–90 s); A06/A07 (guard blind spots; §5.2.4); A11 (a restart after a fault keeps old done evidence).

11. Practical guide

Back to the index

Every command below was either run for this report (marked [ran]) or is copied from an in-repo guide with its source named. Secrets are referred to by name only. Commands that move the arms are for a person at the robot with a hand on the e-stop.

11.1 Install

Prerequisites: Python 3.12 or 3.13, git. The core harness needs only the standard library + PyYAML; the detector needs numpy and Pillow; the cable-end classifier needs OpenCV; the MuJoCo sim needs `mujoco` and Dream's `dreamlabs==1.0.37` (for `dream_examples.yam_mujoco`); headless rendering needs OSMesa or EGL.

Clone the three repos next to each other:

```
git clone https://github.com/darshp848/hill-climb-harness.git
```

```
git clone https://github.com/darshp848/task_simulation.git
```

```
git clone https://github.com/darshp848/follower_code.git
```

Create an environment and install the test dependencies ([ran] on Linux, Python 3.13):

```
python3 -m venv .venv && . .venv/bin/activate
```

```
pip install pytest pyyaml numpy pillow scipy opencv-python-headless mujoco pyarrow pandas
 ↪  psutil pyjwt
```

Headless Linux rendering for the MuJoCo tests ([ran] on Debian/Ubuntu):

```
sudo apt-get install -y libosmesa6 libegl1
```

```
export MUJOCO_GL=osmesa PYOPENGL_PLATFORM=osmesa
```

Optional extras: `pip install "dreamlabs==1.0.37"` (Dream SDK, needed for `sim_station.py --policy dream` and the scene tests), `pip install livekit` (observer/viewer), `pip install dimos` (DimOS skills). The match laptop pins its environment in `requirements-match.txt` on `mac-arm64`.

11.2 Run the tests

On main ([ran] 9 Nov: **334 passed, 1 failed, 10 skipped** with `tests/test_hill_climb_scene.py` ignored; see §13.1):

```
cd hill-climb-harness
python -m pytest -q --ignore=tests/test_hill_climb_scene.py
```

`tests/test_hill_climb_scene.py` fails at **collection** without Dream's SDK, because `importlib.util.find_spec` raises `ModuleNotFoundError` when the parent package is missing instead of returning `None`. Either install `dreamlabs` or ignore that file. The one failing test, `tests/test_grip_assist.py::test_bot_v1_plan`, asserts that the cable prompt contains no "toss"; 716c4d0 added "toss the cable to the far right" back to `plans/bot_v1.yaml`.

A subset that needs nothing beyond PyYAML and numpy:

```
python -m pytest -q tests/test_runner.py tests/test_stress.py
 ↪  tests/test_prompt_engine.py tests/test_grip_assist.py tests/test_guard.py
```

On another branch, use a worktree so main stays untouched:

```
git worktree add ../wt-cloud origin/cloud-sims
cd ../wt-cloud && python -m pytest -q --continue-on-collection-errors
```

11.3 Run the stress simulator (no keys, no network)

List the scenarios ([ran] 24 on every branch):

```
python experiments/stress_match.py --list
```

All scenarios, 5 seeds, 8 processes, one JSON line per match ([ran] ~1–3 min on 4 cores):

```
python experiments/stress_match.py --seeds 5 -j 8 --out runs_stress.jsonl
```

One match, event by event:

```
python experiments/stress_match.py -s no_release --seeds 1 -v
```

At the judge latency Haiku 5.5 measured, against a chosen plan:

```
python experiments/stress_match.py --latency 6.5-13 --plan plans/hybrid_v4.yaml --seeds
 ↪  5 -j 4
```

Watch one match in the browser at 10× (port 8765):

```
python experiments/live_stress.py -s black_ball --speed 10
```

A/B plan variants over every scenario:

```
python experiments/sweep_variants.py --latency 6.5-13
```

Summarise a JSONL file (mean points, losses):

```
python -c "import json,sys; r=[json.loads(l) for l in open(sys.argv[1])]; print(len(r),
 ↪  sum(x['points'] for x in r)/len(r), sum(x['lost'] for x in r))" runs_stress.jsonl
```

To aggregate the published 185-match sweep ([ran]):

```
cd ../task_simulation/results/stress/final_20261108 && python analyze.py
```

Add a scenario: append a `Scenario(...)` to `SCENARIOS` at the bottom of `experiments/stress_match.py` (behaviour switches, `Fault(at, kind)`s, outages, camera spans, latency, or another `plan=`). If it guards a fix, add an assertion to `tests/test_stress.py`.

11.4 Run a MuJoCo sim match

Prerequisites: `dreamlabs==1.0.37`, `mujoco`, a Dream key with **credit** in `~/.dreamlabs/credentials` (`DREAM_API_KEY=...`). Probe the policy with one call first: a disabled key (DA-1000) fails every match.

Offline smoke test with the scripted policy (no key):

```
python sim_station.py --policy scripted --scene hill_climb
```

Terminal 1, the simulated station with real Dream-1 (README):

```
python sim_station.py --policy dream --scene hill_climb --open
```

Terminal 2, the harness against it:

```
python -m harness plans/sim_fruit_bowl.yaml --live --station http://127.0.0.1:8091
 ↪  --view http://127.0.0.1:8093 --verifier time
```

A scored relay match with the oracle verifier and referee (`experiments/sim_match.py`; on `prompt-engine` add `--clock sim` so timeouts mean sim seconds, and `--filler oracle|default|plan` for the prompt engine):

```
python experiments/sim_match.py --plan plans/engine_v4_sim.yaml --filler oracle
```

For the **rebuilt realistic table**, check out `cloud-sims` first (`sim_scenes/hill_climb_table.py` at 9c89ae3+). Render it without a policy or network (task_simulation `paper` branch):

```
cd ../task_simulation && git checkout paper && python paper/render_sim_table.py
```

On macOS the 3D viewer needs `mjpython`:

```
.venv/bin/mjpython sim_station.py --policy scripted --scene hill_climb --realtime
 ↪  --viewer
```

11.5 Cluster and cloud batches

Home cluster (task_simulation README):

```
docker build -t hill-climb-sim -f cluster/Dockerfile cluster
```

```
PLAN=plans/hill_climb_v4.yaml cluster/run_matches.sh 3 2 my_label
```

Offline sweep on the cluster (task_simulation):

```
cluster/run_stress.sh 50 7 sweep1
```

```
SEED0=51 cluster/run_stress.sh 50 2 sweep1 happy black_ball
```

Google Cloud Batch (branch `cloud-sims`, `cloud/README.md`). Every script defaults to a dry run; `--execute` spends money. Prerequisites: gcloud logged in to the billing account's owner, project `hill-climb-sims` with billing linked, APIs enabled, the Dream key in `~/.dreamlabs/credentials` (setup pipes only `DREAM_API_KEY` to Secret Manager via stdin), and for API judges an `anthropic-api-key` secret uploaded by you.

```
bash cloud/setup.sh --dry-run
```

```
bash cloud/build.sh --tag cloud-v1 --dry-run
```

```
bash cloud/submit.sh --tag cloud-v1 --job hill-climb-pilot-v1 --pilot --parallelism 8
 ↪  --dry-run
```

After a successful pilot (check peak RSS against the 1,280 MiB request, camera freshness < 2 s, sim/wall rate, and that Dream accepts the concurrency — **it did not at ~28 sessions on 8 Nov**), collect:

```
python -m cloud.collect --prefix gs://hill-climb-sims-results/runs/hill-climb-pilot-v1
 ↪  --execute
```

Rules from GOAL.md: one change per arm, same seeds across arms, ≥ 20 valid matches per arm, accept a change only if the 95 % bootstrap CI lower bound beats the previous mean without more losses; log every arm in HILL_CLIMB_LOG.md; review live snapshots every 20 minutes; never let Batch re-run a paid match.

11.6 Run the harness live on the station

Start the station agent with the add-on (on the NUC, in tmux; docs/HOW_THE_HARNESS_WORKS.md, TELEOP_PLAN):

```
source ~/dream-env/bin/activate

PYTHONPATH=~/hill-climb-harness/station_addons dreamcli robot up --view --view-port 8093
```

With station-side hedged Dream calls (plans/bot_v1.yaml:17-18):

```
PAH_HEDGE_N=2 PAH_HEDGE_CUTOFF_S=1.5 PAH_HEDGE_MAX_CALLS=3
 ↪  PYTHONPATH=~/hill-climb-harness/station_addons dreamcli robot up --view --view-port
 ↪  8093
```

Check the add-on's light frames before anything moves:

```
curl -s -o /dev/null -w "%{http_code} %{size_download}\n"
 ↪  http://127.0.0.1:8091/frame/wrist_r.jpg?w=640
```

Dry run (reads status and frames, prints what it would POST):

```
python -m harness plans/bot_v1.yaml --dry-run --detect plans/table_colors.yaml
 ↪  --live-guard
```

The match command (keys injected by your secret manager, never on the command line; --live refuses without the live guard when the plan has a guard). This is the command in plans/bot_v1.yaml:24-26, with the project-specific secret-manager prefix left out:

```
python -m harness plans/bot_v1.yaml --live --no-operator --monitor-model
 ↪  google/gemini-3.1-flash-lite --detect plans/table_colors.yaml --live-guard
```

Before you do: main's plans/table_colors.yaml still has the phone-photo white range, which missed both held-white test frames in our check (§5.2.4). Use the station-retuned colours from fix/classifiers-8nov (or fix/guard-real-frames) and re-check on your own wrist frames:

```
git show origin/fix/classifiers-8nov:plans/table_colors.yaml >
 ↪  plans/table_colors_station.yaml

python -m harness.detect plans/table_colors_station.yaml frame.jpg --out boxed.jpg
```

Practice variants: --skip cable_ (stack + balls only), --skip cable_ --skip stack --skip tower (balls only), --var model=dream-1-ft:... (another checkpoint), python -m harness.panel (operator panel, practice only), --verifier monitor+manual (your y/n overrules the judge).

Read-only tools for the guard on live frames and recorded video (fix/guard-real-frames): tools/guard_watch.py, tools/guard_replay.py. LiveKit viewer for the referee (+10):

```
LIVEKIT_URL=wss://<your-project>.livekit.cloud LIVEKIT_ROOM=<room> python -m
 ↪  harness.livekit_view --port 8098
```

(with LIVEKIT_API_KEY / LIVEKIT_API_SECRET or LIVEKIT_TOKEN in the environment). DimOS skills (+5):

```
python -m integrations.dimos_skills plans/hill_climb_v4.yaml --mcp-port 9990
```

At the end:

```
dreamcli robot down
```

Logs are in `runs/<UTC timestamp>/`. Summarise them with `python -m harness.report runs/ --by-checkpoint` (`--last N` for the newest N); `tools/bot_report.py [--open]` builds an HTML page over the run folders hard-coded in its `RUN_DIRS`.

11.7 Collect teleop data (Metal)

Leader/follower on the station, inside tmux, in the kit's venv (follower_code README). The kit's venv needs Pinocchio first:

```
cd ~/physical-ai-hacks && uv pip install "pin>=3.1,<4" "cmeel-urdfdom>=4,<5"
 ↪  "cmeel-tinyxml2<11" numpy==2.2.6

.venv/bin/python ~/follower_code/metal/leader_collect.py --config configs/station.yaml
 ↪  --dry-run

.venv/bin/python ~/follower_code/metal/leader_collect.py --config configs/station.yaml
 ↪  --task "unplug the USB cable at the end of the hub nearest the right arm"
```

Quest teleop (gravity feed-forward on by default since e87a17a):

```
.venv/bin/python ~/follower_code/quest/quest_collect.py --config configs/station.yaml
 ↪  --dataset-dir ~/hackathon_data/my-task
```

After recording: split long takes, freeze the leader's wrist camera, add EEF columns, deep-check:

```
python tools/split_episodes.py --help

~/dream-env/bin/python tools/add_eef.py ~/hackathon_data/my-task --arm metal --replace

python tools/check_dataset.py ~/hackathon_data/my-task --videos
```

Check tracking per joint, and — new rule from §9 — **screen every folder for steady-state action − state offsets above ~1 cm** before training:

```
.venv/bin/python ~/follower_code/metal/tools/tracking_report.py ~/hackathon_data/my-task
```

Check that a plan's prompts all have demos:

```
python -m harness.tasks plans/hill_climb_v4.yaml ~/hackathon_data/unplug-v1
 ↪  ~/hackathon_data/relay-v1
```

11.8 Fine-tune

From TELEOP_PLAN.md. In a shell that holds the fine-tune account's key (`read -rs DREAM_API_KEY; export DREAM_API_KEY`):

```
tmux new -A -s finetune

~/dream-env/bin/python ~/hill-climb-harness/tools/finetune_watch.py start
 ↪  ~/hackathon_data/ft-final --name relay-final --suffix relay-final --max-requeues 1
 ↪  --i-accept-charges

cat ~/finetune_watch/STATUS.txt
```

Before using a new checkpoint: Dream's own loop/model check (moves the arms in small circles; 8 billed calls):

```
~/dream-env/bin/python -m dream.check --recording
 ↪  ~/hackathon_data/ft-final/quest-split-v4-final-eef -- ~/dream-env/bin/dreamcli robot
 ↪  run --checkpoint dream-1-ft:<account>-<suffix> --task "unplug the USB cable at the
 ↪  end of the hub nearest the right arm"
```

Then a 5 + 5 A/B on Stage 1 against base, alternating run by run:

```
python -m harness plans/bench_cable.yaml --live --verifier monitor+manual --runs-dir
 ↪  runs/bench --var model=dream-1-ft:<account>-<suffix>
```

```
python -m harness.report runs/bench --by-checkpoint
```

For a fine-tune, prompts go byte for byte (`prompt_engine.mode: verbatim`, `--no-rewrite`, no `tower_prompts`). To correct the Quest gravity sag, follow §9.6 (relabel actions as measured joints ~3 frames later, then `add_eef --replace`). Budget ~5 h and ~$100 for the 500-step minimum.

11.9 Add a classifier

1. Put the model in `harness/<name>.py` with a pure function `frames → result` and no network. Keep runtime deps to numpy/OpenCV (the cable-end model shows the pattern: ONNX via `cv2.dnn`, fixed batch).
2. Decide where it plugs in: **(a)** boxes on the monitor's frames (like `TableDetector.annotate_frames` and `CableEndClassifier`), **(b)** a fast guard (`runner.fast_guard` / `LiveGuard(judge=...)` must return `(True|False|None, why)`), **(c)** a decision feature (`decide.DoneWeights.w_*` / `state-pace` inputs, weight 0 until a sim sweep says otherwise), or **(d)** a gripper-style sensor note (`runner._grip_note`).
3. Add a CLI flag in `harness/cli.py` next to `--detect` and wire it in `main()`.
4. Tests: a unit test on committed frames (people-free crops only), and if it can stop the robot, an equivalence or regression test like `tests/test_guard_real_frames.py`. If it can change a guard verdict, add a stress scenario.
5. Evaluate on **station frames held out by run**, report precision/recall at the operating threshold, and say which frames were used (keep frames with people out of the repo).

11.10 Add a plan stage or prompt

A stage (see `plans/bot_v1.yaml` and `harness/plan.py` for every key):

```yaml
  - name: ball_extra                  # unique; prefixes matter: cable_,
↪ stack/tower/higher, ball_
    prompt: "With the right arm, pick up the orange golf ball on the right side of the
↪ table and drop it into the white basket."
    success: "Compared with the stage-start images, is there one more legal golf ball
↪ inside the bin?"
    confirm: 1                        # 2 for any pass bar
    confirm_check: "Count the legal balls in the bin at STAGE START and NOW. Is NOW
↪ higher, with no white or black ball in the bin?"
    timeout_s: 60
    max_attempts: 2
    on_fail: skip                     # never skip a pass bar: that is a DQ
    requires_stage: tower             # gates: balls only after the tower passed
```

Rules learned on the station: one short action naming one object by colour and position; no negations; no arm named for stacking; the safety sentence (`prompt_prefix: "{safe}"`) on every cable prompt, and only while it is true; never re-prompt while a gripper holds the target; lock a pass with `stop_on_pass` + `lock_prompt`; give pick-and-place steps at least 40–60 s.

A template in the prompt engine (`plans/engine_bot.yaml`):

```yaml
prompt_engine:
  templates:
    ball:
      text: "<arm> pick up <ball> <where> and drop it into the white basket"
      slots: {arm: arm, ball: ball, where: where}
      note: "one named ball near the right arm"
```

Then reference it with `template: ball` (and `variants: [ball_goal]` for second tries) in the stage. `load_plan` lints every vocabulary option at load and refuses the plan on a negation, an arm in an `arm: none` template, a `forbid` phrase or an unfilled slot. Run the plan tests and the stress sim after any change:

```
python -m pytest -q tests/test_match_plans.py tests/test_prompt_engine.py
python experiments/stress_match.py --plan plans/engine_bot.yaml --seeds 3 -j 4
```

12. Known issues, open TODOs, dead ends, and what we would do next

Back to the index

12.1 Known issues on main (acc1672), most serious first

#	Issue	Evidence	Fix
1	**The live guard's white range on main is the phone-photo range**; it missed both held-white-ball test frames that the station retune catches	§5.2.4 (our run); audit A07; `fix/guard-real-frames` dcd76c5	Port `plans/table_colors.yaml` and the `detect.py` changes from `fix/classifiers-8nov` (PR 7) to main; re-check on fresh wrist frames with a real held white ball
2	A missing or frozen wrist frame on the carrying arm is masked by a clear view on the other wrist (`TableDetector.guard` returns "none held")	audit A06	Require the carrying arm's wrist frame to be fresh; report blindness per camera
3	Blocking I/O can carry motion past the deadline (station client timeout 70 s, monitor transport 60–90 s, synchronous entry/confirm checks)	audit A09; `harness/station_client.py:31`	Bound every synchronous call by the time left; send `/stop` when the deadline passes during a call
4	A station restart after a fault keeps old done evidence (`_poll_health` → `_start_run`, not `_issue`, so `notify_prompt_change` and `last_done_t` are not reset)	audit A11; `runner.py:341`	Clear done state and bump the verifier generation on every restart
5	Correlated confirming looks: two in-flight calls on near-identical frames can satisfy `confirm: 2`; the 15 s sticky-done window can pass a re-placed tower	stress sweep (2 DQs); audit A03	Minimum gap between confirming verdicts; second look from a different model (as the bot line's Sonnet `confirm_check` does for unplug)
6	`tower_pass` in `hill_climb_v4.yaml` passes on one synchronous entry look (`skip_if_done: true`) without the 3 s dwell	audit A02 (critical); `plans/hill_climb_v4.yaml:188`	Drop `skip_if_done` there (done on `prompt-engine`'s relay plan); `bot_v1` is not affected
7	`test_bot_v1_plan` fails: the cable prompt contains "toss"	§13.1; 716c4d0 vs bec9975	Decide which is right (the only confirmed 13:57 unplug used "toss"; earlier "toss it" sent Dream to the balls) and align test and plan
8	`tests/test_hill_climb_scene.py` errors at collection without `dreamlabs`	§13.1	Wrap `find_spec` in `try/except ModuleNotFoundError` or use `pytest.importorskip("dream_examples.yam_mujo`
9	Main lacks the last bot fixes (`confirm_every_s` tower re-check, stacking option B, orange-cable prompt)	§2.4.2	Merge `claude/engine-bot-live` (6 commits)
10	Pull-force add-on fires on every right-gripper close, including empty closes at home	prompt-study replay	Gate to unplug steps or keep off
11	The rule joint observer's `limit_approach` fires ~90–100 times per joint-hour on real runs (shoulder_lift and elbow_flex rest at the 0° soft limit at home)	§5.5.2	Fix the margin or the home pose before enabling the rules (not on main anyway)
12	The subscription transport sends only `CLI_SYSTEM_PROMPT`, dropping the plan's system prompt	audit A21	Pass the plan's system prompt through `claude -p`

12.2 Open TODOs carried over (from `docs/LAST_NIGHT_2026-11-08.md`, `STATE_MACHINE.md`, the paper)

- Run full five-minute harness matches **on the station with per-stage logging**; there is still no real per-stage success rate.
- Compare the harness against plain Dream-1 on the station with the same prompt.
- A one-call Dream probe before every batch that stops on DA-1000.
- Fix the failing tests on `cloud-sims` (at 13:07 UTC 8 Nov: 407 passed, 4 failed, 5 skipped; see §13.1 for today's run).
- Test the exact match prompts for refusals; keep hands out of the camera view.
- Record more stack and golf-ball demos, and unplug demos from the home pose.
- Rerun cloud round 1 with fewer sessions and credit checked first; run the **real judge against the MuJoCo referee** on the rebuilt table to measure false and late done rates.
- Fine-tune on gravity-corrected data and compare with base on the same seeded scenes (needs a Metal sim or station slots).
- Build the USB port finder and the insert skill if plug-in comes back (`STATE_MACHINE.md`: "Not built: the harness insert skill, the real-table port and cable classifier slots, and a Claude filler prompt for plug-in").
- Decide whether pass-bar stages follow Darsh's "fail twice, move on" (they currently cannot without a DQ).
- Measure the block height: `tower_pass` depends on the 4-inch bar, and if two blocks already exceed it the third is wasted risk (audit A14).
- Benchmark the detector on the actual MacBook Air under sustained load.
- Re-tune and test `quest-continuity` on the NUC (`quest_collect.py --dry-run`, the 5 new tests).

12.3 Dead ends and things that did not work

Idea	What happened	Source
The fine-tune `relay-final`	worse than base on every task: aimed 5.5 cm high (Quest gravity sag), 0 cable grips in ~10 min, cold starts of 3+ min	§9
Monitor-written free-text rewrites	judge-written micro-rewrites unplugged nothing; 9 of 15 were sent mid-grasp and 5 dropped the plug; Flash-Lite rewrote 11 times in one run	§7.6; `bot_v1.yaml` notes
Long, coached prompts	a 574-character cable prompt unplugged nothing in 286 s; motion scripts for stacking stacked 0 of 10	§7.6
Naming an arm	0/4 in sim; "the right arm" sent the far arm for stacking on the station; left-arm prompts 0 of 4 on 8 Nov	§7.5.2
Two arms in one prompt	no grip (station), 2/4 single-goal successes (sim)	§7.5.2, §7.6
Watcher arm	depends on Dream holding one arm still on command, which it does not reliably do	`DECISION_MODEL.md` §2
Plug-in stages	base Dream-1 never seated a plug; dropped from the match plan on 7 Nov; plug-in prompts were refused in the sim	TELEOP_PLAN, §10.1
Blender synthetic data for the cable-end CNN	synthetic + real 92 % vs real-only 94 %, worse on photos	§5.3
`hybrid_v4_long.yaml` (whole-task prompts)	lost 34 of 185 sim matches to the illegal ball and ended early in 134	§7.2.2
Second unplug in the v4 plan	+6 points in the sim but two more illegal-ball losses where only the monitor guards	task_simulation README
Gravity scale alone to fix the fine-tune	no env-var-only setting reached grasp height in the replay	§9.5
Cloud round 1	killed by DA-1000 (out of credit) with ~28 sessions; 10 of 29 matches valid, arm B none	§7.7
~28 concurrent Dream sessions	the key was disabled twice	§10.1.2
Pull-force add-on	fired 70 times, mostly on empty or irrelevant closes	§3.6
A harness wait at timeout for an in-flight verdict	no difference (54.9 vs 54.8), dropped	task_simulation README

12.4 What we would do next, in order

1. **Make main safe to run as is**: port the station colour retune (issue 1), fix the guard's per-camera blindness (2), bound blocking calls by the deadline (3), clear done evidence on restart (4), merge `engine-bot-live` (9), and get the test suite green (7, 8).
2. **Get real per-stage numbers**: 10+ full matches on the station with the bot plan, logging each stage, plus a same-prompt harness-off control.
3. **Close the sim gap**: run the real judge (Flash-Lite + Sonnet confirm) against the MuJoCo referee on the rebuilt table; calibrate the stress sim's probabilities (unplug ~1/6 per try, put-down, stacking) from step 2; only then tune the decision model's weights (§4.6).
4. **Fine-tune again on corrected data** (§9.6), with unplug demos from home and an action − state screen in `check_dataset.py`; A/B against base on Stage 1 (5 + 5 runs).
5. **Serving loop**: wired network and light frames everywhere; measure the async share before and after; A/B the station tuning knobs (`PAH_FF_TAU_S=0.15`, `PAH_METAL_TICK_HZ=60`) one at a time.
6. **Local evidence first**: let the gripper-width rules, the detector and the cable-end CNN pass non-pass-bar stages (weights in `decide.py`), keeping the VLM for pass bars with a second-model confirm.

13. Verification log

Back to the index

What was run, re-derived or only read for this report (9 Nov 2026, Linux container, Python 3.13.18, numpy, Pillow, OpenCV 5.0.0, MuJoCo 4.0.0 with OSMesa, scipy, pyarrow, psutil; **no `dreamlabs`, no `i2rt`, no `livekit`, no `dimos`, no torch**). No Dream, Anthropic, OpenRouter or other paid API was called. No robot, station or cloud resource was touched.

13.1 Test suites

`python -m pytest -q --continue-on-collection-errors` in a clean worktree of each branch head (main also with `--ignore=tests/test_hill_climb_scene.py`).

Branch @ head	Passed	Failed	Skipped	Collection errors	Failures and why
main @ acc1672 (scene test ignored)	**334**	**1**	10	(1 if not ignored)	`test_grip_assist.py::test_bot_v1_plan`: the cable prompt contains "toss" (716c4d0) — **a real plan/test disagreement**
final-harness @ 777e389	303	0	7	1	—
start-pose-live-guard @ c2098dc	270	0	6	1	—
hedged-calls @ bf33014	348	0	7	1	—
mac-arm64 @ ad2a55f	354	0	7	1	—
cloud-sims @ d2203e0	392	5	27	1	5 × `test_async_pacing.py`: need Dream's SDK (environment)
fix/classifiers-8nov @ 3202983	407	5	27 (+1 xfail)	1	same 5 (environment)
joint-fault-encoder @ 6d8ea7e	398	5	28	1	same 5 (environment)
station-tuning @ 8a96aac	249	0	8	1	—
prompt-engine @ 3e7866c	275	0	7	1	—
claude/harness-stitch @ 28336b0	300	0	7	1	—
claude/engine-bot-live @ d7aac1e	316	1	7	1	`test_bot_v1_plan` ("toss"), as on main
relay-on-stitch @ f8771cd	320	1	7	1	`test_livekit_observer.py::test_minted_token_i` imports `livekit`, but only skips on a missing `jwt` (environment)

So the only **logic** failure on any branch we ran is `test_bot_v1_plan`, on main and `engine-bot-live`.

Every branch has the same collection error in `tests/test_hill_climb_scene.py`: `importlib.util.find_spec("dream_examples.yam_mujoco")` raises `ModuleNotFoundError` when the parent package is absent, instead of returning `None`, so the module-level skip never runs. The five `tests/test_async_pacing.py` failures on the cloud-sims family all end in `SystemExit: Cannot find Dream's yam_mujoco example`: they need Dream's SDK and fail instead of skipping (Codex audit A24). On `LAST_NIGHT_2026-11-08.md`'s machine (with the SDK) cloud-sims was 407 passed, 4 failed, 5 skipped at 13:07 UTC 8 Nov. The single real logic failure on main is `test_bot_v1_plan` ("toss" in the cable prompt, 716c4d0). Skips are optional dependencies (`pyarrow/dimos/jwt/livekit/dreamlabs`).

follower_code: `main` 1 failed (`tests/test_sim.py`: no `i2rt`), 2 collection errors (`metal/tests`, `quest/tests` need the hackathon kit); `quest-continuity` **7 passed** (the kit-free `quest/tests/test_continuity.py` 1 failed, 2 errors for the same reasons. The kit tests were reported passing on the NUC (10 Metal, 14 Quest) [D].

13.2 Simulations run

Run	Result
`experiments/stress_match.py --seeds 5 -j 8` on main @ acc1672	120 matches, mean 71.8 pts, 17 losses (4 DQ, 13 illegal ball) — §7.2.3
same on cloud-sims @ d2203e0	120 matches, mean 53.5 pts, 7 losses (4 DQ, 3 illegal ball)
`results/stress/final_20261108/analyze.py`	reproduces every row of the 185-match table (§7.2.2) from the committed `out/*.jsonl`
Guard check: `TableDetector.guard` on the six `tests/data/guard_real/` frames with main's and fix/classifiers-8nov's configs	main 0/2 held-white detected; retune 2/2; no false trips on 3 not-held frames (§5.2.4)
PDF build (pandoc 3.1.3 + XeLaTeX)	62 pages, no missing glyphs

Not re-run: MuJoCo matches with the live policy (needs a Dream key with credit), cloud batches (spend money), judge benchmarks (paid APIs), cable-end and joint-fault training (frames with people are not in the repos; joint traces were not copied), the Metal replay sim (needs the NUC's copied run files and Dream's driver).

13.3 Numbers checked

Claim (brief / paper / docs)	Check	Verdict
52 runs, 1,650 chunks; RTT median 1,116 ms, p10 800, p90 2,733, max 8,655	recomputed from `robots-inference.jsonl`	**verified**
67 % of chunks stop-and-wait (1,101/1,650); sync gap 3.54 s, async 1.55 s	recomputed	**verified** (66.7 %)
Serve 293 ms, network 40 ms	recomputed	**verified**
~650 ms fixed gateway overhead; sync above ~900 ms median RTT	fast-uplink chunks: 900 ms RTT − 293 ms serve ≈ 607 ms; 900 ms threshold from the station log quoted in `DECISION_MODEL.md`	**consistent** (threshold read from docs, not code)
Fault mix DA-4000 ×4, SVD ×3, buffer ×2, expired ×1 (digest) vs 4/4/4/4 (paper)	both logs	**both verified**: 7 Nov subset vs full log
Frame every 3.4 s (last-run.csv)	`last-run.csv` present; not recomputed	documented only [D]
Stress: 69.8 / 66.6 / 60.5; 28 illegal-ball losses for the fast judge without the guard	`analyze.py` on raw rows	**verified**; CI 65.3–74.4 not recomputed
37 scenarios	raw rows contain 37; committed code has 24	**partly**: 13 scenario definitions not committed
Before/after fixes table	raw `.txt` files	**verified**; the paper's "83–90" for corrupt frames should read 83, 94, 95, 95

Claim (brief / paper / docs)	Check	Verdict
Judge benchmark: Terra none 98/97 at 2.8 s, $0.047/match; low 100/100 at 3.7 s; Sonnet 100/100 at ~7 s; Haiku low 100/93 at 4.5 s	`bench_fair/*/summary.csv`	**verified**
Judge benchmark “30 configurations”	25 in `bench_fair` (one all-failed)	**not verified** (5 missing)
Haiku caught a held 8-ball 55 % on a 120-decision run	`bench_fair/TABLE.md`	**verified**
Detector 20/27 → 26/27 balls, 16/23 → 23/23 blocks, bin 1,286 → 0 of 2,010	labels and frames kept outside the repos	documented only [D]
Detector 76 ms on two wrists (236 ms before)	`HARNESS_SPEED_STATE_SPACE_2026-11-07.md` (report-inputs); mac-arm64 bench JSON gives per-frame ms on other hardware	documented only [D]
Cable-end USB-A recall 0.68 → 0.84; 891 + 478 = 1,369 crops; Blender 92 % vs 94 %	`CLASSIFIERS_8NOV.md`, commit ae30720; crops not in the repos	documented only [D]
Joint faults: d32×2 blocked 0.78 vs TCN 0.20 without torque	`sweep_real.json`	**verified** (timely recall; plain recall ≈ 1.0)
Joint faults on sim with torque: TCN 0.91 timely at 0.2 FA/joint-h; rules 0.50 at 197	`JOINT_FAULT_NET.md`; `joint_fault_search.json` not re-parsed	documented only [D]
MuJoCo v1: 14 of 25 with a cable out; 3 DQs, all hub-left	`cluster_mujoco/*/matches.jsonl`	**verified**
First unplug median 57 s (28–269 s); put-down 0/54; stack 0/25	event logs present; not recomputed	documented only [D]
Sim-time six matches 42, 42, 41, 15, 15, 15; relay order 12/12 and 8/12	no logs in any repo	**not verifiable** [U]
Unplug stage curve p_max 0.64, none before ~23 s, **median 56 s**	`stage_curves_sim.json`: p_max 0.64, t_min 22.84 s, τ_c 105.6 s → half-ceiling time ≈ 96 s	**inconsistent** with the JSON’s parametric fit; the 56 s is probably an empirical median not stored [U]
Shadow decide = runner on 5,000 random sequences; state-space on 3,000	test code read (`tests/test_decide.py:48`, `STATE_SPACE.md`); tests pass in the branch runs	**verified** (tests)
5.96 requests/s at 0.63 s median	`live/status.json` of one laptop match: 5.96 Hz, p50 0.625 s; its final `match.json`: 5.52 Hz	**verified** (live snapshot)
Cloud round 1: 10 of 28 valid, 0–42 pts 101 rollouts, 171 steps; 8 of 12 pulls kept the plug; 9 of 15 rewrites mid-grasp	29 `match.json`, 10 valid, points 0–42 `families_20261108.json` steps sum to 171; the rest from the README (raw segments not committed)	**verified** (29, not 28) **171 verified**, rest [D]
FT: 170 episodes = 73 Quest + 97 leader; 48,413 frames	TELEOP_PLAN folder table sums to 73 + 97	**verified** (sums); offsets [D]
FT offsets +0.078/+0.047/+0.043 rad, ~5.5 cm; 9 of 69 unplug demos near home	FT diagnosis (two identical copies)	documented only [D]
Metal driver bench RMS table	`STATION_TUNING.md`	documented only [D]
mac-arm64 guard 3.6× (x86), 6.0× (GHA M1)	x86 from doc; GHA from `bench_results/mac_bench_gha_macos15.json`	**GHA verified**, x86 [D]
Codex audit: 26 issues, 2 critical	`AUDIT_FINDINGS.md` headings A01–A26; A01, A02 critical	**verified**
Dream key disabled twice at ~28 concurrent sessions	one DA-1000 burst (12:04:48 UTC, 83 lines) is in the logs; “twice” from the digest	**partly**
Hold torque 2.0 N·m since 19:14 UTC 8 Nov	`nuc/README.md` on station-tuning	documented only [D]
`test_bot_v1_plan` fails on 716c4d0 (“toss”)	ran it	**verified**

13.4 `report-inputs` and the local-only material

The brief said to check for a `report-inputs` branch in task_simulation at the end. It **did not exist** when this work started (`git fetch --all` showed only `main` and `paper`) and **appeared later** (6cc3224, 9 Nov 07:09 UTC, darshp848). It was fetched and folded in:

report-inputs file	Status	Where it is used
`docs/HARNESS_DECISION_MODEL_2026-11-07.md`	**byte-identical** to `docs/DECISION_MODEL.md` on `final-harness`	§4
`docs/HARNESS_SPEED_STATE_SPACE_2026-11-07.md`	new to us; source of the 76 ms detector, 12.4 ms cable-end, 5–8 s per stage boundary, estimator proposal	§3.7, §5.0, §13.3
`docs/HARNESS_DIRECTIONS_2026-11-07.md`	new to us; seven directions from the literature (LIBERO-Plus start state, Code-as-Monitor, conformal calibration, STEP/SIMPLER trial counts, RaC, AHA)	§13.5
`docs/HARNESS_SIM_INSTRUCTIONS.md`	Windows sim instructions (dreamcli needs `fcntl`, so it does not run on Windows; `sim_station.py` does); 7 Nov test result 127 passed, 1 failed (a timing race in `test_v2.py`)	§11, §13.5
`docs/FT_DIAGNOSIS_2026-11-08.md`	identical to `ft-diagnosis` except four redactions (dataset owner, a name, image references)	§9
`docs/codex_cloud_report.md`	Codex's report on the GCP runner: 318 passed; smoke match 301 s wall, 137 CPU-s, 689 MiB peak; 1,000 matches ≈ $2.50 of Spot VM at an **assumed** $0.030/h, Dream usage extra	§6.5
`docs/codex_gcp_console_report.md`	budget $50 with 50/90/100 % alerts; **both CPU quota requests (→ 200) denied**, limits stayed 32 and 24; pilots `pilot2-*` ended CANCELLED with upload failures	§6.5, §12
`docs/codex_openrouter_report.md`	`bench_judges.py` + 32 tests; 354 passed; dry-run 108 trials over 18 configs before any paid call	§5.1.3
`codex_branch_patches/0001-0003`	9e97ba8 (match supervisor), 569607e (LiveKit observer), 189c39d (validate the decision tree on 300 s runs)	these three commits are **also on GitHub**, reachable from `codex/hill-climb-review-fixes` and `codex/dual-arm-experiment`

The same seven design docs are also on **task_simulation `main`** under `docs/notes_20261108/`, so the digest's "local only" list was partly out of date even before `report-inputs`.

Still **not on GitHub** in any form: `codex/hill-climb-five-minute-validation`, `codex/hill-climb-livekit`, `codex/hill-climb-match-supervisor` and `port-sim` as **branches** (their known commits are reachable as above); three commits of `stress-test-fixes`; `port-to-main`; `nuc-snapshot` and `ft-joint-offset` as branch names (their content is on `station-tuning`); the raw prompt-study segments; detector labels and station frames (they show people); the 13 extra stress scenarios; the sim-time batch logs; the GCS-only `joints.jsonl` traces; the 79 station rollouts and the 3 GB `nuc_data`; and any match-day results.

13.5 Extra material from the 7 Nov directions doc

`HARNESS_DIRECTIONS_2026-11-07.md` ranked seven directions given that the harness can only send prompts and start/stop/home/reset: (1) match the start state to the training data (LIBERO-Plus: initial state −59.9 points vs language −15.6) → built as the start-pose gate; (2) close the guard race with live frames (Code-as-Monitor) → built as the 10 Hz live guard; (3) geometric stage checks first, VLM as tie-breaker → designed as `decide.py` weights, not enabled; (4) calibrate the monitor's thresholds with conformal prediction on successful runs (FAIL-Detect, FIPER, SAFE) → not done; (5) enough trials (STEP: 30–50-point gaps need ~7–35 trials, a 9-point gap ~500) → the GOAL.md ≥ 20 paired matches rule; (6) record harness-triggered recovery + correction episodes for training (RaC, IntervenGen) → not done; (7) failure-reasoning models (AHA) → not worth it that weekend.

13.6 Branches found but not fully explained

Branch	What we could and could not establish
`assets`	One commit of PR 9 screenshots on an otherwise unrelated history (1 ahead / 123 behind); not opened here because screenshots may show people
`claude/engine-bot-local`	Same commit as `claude/engine-bot-live`; why a second name exists is not recorded
`kaushik/cloud-sims-bot-plan`, `claude/sim-hillclimb`	Pushed at 03:16–03:17 UTC 9 Nov, after the event; whether they were ever run is not recorded
`claude/bot-prompt-writer` tip (21bad3a)	`tools/full_run.sh` pushed after the event; usage unknown
`sim-and-move-fix`	Kaushik's separate MuJoCo station sim (`sim/`, 5.7k lines, calibration JSONs); never compared with `sim_station.py`
`hybrid-harness` tip (b5883e4)	One operator lesson added after the event; the "run 055635Z" it cites is not in the repos
Which config ran in the actual match	Not recorded anywhere: main@acc1672 or engine-bot-live@d7aac1e, which `table_colors.yaml`, base or fine-tune per stage, and any score